

Deep6 Engine and File Format Reference

Wizards & Warriors (Heuristic Park, 2000) - deep6.exe

OpenDeep6SDK v1.0.0, compiled 2026-10-08 from the SDK documentation.

Everything here was found by reverse engineering the GOG release of the game (build 1266995554): reading the executable with its Watcom symbols, running it and round-tripping every data file. It is meant to be enough to write your own tools for the game; the Python modules in `formats/` are working references for every format. No game data is included.

Contents

- [Chapter 1. The Deep6 engine: an overview for SDK authors](#) (`docs/engine.md`)
- [Chapter 2. Deep6 indoor level formats \(Wizards & Warriors, 2000\)](#) (`docs/formats/levels.md`)
- [Chapter 3. Outdoor terrain \("spoke"\) file formats](#) (`docs/formats/terrain.md`)
- [Chapter 4. Terrain walls, canopy, foliage and texture import](#) (`docs/formats/walls_textures.md`)
- [Chapter 5. Deep6 content / scripting data formats](#) (`docs/formats/data.md`)
- [Chapter 6. Deep6 content databases - record layouts](#) (`docs/formats/databases.md`)
- [Chapter 7. Deep6 3D models \(`models/\*\*/\*.mdl`\)](#) (`docs/formats/models.md`)
- [Chapter 8. Spells and visual effects](#) (`docs/formats/effects.md`)
- [Chapter 9. Sound, music, speech and the journal](#) (`docs/formats/audio.md`)
- [Chapter 10. NPC and guildmaster dialogue \(NPCDATA.PAK, GMDATA.PAK\)](#) (`docs/formats/npc.md`)
- [Chapter 11. Deep6 area transitions \("exits"\)](#) (`docs/formats/exits.md`)
- [Chapter 12. Level geometry: TrenchBroom -> Deep6](#) (`docs/geometry.md`)
- [Appendix A. deep6.exe modules](#) (`docs/exe_modules.md`)
- [Appendix B. In-game verification log](#) (`docs/verification.md`)

Chapter 1. The Deep6 engine: an overview for SDK authors

Source: docs/engine.md

Wizards & Warriors (Heuristic Park / Activision, 2000) runs on an in-house engine the developers called **Deep6** (deep6.exe, sources under D:\d6\work\). No SDK or editor was ever released. This chapter describes how the engine is put together: the executable, how the world is organised, how areas are loaded, how the three script machines work, how saves override map data, and which tables are compiled into the exe. The file formats themselves follow in their own chapters; this one says how they fit together and where to start if you want to write your own tools.

0. How this was found, and how sure it is

- **Static analysis** of deep6.exe (GOG build 1266995554). The exe carries Watcom debug information: 4,906 named functions and 3,137 named globals, each with its source module (no types, locals or line numbers). Names in this reference (LoadSpokeSegment_, _gSpokeNumber) are those symbols; addresses are virtual addresses in that exe (image base 0x400000).
- **Running the original code:** the OpenDWWandW loader maps the unmodified exe on Linux and replaces only the Windows APIs, so data changes can be checked in the real game logic. Memory reads (/proc/pid/mem) and the --start-at option of OpenDWWandWExpanded were used to test placements, exits and scripts.
- **Round trips:** every supported file is parsed and written back, and the result must be byte-identical to the shipped file (907 data files including the 728 models, the 66 levels, the 158 dialogue scripts).
- **Confidence marks** used in the chapters: **H** (high) read from the loader or consumer code and confirmed in the data; **M** (medium) from usage, consistent with the data; **L** (low) guessed or unused.
- docs/verification.md lists what was additionally checked by playing.

1. The executable

format	32-bit PE, Watcom C/C++ 10.x, image base 0x400000
code	section BEGTEXT at 0x410000, 0x1A5000 bytes
data	DGROUP at 0x5C0000: 0x33600 initialised, BSS to about 0x978E00
debug info	Watcom v3.0, 221,843 bytes appended to the exe
modules	392: 191 game C files, 19 game asm files, 2 C++ files (cheatkey.cpp, tracklen.cpp), the rest Watcom run-time (209 game modules have named symbols)
linked in	libpng 1.0.5, zlib, lJG libjpeg

Calling conventions (needed to read the code or call it):

- name_ (trailing underscore): Watcom register call. Arguments in EAX, EDX, EBX, ECX, then the stack; the callee pops; every register except EAX is preserved.
- _name@N: stdcall (WinMain, window procedure, callbacks).
- Varargs functions (sprintf_, fprintf_, LogFile_, open_) are cdecl.
- Some asm routines (_span, _tpoly, _mpoly, _zbuf) may use their own register sets; check each by hand.

Platform layer (what a port replaces): DirectDraw 4 + Direct3D 6 (two renderers: a 16-bit software renderer and a Direct3D one), DirectInput, DirectSound plus the Miles Sound System (MP3 music through Mp3dec.asi), Bink (two movies), Video for Windows (town hub AVIs), DirectPlay/Winsock (multiplayer), the registry and a CD check. The appendix *deep6.exe modules* lists the 209 game modules that have named symbols, with their function counts.

The game logic runs in 640x480; the 3D view is the rectangle (100, 0)-(540, 374), the side panels and message bar are 2D bitmaps.

2. The game folder

Files are opened **case-insensitively** (the data mixes TEMPLEA.BOL, templea.bsp, D6Boun04.dat), so tools must match names that way. The appendix at the end of this chapter lists every file kind with its chapter. Broadly:

- D6*.DAT - global databases (items, monsters, props, NPCs, treasure, shop stock) and per-spoke scripting tables (D6TRI6nn, D6BOUNnn, ...).
- EVENTS.DCL / EVENTS.COD - the event script (names and bytecode).
- NPCDATA.PAK / GMDATA.PAK - dialogue bytecode.
- D6STRING.DAT, TEXTPAK.000, pctalk/TALKPAK.nnn - text.
- Per indoor level <name>: .bsp .twd .lf .lfs .ls .lss .lgt .rgb .nvs .l2n .bol.
- Per terrain spoke nn: SPOKEnn.TMR .TOL .NAV .FOL (+ editor sources .LIT .PAR .tom the game never opens), tiles/80x/, maps/lm0b<nn>.lm.
- models/**/*.*.mdl (728 models), efxgfx/ (effects), emitters.dat (particles), itemicon/ (246 icons used).
- sounds/, Speech/, music/, movie/, townavi/.
- Runtime files written by the game: save/, D6SEGnn.GAM, D6ARCHIV.DAT, D6WORLD.DAT, ROSTER.DAT, JOURNAL.nnn, PCSNAP.nnn, maps/*.fog, maps/*.mrk, gameopt.dat, default.kbd, debug.log (and D6SMIT00.DAT, shop stock, which is data but rewritten in play).

3. The world

3.1 Towns and spokes

The world is three **town hubs** (2D screens: a panning AVI with clickable buildings) and thirteen **spokes** (3D areas, _gSpokeNumber 0..12).

spoke	kind	geometry	reached from
0	terrain	SPOKE00 (Valeia wilderness) + 14 BSPs	Valeia (2 gates), Ishad N'ha
1	dungeon	CRYPTA, CRYPTB	tcrypt in spoke 0
2	dungeon	TEMPLEB, TEMPLEA	ttemple, ttmpcave in spoke 0
3	terrain	spoke03 + BSPs	Ishad N'ha, Brimloch Roon
4	dungeon	SHURU	tshurub, tshuruc in spoke 3
5	dungeon	MINESA..D	tmines in spoke 3
6	dungeon	OGREA..C	togrea, togreb in spoke 3
7	dungeon	DRAGONA..C	tdragona, tdragonb in spoke 3
8	dungeon	LICHA, LICHB	tllich in spoke 11
9	dungeon	SUNKENA, SUNKENB	tsunkena, tsunkenb in spoke 11
10	dungeon	SHRINEA	tshrinea..c in spoke 11
11	terrain	spoke11 (desert) + BSPs	Brimloch Roon
12	dungeon	PYRAMA, PYRAMB	tpyram in spoke 11

The towns are 0 Valeia, 1 Ishad N'ha, 2 Brimloch Roon. The chain is Valeia - spoke 0 - Ishad N'ha - spoke 3 - Brimloch Roon - spoke 11; the dungeons hang off the terrain spokes. The game always starts in a town, so a spoke is entered either through a town gate or through the script op LOADSEGMENT. **Nothing in the data lists the spokes:** the spoke switch in LoadSpokeSegment_ and the gate table in GetEntryPosition_ are code. New spokes, routes and entry points therefore need engine changes.

3.2 Units and axes

- **World units:** x and z horizontal, **y up**, left-handed. One terrain tile is 1024 units.
- **BSP units** = world units / 16, relative to the BSP's origin (World2BspTrans_: bsp = (world - origin) * 0.0625). One tile = 64 BSP units. Map editors (TrenchBroom, Quake style) are z up; the SDK swaps y and z when converting.
- **Angles:** 1024 per turn. The sine/cosine tables are filled with $\sin(i * 6.28 / 1024) - 6.28$, not 2 pi; positions computed by the game (party placement) depend on that (checked against a saved position).
- **Terrain arrays** are row major, row = z, column = x.

3.3 BSP slots and origins

Every spoke has a list of loaded BSPs, _bspListBSP[]; scripts address them by **slot** (BSPNUM). Each BSP has an origin in world units (bsp+0x118EC).

- **Terrain spokes:** the .TOL 'B' records place the small entrance BSPs (tcrypt, tgatea, ...). The record gives the tile position, the slot byte and signed tile offsets; origin = position - offsets x 1024.
- **Dungeon spokes:** slot 0 is a null BSP, the dungeon levels are slots 1..n, loaded with hard-coded Terrain_BSPLoad_ calls (terrain, name, bx, by, bw, bh, offx, offy, offz, p10, p11, defer); origin = ((bx-offx)*1024, -offy*1024, (by-offz)*1024). The SDK carries the table (d6level.SPOKE_TABLE).
- When the spoke changes, LOADSEGMENT keeps the party's position relative to the BSP it stands in and re-bases it onto the destination BSP's origin (subtract the old origin, add the new one). A dungeon and its terrain entrance are built with matching local geometry; their world origins may be equal (tcrypt and CRYPTA) or not (tshurub and SHURU).
- "Which BSP am I in" (InBSPArea_) uses each BSP's tile rectangle: on terrain through a leaf tree built by TerBSP_Calculate_, in dungeons by the first rectangle containing the tile.

3.4 Id spaces

id	meaning
object id < 100000	1-based record of the spoke's .TOL
object id n*100000 + k	record k of BSP slot n's .BOL (n = 16 means slot 0)
door / entity id	bsp*100000 + d6entid of a brush entity (slot 0 is 0 here)
nav id	the .nvs u16 id (< 20000), mapped per BSP to an index
table records	1-based record numbers of D6TRIG/BOUN/SWIT/SPEC/TRAPnn
world state	_WState[spoke*256 + n], one byte each, 256 per spoke
quest flags	256 per PC (qflag), set by NPC scripts and some events

Record numbers are ids throughout: deleting a record moves every later id, so editors blank records instead (d6edit does).

4. Loading a spoke

LoadSpokeSegment_ (d6spoke.c) for a terrain spoke, in order:

1. Sky and tree backdrops (SKYDROP.M2K, TREEDROP/TTOPDROP, palm versions for spoke 11, sun and moon maps), syswat.twd (water).
2. SPOKEnn.TMR - terrain tiles, heights, textures, walls.
3. SPOKEnn.NAV - terrain nav points.
4. SPOKEnn.TOL 'B' records - the BSPs (each loads its level files).
5. D6LINKnn.DAT - joins terrain and BSP nav graphs.
6. SPOKEnn.FOL - foliage.
7. D6SEGnn.GAM - **the saved state of this spoke**; only if it does not exist are the objects read from SPOKEnn.TOL ('I', 'M', 'P' records) and each BSP's .BOL.
8. maps/lm0b<nn>.lm (+ .fog, .mrk) - automap.

A dungeon spoke loads a null terrain, its BSP list, D6LINKnn.DAT and the same step 7. In both cases the scripting tables (D6TRIGnn,

D6B0UNnn, D6SWITnn, D6SPECnn, D6TRAPnn) are loaded by InitEvents_ / InitSpokeTraps_.

Consequence for tools: once a spoke has been visited, D6SEGnn.GAM holds its objects and the placement files are ignored. An editor must move those files (and the archive of a save) out of the way, or the player will not see the edits.

5. Saves and runtime state

- save/ - the save slots (game00.sav ...). Their header resembles D6ARCHIV.DAT (version 0x140) but was not decoded; the PC position is stored twice (x, y, z floats; the second copy has y + 832).
- D6ARCHIV.DAT - archive of the whole runtime state (header of 173 u32: version, flags, offsets and sizes of ROSTER, D6WORLD, each D6SEGnn.GAM, the journals and 24 shop files). Decoded for listing and extraction (data.md section 7).
- D6WORLD.DAT - world states; ROSTER.DAT - the party; D6SEGnn.GAM - per-spoke objects (positions, health, opened chests).
- Guildmaster flags are **not** saved: they reset on every visit.

6. Scripting: three machines

6.1 Triggers and events (events.c)

The world reacts through five per-spoke tables and one global script:

- **Bound areas** (D6B0UNnn, boxes) test who is inside (any PC, the whole party, a monster, an object) and set a world state or fire a trigger.
- **Switches** (D6SWITnn) are clickable objects or brush entities; they can be linked in rings and fire triggers.
- **Specials** (D6SPECnn) and **traps** (D6TRAPnn: locks, traps and treasure of containers).
- **Triggers** (D6TRIGnn, 0xB8 bytes) name an event of EVENTS.DCL, carry 30 parameter slots (shipped events use up to 14) and the world state they watch.
- **EVENTS.COD** holds 115 events as bytecode: u16 opcode (bit 15 = queued op that runs over time), i16 n, n x i16 parameter indices, and a u32 target for jumps and queued ops. 77 opcodes (0x01-0x56): doors, states, switches, items, monsters, messages, sounds, teleports, spoke changes (LOADSEGMENT 0x2A, GOTOBSP 0x21, ENTERTOWN 0x4F, ENDGAME 0x54).

Exits between areas are such events; exits.md lists all of them and the rules the engine uses to place the party (re-basing on BSP origins, a 256-unit y shift for spoke 8, the party ring search).

6.2 NPC dialogue (npc.c)

NPCDATA.PAK holds one bytecode program per NPC id (137 used of 160), with a string table and keywords. Execution always starts at offset 0; the script registers handlers (attacked, talk, item given, ...) with ONACT. SAY, REPLY, WAIT and similar ops yield until the player has read or answered. State: 32 flags per NPC, a register and an attitude per PC and NPC, 256 quest flags per PC, and the world states.

6.3 Guildmasters (guild.c)

GMDATA.PAK holds 32 programs (21 used), one per guildmaster; the town buildings pick theirs from tables in the exe (_gHallGM, _gSmitGM, ...). Only the building's buttons drive them (ACTBUTTON); flags reset per visit; EXIT (0x26) leaves the building. The response table in the PAK is never used by the game.

7. Rendering

- **BSP levels** are Quake-1 derived: .bsp version 28 with 15 lumps (lump 0 entities with d6* keys, an empty texture lump), mirrored axes, 68-byte models, 52-byte texinfo, u16 edges, and a lighting lump of one u16 per surfedge. Textures come from the level's .twd (128x128 8-bit with 16-bit shaded palettes). Lightmaps are separate files: .lfl/.ls (Direct3D, RGB565) and .lfs/.lss (software, 8-bit). Coloured point lights .lgt/.rgb light the models. A face is drawn only if its last vertex is a corner.
- **Terrain** is a tile grid (.TMR): heights at tile corners, a texture per tile from tiles/80x/ (transition tiles between terrain types), baked light, water per tile, and walls (tree walls, castle walls, canopy) built from per-tile flags at draw time.
- **Shaded palettes** (.p16, model palettes): 32 shade rows of 256 16-bit colours, row 0 black, rising with the row (the exact scale differs between terrain and models, see their chapters).
- **Models** (.mdl versions 9 and 10): parts, morph-target frames stored as packed vertex paths, an animation table of 256 entries, attachment points (weapons, helm), up to 4 levels of detail. The models/pc/*.seq files are build-time frame lists for pcreduce.exe; the game never opens them.
- **Effects:** particle emitters (emitters.dat), animated textures (.ant), glows (.alf), sprites, and a spell table in the exe.
- **Automap:** pre-drawn pages per spoke (maps/lm0b<nn>.lm) plus the fog-of-war and markers the game writes.

8. Tables compiled into deep6.exe

An SDK cannot get everything from data files. These live in the exe and are read (and in a few cases patched) by the SDK:

table	where	SDK
spoke -> BSP list and placement	LoadSpokeSegment_code	d6level.SPOKE_TABLE (read)
town gate arrival points	GetEntryPosition_code	d6terrain.ENTRY_POINTS (read)
model file names (items, monsters, props)	mdldata.c tables, _ItemMDLData 0x5DB3B8, _PropMDLData 0x5E27D5	read; d6edit can rename a slot
spell table (105 spells)	deep6.exe data	read and patched (effects.md)
item icon table (246 entries)	exe	read

guildmaster per building	_gHallGM, _gSmitGM, ...	read
record limits (999 items/monsters, 255 props, ...)	loaders	documented (databases.md 9.1)
sin/cos tables	filled at start-up (6.28 / 1024)	reproduced

9. Writing your own SDK: a suggested route

1. **Start with the containers.** Most files are simple record arrays: record 0 is a count header, record i lives at $i * \text{size}$. Read, write, and compare bytes with the original before decoding fields; a byte-exact round trip is the best early test.
2. **Keep unknown bytes.** Several structures have fields nobody has explained; a tool that carries them through unchanged stays safe.
3. **Get the coordinates right early.** World y is up and left-handed, BSP files are world/16 relative to an origin that comes partly from the exe (section 3.3); terrain BSP origins come from the .T0L.
4. **Treat record numbers as ids.** Blank records instead of deleting; append new ones.
5. **Mind the saves.** D6SEGnn.GAM and D6ARCHIV.DAT override map data (section 4).
6. **Geometry needs a compiler.** A changed BSP needs new tree, collision hulls, PVS, lightmaps and nav data. Standard Quake tools (qbsp/vis) work with the Deep6 collision hulls and a converter to version 28 (geometry.md).
7. **Test in the game.** The original exe running under the OpenDWWandW loader, with a way to start at any position (--start-at in OpenDWWandWExpanded), turns most questions into a quick experiment.

The SDK's Python modules in `formats/` are working references for every format in this document; each has a `--selftest` that round-trips the shipped files.

10. What is still unknown

Collected from the chapters (details there):

- The .ldl files (launcher strings, XOR-encrypted, key not recovered; the game does not read them).
- Save slot files (save/game*.sav): header not decoded.
- Fonts (.FNT, Font_Load_font.c) and pointer files (.ptr, text, loaded by LoadPointers_Imouse.c): loaders known, layouts not decoded.
- A few database fields (databases.md section 11), terrain header floats and some tile bytes (terrain.md), two NPC opcodes (npc.md), part header words in models (models.md section 5).
- Effects hard-coded per spell id (damage, visuals and sounds are code, not data).

Appendix: file inventory of the game folder

files	count	what	chapter
deep6.exe	1	the game	this chapter
D6MONS.DAT D6ITEM.DAT D6PROP.DAT D6NPC.DAT D6TREAS.DAT D6TRLIST.DAT D6HELM.DAT D6MONSND.DAT	8	global databases	databases, data
D6SMIT00.DAT	1	shop stock (the game rewrites it; the other building files - MAGE, PAWN, DOJO, YARD, TMPL, HALL, TVRN, 00..02 - only exist inside save archives)	data, databases
D6TRIGnn D6BOUNnn D6SWITnn D6SPECnn D6TRAPnn D6LINKnn .DAT	6 x 13	per-spoke scripting tables	data
ENTRssnn.DAT	17	named entry points (text)	data, exits
EVENTS.DCL, EVENTS.COD	2	event script	data
NPCDATA.PAK, GMDATA.PAK	2	dialogue bytecode	npc
D6STRING.DAT, TEXTPAK.000	2	strings, messages	data
pctalk/TALKPAK.000-009	10	PC remarks ("On my way...") per voice; same layout as TEXTPAK (u32 n, n x {i32 id, u32 offset, u32 size}, NUL strings) (H for layout)	this appendix
*.bsp .twd .lf .lfs .ls .lss .lgt .rgb .nvs .l2n	66 each	indoor levels	levels
*.B0L	53	objects per level	levels, data
.tc .pt1 .pt2	6	tool output, not read	levels
SPOKEnn.TMR .T0L .NAV .F0L	3 each	terrain spokes	terrain, data
SPOKEnn.LIT .PAR, spokenn.tom, tilebmp.lst		editor sources, never opened by the game	terrain
tiles/80x/		terrain tile textures and palettes	terrain, walls_textures
skydrop.m2k, *DROP.bmp/.p16, SUNMAP, MOONMAP		sky and tree backdrops	terrain
maps/lm0b<nn>.lm, .fog, .mrk	13 + runtime	automap	terrain
models/**/*,mdl	728	models	models
models/pc/*,seq	33	build-time frame lists for pcreduce.exe, not read by the game	models
models/*,lst,models/monster/model.lst	6	model lists (only chardemo reads model.lst)	data, models
models/monster/*,bsp	6	catapult, minecart, warship, raft, horse, hydra: not examined	-
emitters.dat,efxgfx/*,ant .alf .p16 .bmp, gfx/		particles, animated textures, glows	effects
itemicon/, I99Icon/, dragicon/, roleicon/, portrait/		interface bitmaps (item icons: 246 in the exe table)	databases
D6FNT*.FNT + D6FNT*.P16	51 + 33	fonts and their palettes (Font_Load_, Font_LoadPal16_)	not decoded

palmdrop palmsand ptopdrop treedirt treedrop ttopdrop .p16	6	backdrop palettes	terrain
*.ptr + bitmaps	32	mouse pointers: text with the bitmap name, sizes, hot spot and per-frame "index time" lines (M)	not decoded
DEEP6.PAL	1	8960 bytes = 768-byte RGB palette + 8192 bytes (32 x 256) (M)	not decoded
MONSOUND.DAT, sounds/		sound effect table and wavs	audio
Speech/*/*.wav	5954	spoken lines: NPCs, narrator (479 in 000-1611WAV), Gareth (27 in k21)	audio
music/*.mp3	8	music (Miles MP3)	audio
speech.tag, music/MUSIC.TAG, Speech/SPEECH.DIR	3	small text tag files (CD-era data location markers) (L)	-
JOURNAL.nnn		journal pages (runtime)	audio
townavi/townhubN.avi, townmskN.avi		town hub pans (MS Video 1) and click masks (uncompressed DIB)	terrain
Town000.hub	1	not opened by deep6.exe (no file name string)	-
movie/*.bik	2	Bink movies	-
*.ldl	4	launcher strings, encrypted, not read by the game	data
D6ARCHIV.DAT, D6WORLD.DAT, ROSTER.DAT, D6SEGnn.GAM, PCSNAP.nnn, save/		runtime state (section 5)	data
gameopt.dat, *.kbd		options, key bindings (runtime)	-

Chapter 2. Deep6 indoor level formats (Wizards & Warriors, 2000)

Source: docs/formats/levels.md

Status: reverse engineered from the Ghidra decompile of deep6.exe (module names below are the original source modules) and checked against all retail data. Reader/writer: formats/d6level.py. Running python3 formats/d6level.py --selftest <gamedir> parses and rewrites every level file and confirms each one is byte-identical (736/736 files: 66 levels plus syswat.twd, TOL, D6LINK, .tc, .pt*). Visual check: formats/render_level.py <gamedir> <level>.

All values are little endian. Confidence levels: **H** = read directly from loader code and confirmed by data, **M** = inferred from usage or data statistics, **L** = guess.

1. Overview and coordinate systems

An indoor level <name> is a set of files sharing one base name. They are opened case-insensitively (the data mixes TEMPLEA.B0L, templea.bsp and so on).

ext	content	loader (module)	needed
.bsp	geometry, BSP tree, collision, entities	CreateBSPFile_ / CompleteBSPFile_ (bspfile.c)	yes
.twd	textures + 16-bit shaded palettes	Texlist_LoadBSPWadfile_ (texlist.c)	yes (fatal error if missing)
.ls / .lf	hardware lightmaps (RGB565) + per-face info	BSP_LoadSurfaceLight_ (bspfile.c)	optional
.lss / .lfs	software lightmaps (8-bit) + per-face info	BSP_LoadSWSurfLight_ (bspfile.c)	optional
.lgt / .rgb / (.int)	coloured point lights	Load_BSP_ColoredLights_ (cardlite.c)	optional
.nvs / .l2n	AI nav graph, leaf->nav lookup	LoadNavSystem_ (nav_sys.c)	optional
.bol	props, monsters and items placed in the BSP	LoadBspObjects_ (scenload.c)	optional
.lt	per-surfedge light bytes (no retail files)	LoadLightFile_ (bspfile.c)	unused
.tc, .pt1, .pt2	text tool output	not read by the exe	-

Coordinates:

- BSP units are Quake-like. **Y is up** (nav points, door axes and floors all confirm this). The renders treat X/Z as the ground plane.
- World units = BSP units * 16 (World2BspTrans_: bsp = (world - origin) * 0.0625).
- One terrain tile = 1024 world units = 64 BSP units. A 128x128 texture at the usual texinfo scale of 0.5 covers exactly one tile.
- Each loaded BSP has an origin in world space (bsp+0x118ec). See section 12.

2. .bsp (H)

This is the Quake 1 BSP layout with version = 28 and the usual 15 lumps: a header of int32 version, then 15 x {int32 ofs, int32 len}, 124 bytes in total. The compiler wrote the lumps in qbsp order (1,10,3,5,6,7,9,11,13,12,14,8,4,0), starts each lump on a 4-byte boundary with zero padding, and pads the file end to 4 bytes. Lump 2 (miptex) is always empty with ofs=0; textures live in .twd. Only lumps 0, 1, 5, 7, 9, 10, 14 (plus vertices) are read by CreateBSPFile_. The rest (vertices, texinfo, faces, marksurfaces, surfedges, edges, vis, lighting) are read later by CompleteBSPFile_, which runs when the player comes near (UpdateBSPFileStates_). RevertBSPFile_ unloads them again.

#	lump	record	fields
0	entities	text	Quake { "key" "value" } blocks, \n line ends, trailing NUL (H)
1	planes	20	float normal[3], float dist, int32 type (0..2 axial X/Y/Z, 3..5 major axis) (H)
2	textures	-	always empty (H)
3	vertices	12	float x,y,z (H)
4	visibility	bytes	Quake RLE PVS, indexed by leaf.visofs (H by Quake analogy; M)
5	nodes	24	int32 planenum; int16 front, back (<0: leaf -(n+1)); int16 mins[3], maxs[3]; u16 firstface, numfaces (H, child order confirmed in FindCurrentNavPt_r_)
6	texinfo	52	float s[4], t[4]; float origin[3]; int32 flags; int32 runtime (see below) (H)
7	faces	20	int16 planenum, side; int32 firstedge; int16 numedges, texinfo; u8 styles[4] (always 255); int32 lightofs (always -1) (H)
8	lighting	2/surfedge	u16 per-vertex (per surfedge) light; g_fullbright if smaller than numsurfedges; overwritten by Bsp_SetBorderLights_ / .lt (M)
9	clipnodes	8	int32 planenum, int16 children[2] (H, Quake hulls 1 and 2; sizes measured from the data: hull 1 x/z +-32, y -2..+4; hull 2 x/z +-64, y -2..+4; the exe's _cliphull_size table says (-32,-4,-32)-(-32,2,32) and (-32,-16,-32)-(-32,96,32) but the compiled clip planes follow the measured sizes)
10	leafs	28	int32 contents, int32 visofs, int16 mins[3], maxs[3], u16 firstmarksurface, nummarksurfaces, u8 ambient[4] (H; ambient meaning L)
11	marksurfaces	2	u16 face index (H)
12	edges	4	u16 v0, v1 (H)
13	surfedges	4	int32 (negative = reversed edge) (H)
14	models	68	float mins[3], maxs[3], origin[3]; int32 headnode[4], visleafs, firstface, numfaces; int32 entity (runtime: index into the runtime entity table, written by SetRuntimeEntities_ ; file holds 0,1,2...) (H)

Leaf contents (H, from InBspWater_): -1 empty, -2 solid, -3 water, -5 lava (-4 slime and -6 sky are Quake values not seen in use).

Texinfo (bsp_3DCard_PolyDraw_, d_span.c, Texlist_AssignFaceTex_):

- Texture coordinate (H): $u = ((P - \text{origin}) \cdot s.\text{xyz} + s.w * 0.25) / 32$ gives normalised UV (1.0 = one texture repeat). In texels: $u_tex = 4 * (P - \text{origin}).s + s.w$. Unlike Quake, Deep6 stores a texture origin point per texinfo. Confirmed visually: floors tile seamlessly.
- flags (int32):
 - 1: no texture, the face is not drawn (H)
 - bits 0-11: index into the level's .twd texture list (H)
 - 0x2000: lava/translucent pass (M)
 - 0x8000: water pass, drawn with syswat.twd (H for the bit; the mapping to the curwater/curlava texture is M)
 - 0x20000: drawn as a terrain polygon when horizontal (M)
 - 0x1000 and 0x10000 are cleared on load (runtime use)
- runtime: 0 in the files; used at load time as an "already assigned" flag.

Entities (H, SetRuntimeEntities_): worldspawn, light (d6type 3, origin, d6light, d6entid; these duplicate .lgt), and func_door (d6type 1 = slide or 2 = rotate). A door has model "*/n", d6axis (0/1/2), d6maxmove (slide distance in BSP units, signed), d6rotx/y/z, d6flags, d6entid (id that events use to address the entity, via the table at bsp+0x5c90), and an optional d6string (name shown to the player, max 15 chars). Only entities with model become runtime entities. Engine limits: 9999 nodes, 19999 faces.

3. .twd texture wad (H)

```
[int32 -666]          present => 4 mip levels, absent => 1 (all retail files have it)
int32 npal, int32 ntex
npal x u16[32][256]  0x4000 bytes per palette: 32 shade rows x 256 RGB565 colours,
                      row 31 = full brightness, row 0 = black (M: linear ramp observed)

ntex x {
  char  name[64]      NUL padded
  int16 palette        index into this file's palettes
  int32 width, height  always 128x128 (the texture cache assumes 0x80>>mip)
  u8    mip0[w*h], mip1[w/2*h/2], mip2, mip3    8-bit palette indices
}
```

At load time palettes and textures are merged into a master table shared between BSPs. Textures are matched on their 16x16 mip; texinfo.flags & 0xfff is remapped to the slot. syswat.twd holds the shared water and lava textures. LoadTexWad_ uses the same format.

4. .lf / .lfs per-face lightmap info (H)

There is one 24-byte record per BSP face (record count = numfaces). Six int32 fields: smin, tmin, width, height, count(=w*h), offset. For .lf the offset is a byte offset into the .ls blob, with 2 bytes per texel. For .lfs it is the index of the texel byte in the .lss blob. The two files have different luxel grids (the hardware path uses about 9x9 where the software path uses 8x8, for example). The engine converts these into 0x34-byte runtime surface records. The software path picks a mip level from max(w,h). Hardware lightmaps (.lf, H, checked against every face of the retail data and in the game with compiled levels):

- Axis pair by plane type (MakeLightVecs_, table _g_tvecs): type 0/3 (x major) s = z, t = y; type 1/4 (y major) s = x, t = z; type 2/5 (z major) s = x, t = y. A type 4 plane with |ny| < 0.95 uses the type 3 pair if |nx| > |nz|, else the type 5 pair.
- smin = (floor(min s / 32) - 1) * 32, width = ceil(max s / 32) - floor(min s / 32) + 3 (one luxel of border on each side), same for t.
- The card path (bsp_3DCard_PolyDraw_) samples the lightmap at ((P.s - smin) / 32) / 16: the lightmap is put into a 16x16 RGB565 texture (CC_SetLMapCache_), so luxel i is centred at smin + 32 (i + 0.5). A face whose lightmap is wider or taller than 16 luxels is **not drawn**.
- The texels modulate the texture (31/63/31 = full brightness).
- The software files (.lfs/.lss) use another grid (L); the card renderer does not read them.

Without .lf/.ls the card renderer crashes (bsp+0x5c is NULL): a level must have hardware lightmaps.

5. .ls / .lss lightmap texels (H)

int32 nbytes, then the texel blob. .ls holds u16 RGB565 texels, adjusted on load by FuckLightData_ (brightness offset) and converted to 555 if needed. .lss holds u8 intensities 0..31 that index the palette shade rows. Both files must be present together with their .lf/.lfs, or lighting is disabled.

6. .lgt / .rgb / .int coloured lights (H)

- .lgt: N x {float x,y,z (BSP units); float intensity}. N = filesize/16. The loader multiplies intensity by 16. The positions match the light entities.
- .rgb: N x uint32 colour: R = byte 0, G = byte 1, B = byte 2, byte 3 unused. If the file is missing every light is white.
- .int: N x int32 (purpose unknown, L). No retail files; missing means 0. Each light becomes a LightObj at Bsp2WorldTrans(pos).

7. .nvs nav points (H for layout, M for fields)

```
int32 magic (-666), int32 unknown (varies 1..1375; probably a next-id counter, L), int32 count
count x 32 bytes:
  float x,y,z      BSP units; converted to world on load (H)
  float radius?    nearly always 0 (one file has 768.0) (L)
  int16 link[6]    0-based neighbour index, -1 = none; +1 on load because runtime is 1-based (H)
  u16  flags       0x10 => also sets 0x20; 0x02 lava and 0x04 water are set at runtime;
                  0x01, 0x08, 0x20 seen in data (meaning L)
  u16  nav_id      unique id < 20000, used by D6LINK files and events (H)
```

8. .l2n leaf to nav (H)

One int16 per BSP leaf, giving the 0-based nav index (-1 = none; +1 on load). Used by FindCurrentNavPt_. The entry count equals numleafs.

9. .bol object list (H for layout, M for some fields)

These are 64-byte records. TOL/FOL files use the same layout.

- Record 0 is the header: int32 total record count (including the header) followed by 60 zero bytes.
- Each object record:

off	type	meaning
0	char	'P' prop, 'M' monster, 'I' item; 0 = deleted slot (still counted for object tags) (H)
1	char[3]	ASCII decimal record number in D6PROP.DAT / D6MONS.DAT / D6ITEM.DAT (H)
4	float[3]	position, world units relative to the BSP origin (render confirms /16 = BSP units) (H)
16	float[3]	rotation x,y,z (mostly y; values such as 256, 512, -192; unit L) (M)
28	4 bytes	zero
32	u8	prop: mine-cart group id (spoke 5 only); monster: extra parameter passed to MonsterEntry_ (M)
33,34	u8	unused in .bol
35	u8	prop: non-zero = starts switched off/emitter off (M)
36	28 bytes	zero

Object tags are numbered gBspObjFirst[bsp] + record index. Events refer to objects as bsp*100000 + index (GetObjNum_).

10. .tc, .pt1, .pt2 (H: not used by the game)

- .tc: CRLF text "Texture Usage Report For " followed by lines of the form NAME: %-37s COUNT: %d.
- .pt1/.pt2: CRLF text with one %f %f %f point per line, in BSP-like coordinates. They look like recorded paths. There are no references in the exe.

11. How levels are listed and connected

There is no data file that lists the levels. The world is split into 13 "spokes" (0..12). LoadSpokeSegment_ (d6spoke.c) switches on gSpokeNumber:

- Terrain spokes 0, 3, 11** load SPOKExx.TMR (terrain), SPOKExx.NAV and SPOKExx.TOL. The TOL 'B' records place the small "t*" BSPs (tgatea, tcrypt, ttemple, tshurua, ...) into the terrain (LoadTerrainBsp_). A 'B' record has: pos (x, 0, z) = tile*1024; float width at +16 and height at +24 (tiles); u8 slot at +32; int8 offx, offy, offz at +33..35; name at +48. BSP index = slot (0-based).
- Dungeon spokes** use a null terrain plus a hard coded Terrain_BSPLoad_list (index 0 = null BSP, the dungeons are 1..n): 1 CRYPTA, CRYPTB; 2 TEMPLEB, TEMPLEA; 4 SHURU; 5 MINESA-D; 6 OGRE-A-C; 7 DRAGONA-C; 8 LICHA, LICHB; 9 SUNKENA, SUNKENB; 10 SHRINEA; 12 PYRAMA, PYRAMB. The parameters are in d6level.SPOKE_TABLE (M: decoded from Watcom register arguments). Terrain_BSPLoad_(name, bx, by, bw, bh, offx, offy, offz, p10, p11) sets the origin to ((bx-offx)*1024, -offy*1024, (by-offz)*1024) world units and the tile rectangle to (bx+p10-offx, by+p11-offz, bw, bh). The rectangle is used for streaming (CompleteBSPFile_ within range) and for border lights. Example: the overworld tcrypt and spoke-1 CRYPTA share the same origin.
- Objects: terrain spokes use TOL plus each BSP's .bol (LoadAllBspObjects_). Dungeon spokes load the listed .bol files with BSP index 1..n. The debug viewer FetchObjects_ (mapobj.c) holds the same list of names. SPOKExx.FOL holds static foliage ('F' records).
- Connections inside a spoke:** D6LINKxx.DAT, which has an int32 count, 4 unused bytes, then count x {int16 nav_a, nav_b, bsp_a, bsp_b}. bsp = -1 means a terrain nav point (TOL 'N' id). This joins the nav graphs of terrain and BSPs (and BSP to BSP, for example CRYPTA nav 80 <-> CRYPTB nav 80). python3 d6level.py --spokes <gamedir> prints everything.
- Travel between spokes** (dungeon entrances and exits) is not in these files. It is in the per-spoke event/trigger data (D6TrigXX/D6SwitXX/D6SEG). See exits.md and formats/d6exits.py: LOADSEGMENT re-bases the party's BSP-local position onto a BSP slot of the new spoke, and GOTOBSP does the same between BSPs of one spoke.

12. Notes for an editor

- Everything round-trips, and all record types are exposed as namedtuples or arrays with named fields.
- Faces (H): the engine draws a face only if its **last** vertex is a corner (SetWorldCornerFlags_ sets a bit per non-collinear vertex, bsp_3Dcard_PolyDraw_ tests the bit of the last one). Face windings are clockwise seen from the front, in the left-handed engine space.
- Leaf ambient[0] is the light level (0..31) used for objects standing in the leaf (set_ambient_light_); the other three bytes are 255 or 0.
- The compiler in this SDK: formats/d6bspc.py (see docs/geometry.md).
- A new or changed BSP needs a compiler. The BSP tree, clipnodes (collision), PVS, marksurfaces and per-face .lfl.ls lightmaps are regenerated together. They are standard Quake-1 structures: the SDK runs a qbsp/vis with the Deep6 hulls and converts the result to version 28 (68-byte models, 52-byte texinfo, lump 8 as u16 per surfedge, mirrored axes).
- Things that can be edited without recompiling: entities (door parameters, names), .bol objects, lights (.lgt/.rgb, although baked lightmaps will not follow), nav graph (.nvs/.l2n), textures (.twd, keep 128x128 and indices).

Chapter 3. Outdoor terrain ("spoke") file formats

Source: docs/formats/terrain.md

Reverse engineered from deep6.exe (Watcom debug symbols, Ghidra decompile in re/decomp/). Parser/writer: formats/d6terrain.py (byte-identical round trip of all 951 files covered; run python3 formats/d6terrain.py --selftest <gamedir>).

All multi-byte values are little endian. Confidence is given per field: **H** = read directly from the loader code, **M** = loader plus data statistics, **L** = guessed from the data only.

1. World model and coordinates

- The world is split into 13 *spokes* (`_gSpokeNumber` 0..12). Only spokes **0, 3 and 11** are heightmap terrain (case 0/3/0xb in `LoadSpokeSegment_`, `d6spoke.c` 0x483038). The others are pure-BSP dungeons that call `Terrain_Null_()` and load hardcoded BSPs:

spoke	type	content (BSP files)	terrain-side entrance BSP (in .TOL)
0	terrain	SPOKE00.* (forest, river)	-
1	bsp	CRYPTA CRYPTB	tcrypt in spoke 0
2	bsp	TEMPLEB TEMPLEA	ttemple, ttmpcave in spoke 0
3	terrain	spoke03.*	-
4	bsp	SHURU	tshurua..j in spoke 3
5	bsp	MINESA..D	tmines in spoke 3
6	bsp	OGREA..C	togrea/b in spoke 3
7	bsp	DRAGONA..C	tdragona/b in spoke 3
8	bsp	LICHA LICHB	tllich in spoke 11
9	bsp	SUNKENA SUNKENB	tsunkena..c in spoke 11
10	bsp	SHRINEA	tshrinea..c in spoke 11
11	terrain	spoke11.* (desert, palms)	-
12	bsp	PYRAMA PYRAMB	tpyram in spoke 11

(The pairing follows from the exits: the transition is the `EVENTS.COD` op 0x2A `LOADSEGMENT`, which re-bases the party onto a BSP slot of the new spoke; every shipped exit is listed in `exits.md`.)

- Spoke 0 also embeds BSPs `torcave`, `ttoada/b`, `truinsa..f` (ruins, caves) and spoke 3 `thutb`, `tshackb`; these have no dungeon spoke of their own.
- Units: 1 terrain tile = 1024 world units in x and z; y is up. Height map values and object y are in the same units. Angles are 1024 per turn (`_gSin/_gCos` tables, entry dirs 0x100/0x300).
- Tile arrays are row major: row = z, column = x (`Terrain_XYToTile_` = tiles + (x + z*width)*0x1c). In this document "up" in an image means -z.

Towns, gates and entry points (H)

`SetSpokeEntry_` (deep6.c 0x4134b8) picks the spoke when leaving a town, `GetEntryPosition_` (d6spoke.c 0x489d20) holds hardcoded spawn points (full analysis, party placement and dungeon exits: `exits.md`):

from	to spoke/entry	spawn (tile x, z)	gate BSP nearby
town 0 (gate field 0)	0 / 0	(212.5, 81.5)	tgatea
town 0 (gate field 1)	0 / 1	(181.5, 72.5)	tgateb
town 1, entry 0	0 / 2	(10.5, 275)	tgatec (spoke 0)
town 1, entry != 0	3 / 1	(277.5, 10.5)	TGATED (spoke 3)
town 2, entry 0	3 / 0	(7.8, 277.5)	TGATEC (spoke 3)
town 2, entry != 0	11 / (1)	(374, 25)	TGATEE

(Taken literally from the code; note the gate BSP names do not line up with the towns - `tgatec`/`TGATEC` are probably the same gate model reused. The town side of each transition is in `townhub.c`, `_gSpokeEntryNum` = `_TownAVI_fieldVal`.)

So the chain is: Town0 = spoke 0 = Town1 = spoke 3 = Town2 = spoke 11, with dungeons hanging off the terrain spokes. Spokes do not connect directly to each other; each terrain edge transition goes through a town. Dungeon spokes use one entry each (table in `d6terrain.ENTRY_POINTS`).

Files a terrain spoke loads (order of `LoadSpokeSegment_`)

- SKYDROP.M2K, TREEDROP/TTOPDROP (.bmp.p16; spoke 11 uses PALMDROP/PTOPDROP), SUNMAP.BMP, SUNALPHA.BMP, MOONMAP.BMP (`LoadSkyAndTree_`), `syswat.twd` (water textures).
- SPOKEnn.TMR -> `Terrain_Load_` (tiles, textures, palettes).
- SPOKEnn.NAV -> `LoadTerrainNavPnts_`.
- SPOKEnn.TOL -> `LoadTerrainBsps_` ('B' records only), then `Terrain_LockLevel_`.
- D6LINKnn.DAT -> `LinkNavPoints_`.
- SPOKEnn.FOL -> `LoadStaticObjects_` (unless `_gNoLoadFOL`).
- D6SEGnn.GAM -> `SegRead_Segment_`; **only if that fails** SPOKEnn.TOL -> `LoadTerrain0bjects_` ('I','M','P' records) and `LoadAllBsp0bjects_` (<bsp>.BOL).
- `maps/lm0b<nn>.lm` (+ `.fog`, `.mrk`) -> `jhpMap_Create_(0, spoke)`.

SPOKEnn.LIT, SPOKEnn.PAR, spokenn.tom and tilebmp.lst are **never opened by deep6.exe** (no format string exists); they are level-editor sources. emitters.dat is global (particle system).

2. SPOKEnn.TMR - terrain map (loader Terrain_Load_ 0x414144, Terrain_LoadPage_ 0x4148e0)

```
0x0000 int32[1024] header (_gTomHead)
0x1000 char[1024][4] BLT tile name table (_gTomBLT)
0x2000 width*height tile records, 16 bytes each
```

Sizes: spoke 0 and 3 are 288x288 tiles, spoke 11 is 384x384.

Header (only [0],[1] are read by the game):

off	type	meaning	conf
0	int32	width in tiles (must be multiple of 32, "acre")	H
4	int32	height in tiles	H
20	f32 x2	editor values (e.g. 158976.0, 56422.0); unused	L
32	int32	2 in all files (version?) ; unused	L
rest		zero	

BLT table: entry i is the texture for tile tex == i; entry 0 is empty. Each name is 4 terrain-type characters, one per tile corner (1 2 3 4 5 6 B C G H K L M R S T Y E N O V P ...). Corner order, verified against neighbouring tiles (99.9% agreement), in array orientation: char0 = (x+1, z), char1 = (x, z), char2 = (x, z+1), char3 = (x+1, z+1). **M**

Terrain_LoadTextures_ (0x413c74), **H**:

- name[0] == 'X'/'x', or bit 7 set in name[3]: entry skipped (665 of 776 entries in spoke 0 have bit 7 set; none of them is used by a tile).
- Otherwise the canonical name is the lexicographically smallest of the four byte rotations (Terrain_NameToRotation_); the canonical texture is loaded from tiles/<dir>/<canon>.mip (dir 801..806 found by TileDir_) and the entry stores rotation k. np.rot90(canon_texture, k) equals the pre-rotated file that also ships in tiles/, so an editor can use either.
- Palette = tiles/<dir>/pal.p16 (slot dir-801).
- Texture rows run toward -z: draw flipud(texture) when z grows downward.

Tile record (16 bytes, unpacked into a 0x1c-byte runtime struct):

off	type	runtime	meaning	conf
0	u16	+0x00	tex: BLT index (0 = no tile / void)	H
2	s16	+0x02	vertex height at corner (x*1024, z*1024); planes are built from this and the +x, +z, +x+z neighbours (Terrain_CalcPagePlanes_)	H
4	u8[6]	+0x06 (each byte x3 as RGB; <=3 in hardware mode)	baked light, shade row 0..31 of the p16 palette, 2 triangles x 3 vertices	H (layout) / M (vertex order)
10	s16	+0x04	always 4096 in shipped maps; use unknown	L
12	u8	+0x1a	flags, see below	H/M
13	u8	+0x18	water type 0..6 (Terrain_IsWaterTile_, _gWaterTables: 1,2 -> surface y 0; 3,4 -> 7936; 5,6 -> 5120)	H
14	u8	+0x19 (&0x7f; bit 7 = runtime "seen" by ray caster)	wall type: 0 none, 1 tree wall (forwal / plmwal on spoke 11), 2 castle wall (caswal)	M
15	u8	-	unused, 0	H

Flags (byte 12), **M** from statistics, split bits **H**:

- 0x0f = solid wall tile (forest interior). Single bits 1/2/4/8 are diagonal wall tiles whose open corner is (-x,-z) / (+x,-z) / (+x,+z) / (-x,+z) respectively.
- bits 2 and 8 also select the alternate triangle split diagonal in Terrain_CalcPagePlanes_ / Terrain_AdjustBase_; ray casting (XTileSight_) treats flag bits as sight blockers.
- 0x10 +x face, 0x20 +z face, 0x40 -x face, 0x80 -z face: the side(s) where a wall tile borders open ground (100% match with neighbours); these are where the tree-wall bitmaps (tiles/forest/*wal0n.bmp) are drawn.

3. spokenn.tom - editor twin of .TMR (not loaded)

Same header and BLT table (identical bytes), same tile count, different 16-byte tile layout and **no light bytes**:

off	type	meaning	conf
0	u8[4]	per-corner terrain code (editor)	L
4	u16	tex (equals .TMR except void tiles: .tom 745 vs .TMR 0)	M
6	u16	0 / 0x0ccc / 0x1000 (unknown)	L
8	s16	height (equals .TMR)	M
10	u8	flags (equals .TMR byte 12)	M
11	u8	water (equals .TMR byte 13)	M
12	u8	wall type (equals .TMR byte 14)	M
13	u8[3]	0	

4. 64-byte record containers: .TOL .NAV .FOL .LIT (and dungeon .BOL)

Record 0 is a header: int32 at 0 = number of records **including** the header (H); int32 at 0x1c = 1 in all files, other bytes 0

except NAV byte 0x3e (0x77); preserved raw. Every other record:

off	type	meaning	conf
0	char	kind	H
1	char[3]	id as 3 ASCII digits ("001"); parsed as (c1-'0')*100+(c2-'0')*10+(c3-'0')	H
4	f32[3]	position x, y, z (world units)	H
16	f32[3]	rotation (pitch, yaw, roll) in 1024ths of a turn, copied to GraphObj +0x14	H (copy) / M (units)
28..		kind specific	

The **record index is the object's world tag** (`_g0BJTag[index]`): events, D6SEG saves and D6LINK refer to objects and nav points by index, so an editor must not reorder or delete records (blank them to kind '0' instead, as the shipped files do).

.TOL (terrain objects) - LoadTerrainBsp_ 0x4d6804, LoadTerrainObjects_ 0x4d6b3c

- 'B' terrain BSP placement (id field holds "SPT"), **H**: pos.x/1024 = tile x, pos.z/1024 = tile z, f32 @16 = width in tiles, f32 @24 = depth in tiles, u8 @32 = BSP slot (0..15, must be unique and dense), s8 @33/@34/@35 = offset x/y/z (passed to Terrain BSPLoad_), char[16] @48 = BSP base name (<name>.bsp, objects in <name>.bol). The terrain under a BSP footprint is flattened in the TMR.
- 'I' item, id = D6ITEM record (LoadItemObject_). **H**
- 'M' monster, id = D6MONS record (MonsterEntry_). **H**
- 'P' prop, id = D6PROP record (LoadPropObject_); u8 @32 != 0 starts the prop's emitter inactive (prop 0x1d gets PropEFX_ActivateEmitter_). **H**
- '0' unused slot.

.NAV (nav graph) - LoadTerrainNavPnts_ 0x4d664c

'N' records (max 2000): pos (y = 0, snapped to ground by AdjustTerrainNavPnts_), u32 @32 copied to runtime +0xc (0 in data), u16[6] @36 = links to other nav records by **record index** (0 = none), u16 @48 flags (bits 1,2 cleared on load; 2/4/16 seen), u16 @50 copied, bytes 52..63 editor scratch (two floats + u32), ignored by the game. **H**

.FOL (foliage) - LoadStaticObjects_ 0x4d6dcc

'F' records only (others ignored, spoke 11 has three '0' blanks): pos + rot of a tree prop placed at load time. **The id is ignored**: the prop type is Random() from {1,2,3,4,0xe7} on spokes 0/3 and {0xe4,0xe5,0xe6} (palms) on spoke 11. Trees line the forest-wall edges of the paths. **H**

.LIT (lights, editor only)

'L' records: pos (y 8192 or 9015), rest 0. Not loaded by the exe; the editor baked them into the TMR light bytes. **H** (not loaded) / **L** (meaning)

5. SPOKEnn.PAR (editor only)

72 bytes: char[4] "0.1v", u32 64 (payload size), int32[16]. Values look like generator parameters (e.g. 16383/-16384 and 196/-196 ranges, 0x3f000000 = 0.5f, 20). Not loaded by the exe. **L**

6. Tile textures and palettes

- tiles/80x/NAME.mip, tiles/xxxx.mip (Load_TerrainMipMap_ 0x413bcc): raw 8-bit indices, 128x128 then 64x64, 32x32, 16x16 (21760 bytes). **H**
- *.p16 (Pal16_Load_ 0x44df2c): 0x4000 bytes = u16 RGB565 [shade 0..31][index 0..255]; shade 31 is the brightest but still only ~25% of full scale (the engine applies gamma, Pal16_SetRGBGamma_). **H/M**
- Terrain palette slots (Terrain_LoadPalettes_): 0..5 = tiles/801..806/pal.p16, 6 = tiles/forest/forwal.p16 (plmwal.p16 on spoke 11), 7 = caswal.p16. Wall bitmaps: tiles/forest/{forwal,plmwal}0[0-3].bmp, *can00.bmp (canopy), caswal00.bmp, cascan00.bmp.
- tiles/pal.p16, tiles/water.p16, tiles/termip.exe, MIPTILE.BAT, tiles/forest/bmp2p16.exe are tool leftovers.
- tilebmp.lst: CRLF list of 136 tiles/80x/NAME.mip paths; not read by the exe. **H**
- texture.dat: 8 bytes, u32 2 + u32 check value; read by the CD check (cdprot.c), unrelated to terrain. **H**

7. Sky

- skydrop.m2k (Bmp2K_LoadFromM2K_ 0x53e1f0): int32 width (1024), height (90), ncolors (1532); u16[2048] RGB565 palette (copied to gSkyOrigPalette for time-of-day tinting); u16 pixels[h][w] (palette indices). Loaded on spokes 0/3/11 only. **H**
- skymtn00.bmp, treedrop/ttopdrop/palmdrop/ptopdrop.bmp + matching .p16, sunmap.bmp, sunalpha.bmp, moonmap.bmp: standard BMPs (BmpFile keeps them raw).

8. Automap: maps/lm0b.lm, .fog, .mrk (jhpmap.c)

.lm (jhpMap_Create_ 0x5a5d70, jhpMap_SetBlocks_), **H**:

```
0  int32 magic 0x4c4d ("ML"), int32 version 2, int32 0, int32 0
16 int32 nlevels
20 nlevels * 0x23c level headers
    then per level: image pages, then mask pages (contiguous)
```

Level header: +0 image offset, +4 image size, +8 mask offset, +0xc mask size, +0x10 f32[3] bbox min (x,y,z), +0x1c f32[3]

bbox max, +0x28 int bsp (-1 terrain, n = BSP slot, -5/-7 special origins), +0x2c pages_w, +0x30 pages_h, +0x34 page bytes (0x4000 = 128x128 8-bit), +0x38 mask bytes (0x800 = 128x128 1-bit), +0x3c u16[256] RGB565 palette. Pages are row major; pixel $x = (x - \min.x) * \text{pages_w128} / (\max.x - \min.x)$, pixel row $= (\max.z - z) * \text{pages_h128} / (\max.z - \min.z)$ (rows run from +z to -z). Levels select by viewer position/bsp (several height bands per dungeon). These are hand-painted map images, not derived from the TMR at runtime. `formats/d6automap.py` (editor: Tools > Redraw the automap of this spoke) renders new pages in the same style, keeping each level's palette, bbox and mask: dungeon levels from the BSP floors in the level's height band, terrain levels from the tiles, walls, water and placed BSPs. BSP floors more than 512 units below the terrain (caves under hills) are left out. Checked in the game (spoke 0): the redrawn page lines up with the original (same revealed shapes and PC marker); the hand-painted pages show some BSP areas as rock where the redraw shows their top floors.

`.fog` (written by the game): one chunk per page of every level in order: int32 nbytes, u16 runs[nbytes/2]: bit 15 = bit value, bits 0..14 = run length, 0 terminates; bits MSB-first make the 0x800-byte explored mask. **H**

`.mrk` (written by the game): per level int32 n, n * (int32 x, int32 y, int32 colour, char text[256]). **H**

9. D6LINKnn.DAT - nav graph glue (`LinkNavPoints_ 0x511fac`)

int32 n, 4 unused bytes (absent if n = 0), n * int16[4] (a, b, c, d): d == -1: terrain nav b <-> nav a of BSP slot c; c == -1: terrain nav a <-> nav b of BSP slot d; else BSP-to-BSP. **H**

10. Runtime state (written by the game, not level data)

- D6WORLD.DAT (`Save/LoadWorldGameData_`), 5416 bytes: world clock (4), NPC stop time (4), NPC status 0x140, NPC mbits 0x280, world state flags `wState` 0x1000, portal flag 0x40, base 0xc0, hret 0x18, home 0x48. **H**
- D6SEGnn.GAM (`SegWrite_Segment_`): 100-byte header of 25 int32 (offset,count) pairs: switches, traps, BSP entities, objects, NPCs, monsters, events (+ sub-tables), NPC message queue offset; sections are kept raw by the parser. If this file exists the .TOL objects are **not** loaded. **H** (header) / not decoded (sections)
- `ResetAdventures_` deletes D6WORLD.DAT, D6SEG00..15.GAM and JOURNAL.*. The copies in the analysed game dir are from a play session.

11. emitters.dat (`LoadEmitterDesc_ 0x47bb10`)

int32 n (31), n * 128-byte descriptors: +0 char[24] name, +0x18 u32 flags (bits 1/2/4/8 tested), +0x44 sprite size (<=3), +0x60 frame count (0 = from anim), +0x64 child emitter index+1, +0x68 lifetime, +0x6c rate, +0x74 anim id (0 -> palette mode), +0x78 palette id, +0x7c alpha id. **M**

12. Notes for a map editor

- Editable at runtime level: TMR heights/tex/flags/water/walls/lights, TOL, NAV, FOL, D6LINK. Keep record indices stable.
- Light bytes are baked; there is no in-game relighting from .LIT. An editor must recompute them (or copy neighbours) after changing heights.
- Wall flags must stay consistent with neighbours (face bits 0x10..0x80, wall type) or tree walls render with gaps.
- BSP footprints ('B' records) are flattened in the height map and the automap .lm pages are pre-painted; both need regenerating if moved.
- Delete D6SEGnn.GAM (start a new adventure) to see .TOL edits.
- Unknowns: TMR header floats, tile bytes 10-11, TOM bytes 0-3/6-7, PAR values, NAV bytes 52..63 - all preserved raw.

13. Findings from the editor (d6edit)

- Light bytes are per corner.** The six bytes of a tile are the light of corners (x,z) (x,z+1) (x+1,z) (x,z+1) (x+1,z+1) (x+1,z); every shared corner has the same value in all three shipped maps. **H**
- Bake model.** light = clamp(ambient + k * sum over .LIT lights of max(0, 1 - d/R), ambient, 31), d = horizontal distance. Fitted: spokes 0 and 3 ambient 10, k 17-18, R 10000; spoke 11 ambient 24, k 25, R 9000 (mean error 0.4 - 0.9 shade). **M** (fitted, not read from code)
- Terrain nav ids are record indices (`LoadTerrainNavPnts_` fills `terrnavid2idx[record] = record`); D6LINK and scripts use those ids, BSP nav points are addressed by their `nav_id` (`GetNavPntIdx_`). **H**

Chapter 4. Terrain walls, canopy, foliage and texture import

Source: docs/formats/walls_textures.md

Reverse engineered from deep6.exe (Ghidra decompile in re/decomp/: tnew.c RenderTile_, terrain.c Terrain_LoadTextures_ / Terrain_Draw_, tpoly2.c Terrain_AddPoly_ / Terrain_NewerPoly_, cardctrl.c Card_LoadTreeWall_ / Card_DrawTerrainPoly_, scenload.c LoadStaticObjects_, random.ASM Random_). Python: formats/d6walls.py (walls, canopy, foliage) and the texture writers in d6terrain.py / d6level.py. Confidence: **H** read from code, **M** code + data, **L** guess.

Constants read from the exe data segment are given with their address.

1. Tree / castle walls and canopy (RenderTile_, tnew.c 0x4f55b4)

1.1 Wall textures (Terrain_LoadTextures_, terrain.c 0x413c74) - H

terrain+0x2c holds 7 bitmap pointers (Bitmap_LoadFromBMP_):

slot	spokes 0/3	spoke 11	size	palette
0	tiles/forest/forwal00.bmp	plmwal00.bmp	128x512	slot 6 (forwal.p16 / plmwal.p16)
1	forwal01.bmp	plmwal01.bmp	128x512	slot 6
2	forwal02.bmp	plmwal02.bmp	128x512	slot 6
3	forwal03.bmp	plmwal03.bmp	128x512	slot 6
4	forcan00.bmp (canopy)	plmcan00.bmp	128x128	slot 6
5	caswal00.bmp (castle wall)	same	128x512	slot 7 (caswal.p16)
6	cascan00.bmp (castle top)	same	128x128	slot 7

Palette slot 6 is tiles/forest/plmwal.p16 on spoke 11, else forwal.p16 (Terrain_LoadPalettes_). The BMP's own RGB palette is ignored; the 8-bit indices are looked up in the p16 shade table (32 rows x 256 RGB565).

The BMPs are drawn **opaque**: there is no colour key. Index 0 occurs in the bitmaps (8917 pixels in forwal00) and is a normal dark colour. Software path: asm_PerspectiveScanline79_ (128x512) and Bsp_TexSpanSub16_ (128x128) write every texel. Card (D3D) path: Card_LoadTreeWall_ converts each bitmap to a 16-bit texture with shade row 31 (p16 + 0x3e00) and no alpha. 128x512 walls are point-sampled down to 64x256 (every 2nd column of every 2nd row) and their uv is scaled by 0.5; 128x128 canopies keep full size (uv scale 1.0). Lighting in the card path is per-vertex colour (light < 3, 0..255).

1.2 Per-tile geometry - H

Runtime tile struct (0x1c bytes) relevant fields: +0x02 height, +0x04 unk10 from the file (**wall height**, 4096 in every shipped tile), +0x06.. light, +0x18 water, +0x19 walltype (bit 7 = runtime "seen"), +0x1a flags, +0x1b runtime visibility.

For tile (x, z) with world origin X = x*1024, Z = z*1024 RenderTile_ builds four ground vertices (all y = tile height of the named tile):

name	world (x, z)	height from	light byte (file light[i])
V0	(X, Z+1024)	tile (x, z+1)	light[1]
V1	(X+1024, Z+1024)	tile (x+1, z+1)	light[4]
V2	(X+1024, Z)	tile (x+1, z)	light[2]
V3	(X, Z)	tile (x, z)	light[0]

(at the map edge the missing neighbour height is 0.)

If flags != 0 it also builds the **top** vertices T0..T3: same x/z, y = ground y + unk10 of the same corner tile (so the wall/canopy top follows the terrain 4096 units higher), light = (l0+l1+l2+l3) >> 2 (average of the 4 ground lights; the >>2 is in the code, the sum is the FPU value Ghidra lost - **M**).

Ground triangles (the default split is V0-V2; flags 2 or 8 select the other diagonal V3-V1):

split	tri A (plane +0x14)	skipped if	tri B (plane +0)	skipped if
default	V0 V1 V2 (corner +x+z)	flags & 1	V2 V3 V0 (corner -x-z)	flags & 4
alt (flags&0xa)	V3 V0 V1 (corner -x+z)	flags & 2	V1 V2 V3 (corner +x-z)	flags & 8

A skipped ground triangle is replaced by the same triangle of the top vertices (**canopy**), textured with slot 4 (tree) or slot 6 (walltype 2, castle). So:

- flags & 0x0f == 0x0f: whole tile is canopy at ground+4096 (forest interior).
- flags 1 / 4 / 2 / 8: half tile canopy, half ground; the open corner is (-x,-z) / (+x,+z) / (+x,-z) / (-x,+z).

Canopy uv (texel units, _DAT_005c83f9 = -0.5): T0 (0.5, 0.5), T1 (w-0.5, 0.5), T2 (w-0.5, h-0.5), T3 (0.5, h-0.5), w = h = 128: one canopy bitmap per tile, row 0 at the +z edge. Canopy triangles are only drawn if the camera is above the triangle's plane (backface test against the ground plane normal shifted to the top vertex, eps -1.0 at _DAT_005c8401), and not when the camera is under water.

Vertical wall quads, 4 vertices each, uv (0.5,0.5) (w-0.5,0.5) (w-0.5,h-0.5) (0.5,h-0.5) in that vertex order (w=128, h=512, i.e. the whole bitmap spans one tile edge and the full 4096 height; bitmap row 0 at the top):

flag	quad (top, top, ground, ground)	outward normal (_s_wall_* 0x5e8f8c..)	bitmap slot
0x20	T1 T0 V0 V1 (edge z+1)	+z (_s_wall_N 0,0,1)	A

0x40	T0 T3 V3 V0 (edge x)	-x (_s_wall_W)	B
0x80	T3 T2 V2 V3 (edge z)	-z (_s_wall_S)	A
0x10	T2 T1 V1 V2 (edge x+1)	+x (_s_wall_E)	B
1	T0 T2 V2 V0 (diag V0-V2)	toward -x-z (_s_wall_NE normal is +x+z, drawn when the camera is on the <i>negative</i> side)	C
4	T2 T0 V0 V2	toward +x+z	C
2	T3 T1 V1 V3 (diag V3-V1)	toward +x-z	D
8	T1 T3 V3 V1	toward -x+z	D

Diagonal quads are only emitted when flags & 0xf is neither 0 nor 0xf. Straight faces are culled unless the camera is on the outward side ($\text{dot}(\text{cam}, \text{n}) - \text{d} \geq -1$), i.e. single sided.

Bitmap choice (no hashing, no flags 0x10..0x80 involvement): $c = (x + z) \& 1$ (checkerboard), walltype 1: A = slot c, B = slot c^1 , C = slot 2 + c, D = slot 2 + (c^1) ; walltype 2 (castle): every face uses slot 5 (caswal00) and the canopy slot 6 (cascan00). So forwal00/01 alternate on straight faces and forwal02/03 on diagonals. The diagonal quad is 1448 units wide but uses the same 128 texel width (stretched by $\sqrt{2}$).

All wall polys use draw mode 3 (Terrain_AddPoly_ CL=3 -> perspective correct in software when $1/z > 0x39000000$), canopy mode 0 and ground mode 5 (software stipple) / 0 (card).

1.3 Data rules seen in the shipped maps - M

- flags != 0 <=> walltype != 0 (all three maps, no exceptions).
- face bits 0x10..0x80 only occur together with 0x0f (plus a single 0x18 tile in spokes 3 and 11), diagonal tiles never carry face bits.
- walltype 2 (castle) exists only in spoke 3 (827 tiles).
- unk10 = 4096 everywhere; editing it changes the wall height per corner.

1.4 Visibility and distance - H

- Terrain_Draw_ walks tiles in a square of +-_gViewDist tiles around the camera (_gViewDist default 16, adjustable 8..16 in the options / with the view distance keys), front to back, per terrain BSP leaf; tiles with runtime byte +0x1b == 0 are skipped.
- RenderTile_ first ray casts (RayTerr_RayCast_, rayterr.c) when the camera is less than 2560 above the tile (_DAT_005c83d5) or less than 4096 above its lowest corner, and skips occluded tiles.
- Tiles inside a terrain BSP footprint (leaf with a BSP) draw no ground/walls/canopy, only their objects.
- Under water (_gUnderwater and no transparent water) no walls/canopy are drawn.

2. Foliage (.FOL, LoadStaticObjects_ scenload.c 0x4d6dcc) - H

- For every record after the header (all kinds, including blanks) the loader calls Random_(n) **before** testing the kind; only 'F' records create a prop: LoadPropObject_(n, table[Random_(n)], pos, rot). table = {1, 2, 3, 4, 0xe7} (n=5) on every spoke except 11, {0xe4, 0xe5, 0xe6} (n=3) on spoke 11.
- D6PROP record -> model: 1 Forest Tree (tree-forest.mdl), 2 Cedar (tree-cedar.mdl), 3 Willow (tree-willow.mdl), 4 Sequoia (tree-sequoia.mdl), objtype 0 = drawn as model, others objtype 1 = tree sprite via AddTreeSprite_, 0xe7 Birch (tree-birch.mdl); 0xe4..0xe6 Palm Tree (palm1/2/3.mdl).
- Random_(random.ASM 0x44cc76): seed = seed * 0x1df5e0d (mod 2^{32}); return ((seed >> 16) * n) >> 16. The seed is set once at start-up from MMClockTicks_() (deep6.c, InitRandom_ and WinMain init) and the same generator is used by all game code, so **the tree choice is not reproducible**: it changes on every spoke load. The record id field (0..4 in the data, distribution e.g. spoke 0: 118/1001/864/712/20) is ignored by the game; it is probably the editor's choice. d6walls.foliage_choice() therefore offers (a) mode='id': table[id % n] as a stable editor preview, and (b) mode='random', seed=s: an exact emulation of the loader for a given seed.
- Position: record pos; y is 0 in all shipped records -> LoadPropObject_ snaps it to the terrain (D6_AdjustBase_) when not inside a BSP area. Rotation: record rot copied to GraphObj+0x14 (yaw values 0..6 / 1024 in the data, i.e. effectively unrotated; tree sprites are billboards anyway).
- Trees stand on open ground next to the walls (spoke 0: 1554 on flag-0 tiles, ~1160 on diagonal wall tiles' open half).

3. Shaded palettes (.p16, twd palettes, model palettes)

3.1 Shade rows - H (code) / verified on all files

Pal16_Calculate_ / RefPal16_Calculate_ (pal16.c 0x44e124) build a 0x4000-byte table from 256 RGB888 base colours:

```
for s in 0..31: for i in 0..255:
    r = (R[i] * (s+1)) // 32 ; g = (G[i] * (s+1)) // 32 ; b = (B[i] * (s+1)) // 32
    p16[s][i] = (r >> 3) << 11 | (g >> 2) << 5 | (b >> 3)
```

Row 31 is the full colour, row 0 is 1/32 of it (not black). The file holds the un-gamma'd table; at load time the engine applies Pal16_SetRGBGamma_ (per channel $c8 + \min(c8 * g / 128, g)$, $g = _g_GammaVal$ from the options) and, if the display is not 565, Pal16_ConvertRGBBits_ (555). An SDK must write the plain table.

Checked against every shipped file (exists a base colour reproducing all 32 rows of every entry):

- all tiles/80x/pal.p16, tiles/forest/*.p16, tiles/pal.p16, font and sky .p16 files, efxgfx/*.p16: formula above, 0 mismatches. tiles/forest/forwal.p16 row 31 equals the BMP palette of forwal00.bmp truncated to 565 (the BMP palette is the base colour set).
- tiles/water.p16, gfx/particle/*.p16: hand-made gradients (not this formula).
- .twd palettes: 392 of 435 use the formula above; 43 (e.g. all of crypta.twd) use $(c * s) // 32$ instead (row 0 black, row 31

= 31/32). Both work; new palettes should use the engine formula.

- model palettes (paltype 0, 695 files): formula above (index 0 is forced to black at load time by Model_ResetTextures_).

d6terrain.make_p16(rgb256) implements it (bytes, 0x4000).

4. BSP level textures (.twd) for import - H (texlist.c Texlist_LoadBSPWadfile_ 0x5507cc)

Layout (see levels.md section 3): [int32 -666], int32 npal, int32 ntex, npal x 0x4000 p16 tables (**per wad**, any texture may pick any of them by its int16 palette index), ntex x {char name[64], int16 palette, int32 w, int32 h, mip0..mip3}.

- Size: **128x128 only**. The loader reads into fixed 0x4000/0x1000/0x400/ 0x100 buffers and copies (0x80>>mip)^2 bytes to the master slot; larger textures overflow, smaller ones get garbage. (One shipped texture, shrinea.twd 420009snakewall, is 128x127 and works only by luck.)
- Mips: the shipped wads were mipped with the same algorithm as termip.exe (section 5) using the texture's palette: 507/507 sampled textures reproduce exactly. Keep the -666 header (without it only mip0 is read but the master copy still uses 4 levels).
- Palettes: one p16 per wad entry, built with the section 3 formula; each goes into a new master slot (max 512 palettes over all loaded BSPs, Texlist_FindPaletteSlot_), gamma applied at load.
- Master texture table: max 1024 slots (Texlist_FindTextureSlot_) shared by all loaded BSPs. **Textures are de-duplicated by their 16x16 mip only** (256-byte compare, name and palette ignored): a new texture whose mip3 equals one already loaded reuses that slot and its palette. Make mip3 unique.
- Texinfo: flags & 0xffff = index in this .twd, remapped to the master slot (Texlist_AssignFaceTex_); faces with flags -1, 0x8000 (water) or 0x2000 (lava) are not assigned. Water faces draw syswat.twd texture 0 (warped, _g_watersurf), lava faces (0x2000) syswat texture 1; selection is by position in syswat.twd, not by name.
- Names: copied into the master entry but never compared; no name-based animation, water or sky (unlike Quake). Shipped names are library ids like 02052wall-top. Sky is not a BSP texture (skydrop.m2k).
- Writer: TexWad.add_texture(name, rgb, palette=None|int) and replace_texture(i, rgb, palette='keep'|None|int). palette=int quantises to that wad palette (nearest RGB to its base colours recovered by p16_base_rgb); None builds a new 256-colour palette (PIL median cut) and appends its p16. Mips via termip algorithm; warns on mip3 collisions. build() == to_bytes(), byte identical for all 67 shipped wads; replacing a texture with its own decoded image reproduces the same indices.

5. Terrain tile textures (tiles/80x) - H

- tiles/<dir>/NAME.mip: 128x128 + 64 + 32 + 16 indices (21760 bytes), no header. tiles/<dir>/pal.p16 is the only palette for every tile of that dir (Terrain_LoadTextures_ sets texture palette = _gPalettes[dir-801]), so **all tiles of one dir must share the dir palette**; a tile with new colours needs its own dir or a re-quantised dir.
- Dirs: exactly 801..806 (LoadTilesFromDir_(0x321..0x326), palette slot check 0 <= dir-801 <= 8), no more can be added without code changes. TileDir_ returns the dir whose listing (InitDirList_) contains the canonical name; first match in 801..806 order (M: dir list built in that order). Names are the canonical (lexicographically smallest rotation) 4-char corner code, so a new tile must be stored under its canonical name.
- Mips (tiles/termip.exe, disassembled; reproduces **all 775** shipped .mip files byte for byte): colour table = p16 row 31 split as R bits 11..15, G bits 6..10, B bits 0..4 (5 bits each). Level k from level k-1: for each 2x2 block and channel c = (a+b+c+d + 2*max(a,b,c,d)) // 6, then the palette index with the smallest squared distance (first minimum wins, except on ties when one channel of the target is strictly largest: the candidate closer in that channel wins). Not decimation, not a plain box.
- API: TileMip.from_indexed(img128, palette), termip_downsample, termip_palette, make_p16(rgb256), p16_base_rgb(p16) (inverse; round trips the shipped tables), quantize(rgb, palette_rgb, exclude=()), tile_dir_palette_rgb(game, '803').
- tiles/forest/bmp2p16.exe ([-nolight] in out) is the tool that made the wall p16s from the BMP palettes (output matches section 3).

6. Model textures (paltype 0/3) - H/M

Same 32x256 RGB565 table, same formula (695/695 model palettes fit). Index 0 is forced to black / transparent at load, so quantise new model textures with quantize(..., exclude=(0,)) and write make_p16(base_rgb) as the palette; the engine selects the shade row from the vertex / terrain light (31 = full).

7. Uncertain

- Canopy-top light = average of the 4 corner lights: the >> 2 is in the code, the summed FPU value is inferred.
- The editor meaning of the .FOL id field (0..4) is a guess; the game ignores it.
- TileDir_dir order on duplicate names not traced in detail.
- Software-path ground uses draw mode 5 (stipple scanline) for all tiles; its visual effect was not investigated.
- Preview render: python3 formats/d6walls.py --render <game> 0 196 66 228 98 out.png (oblique ortho, single-sided walls, foliage as circles) shows the clearing at the spoke 0 town gate enclosed by tree walls with canopy on top, as in the game.

Chapter 5. Deep6 content / scripting data formats

Source: docs/formats/data.md

Wizards & Warriors (2000, Heuristic Park, engine "Deep6"). This covers the data a map/content editor needs: per-area scripting tables, object placement, the global record databases, the event script (EVENTS.DCL/COD), text, archives and models.

Reference implementation: formats/d6data.py (parse + byte-identical write, self test, area dump, event disassembler, archive list/extract).

```
python3 formats/d6data.py --selftest GAMEDIR      # round-trip every file
python3 formats/d6data.py --dump-area 0 GAMEDIR   # readable area dump
python3 formats/d6data.py --disasm GAMEDIR        # all EVENTS.COD routines
python3 formats/d6data.py --list GAMEDIR NPCDATA.PAK
python3 formats/d6data.py --extract GAMEDIR D6ARCHIV.DAT OUTDIR
```

Self test result on the GOG install: 907 files, 0 failures (13 x 6 spoke tables, 17 ENTR files, 59 TOL/BOL/FOL lists, 728 .mdl, 5 .lst, all global tables, both PAKs, D6ARCHIV, EVENTS.*, D6STRING, TEXTPAK, emitters, 4 .ldl). D6STRING and TEXTPAK also re-layout identically from parsed strings; D6SMIT00 checksum verifies.

Confidence: **high** = read directly from the loader / consumer code and confirmed in data; **med** = from usage, plausible in data; **low** = guessed or unused.

All integers little-endian. "Watcom" register calling convention in the decompile.

1. Global concepts

1.1 Spoke number (the nn in D6xxxxnn.DAT)

nn = `_gSpokeNumber` (0..12), the "spoke" (area) currently loaded. Files are opened with `sprintf("D6TRIG%02d.DAT", _gSpokeNumber)` in `InitEvents_` (events.c), `D6LINK%02d` in `LoadSpokeSegment_` (d6spoke.c), `D6TRAP%02d` in `InitSpokeTraps_` (traps.c). Mapping from `LoadSpokeSegment_ / FetchObjects_`:

spoke	type	geometry	object lists (BSP number)
0	terrain	SPOKE00.* (Valeia wilderness)	SPOKE00.TOL + terrain BSPs from its B records
1	dungeon	CRYPTA, CRYPTB	crypta.bol (1), cryptb.bol (2)
2	dungeon	TEMPLEB, TEMPLEA	templeb.bol (1), templea.bol (2)
3	terrain	spoke03.*	SPOKE03.TOL
4	dungeon	SHURU	shuru.bol (1)
5	dungeon	MINESA..D	minesa..d.bol (1..4)
6	dungeon	OGREA..C	ogrea..c.bol (1..3)
7	dungeon	DRAGONA..C	dragona..c.bol (1..3)
8	dungeon	LICHA, LICHB	(1),(2)
9	dungeon	SUNKENA, SUNKENB	(1),(2)
10	dungeon	SHRINEA	(1)
11	terrain	spoke11.*	SPOKE11.TOL
12	dungeon	PYRAMA, PYRAMB	(1),(2)

Dungeon spokes call `Terrain_Null_` and `Terrain_BSPLoad_(name, ...)` with hard-coded placement arguments (in `LoadSpokeSegment_`), so BSP placement for dungeons lives in the exe, not in data (decoded: `d6level.SPOKE_TABLE`, `levels.md` section 11).

Default entry points are hard-coded in `GetEntryPosition_` (d6spoke.c), e.g. spoke 0 entry 0 = (217600, 0, 83456) dir 0x100 (just outside the Valeia east gate). They are only used for town-gate arrivals. Spoke changes, dungeon entrances and exits, GOTOBSP and ENTERTOWN are covered in `exits.md` (tool: `formats/d6exits.py`).

1.2 World state

`_WState[spoke*256 + n]` - 256 byte "world states" per spoke. Triggers watch them, boundary areas / specials / switches write them, script ops read/write them. Saved in `D6WORLD.DAT` / segment files. `!STATE` parameters in the DCL are such indices.

1.3 Object ids (`GetObjNum_`, `scenload.c`) - high confidence

Placed objects are referenced by an int:

- id < 100000 - 1-based record index in the terrain object list (SPOKE00.TOL).
- id = n*100000 + k - record k (1-based) of BSP n's object list (.BOL). n == 16 means BSP 0 (since 0*100000 would collide with terrain ids).

Used by switches, traps, specials, bound areas (who=4) and script params such as `MONA..MOND`, `PROPIDA..D`, `OBJNUM`, `PROP`.

Door / entity ids (script `D00R`, `D00RA/B`, switch with `isentity=1`) are a different namespace: `bsp*100000 + entity` where the entity is a BSP brush entity and bsp 0 is literally 0 (`ProcessEventQueue_ -> MoveEntity_(bspListBSP[id/100000], id%100000)`).

1.4 Coordinates

World units, x/z horizontal, y up, terrain tile = 1024 units. Rotations in the object lists are floats in 1024-units-per-turn.

Coordinates in a .BOL, in a bound area with `bsp>=0` and in script params with `BSPNUM>=0` are relative to the BSP origin (`bsp + 0x118EC`); `Vec3_AddVec3_` adds it. For terrain BSPs the origin is the TOL B-record position minus its signed tile offsets x 1024 (bytes +0x21...+0x23, see `FetchTerrainObjects_`). Script coordinates (`XTILE/YTILE/ZTILE`) are int32 world units despite the name.

2. Per-spoke scripting tables

Common container (all five `D6xxxxnn` files): record 0 is a dummy whose first u32 is the record count N; record i (1..N) is at file offset `i*recsize` (loaders `fseek` there). File size = (N+1)*recsize, except `D6LINK04/10` and `D6SPEC07` which are just the 4-byte count 0. Bytes 4..recsize of record 0 are editor garbage (stale memory) - keep. Indices used everywhere (trigger numbers etc.) are these 1-based record numbers.

2.1 D6TRIGnn.DAT - triggers, 0xB8 bytes (`LoadTriggerData_`, `events.c`) max 511

off	type	name	meaning	conf
0x00	char[40]	event	event routine name, e.g. <code>@TEXTMSG</code> ; resolved with <code>FindEventAddress_</code> (case-insensitive) against <code>EVENTS.DCL</code> . Bytes after NUL are stale	high
0x28	u32	<code>rt_codeptr</code>	overwritten at load with <code>evdecl[idx]</code> (stale in file)	high
0x2C	u16	<code>rt_eventidx</code>	overwritten at load	high
0x2E	i16	state	world-state index watched by <code>CheckStates_</code>	high
0x30	u8	enabled	0/1; 0x3F = per-PC bitmask mode (bound oneshot 2)	high
0x31	u8	laststate	last seen value of state	med
0x32	u8	oneshot	clear enabled after firing (2 = per PC)	high
0x33	u8	nparams	overwritten from DCL count at load	high
0x34	u8	mode	state trigger: 0 fire on any change, 1 when state becomes !=0, 2 when ==0. For bound-area use: 1 = once per PC	high
0x35	u8[3]	-	padding	low
0x38	i32[30]	params	event parameters; bytecode operand n reads <code>params[n]</code>	high
0xB0	i32[2]	<code>rt_queue</code>	runtime scratch for queued ops (zeroed)	med

A trigger fires: (a) from `CheckStates_` when state changes per mode, (b) from a bound area / special / switch (`ExecuteEventCode_(code, actor, trig+0x38)`), (c) from script `CALLEVENT`. Param meaning comes from the DCL parameter names (2.7).

2.2 D6BOUNnn.DAT - boundary areas, 0x28 bytes (`LoadBoundAreaData_`, `CheckBounds_`) max 255

off	type	name	meaning	conf
0x00	i16	state	if trigger<1: <code>WState[state]</code> = (something inside) each frame	high
0x02	i16	bsp	BSP the box is relative to; -1 = world. Loader adds BSP origin to min/max	high
0x04	i16	trigger	trigger index fired for each actor inside (<1 = state mode)	high
0x06	u8	enabled	1 on; 0x3F = per-PC mask (oneshot 2)	high
0x07	u8	pcmask	runtime: PCs already handled	high
0x08	u8	oneshot	0 repeat, 1 disable after hit, 2 once per PC	high
0x09	u8	who	0 PCs, 1 party slots 0-5, 2 creatures not hidden, 3 non-party creatures, 4 one object (objid), 5 all creatures	high
0x0A	u16	-	unused	low
0x0C	f32[3]	min	AABB min x,y,z	high
0x18	f32[3]	max	AABB max	high
0x24	i32	objid	object for who==4	high

2.3 D6SPECnn.DAT - specials, 0x1C bytes (`LoadSpecials_`, `CheckSpecials_`) max 127

off	type	name	meaning	conf
0x00	i16	state	set to condition result when trigger<1	high
0x02	i16	?	always -1	low
0x04	i16	trigger	fired when condition true	high
0x06	u8	type	1 world clock in [a,b) (wraps), 2 all listed objects dead, 3 party has item a	high
0x07	u8	enabled		high
0x08	u8	?		low
0x09	u8	oneshot	disable once true	high
0x0C	i32[4]	args	type 1: start,end clock; type 2: up to 4 object ids; type 3: item id	high

2.4 D6SWITnn.DAT - switches, 0x14 bytes (`LoadSwitches_`, `HitTheSwitch_`, `SetLinkSwitch_`) max 254

off	type	name	meaning	conf
0x00	i32	objid	object id (1.3) or, if isentity, <code>bsp*100000+entity</code>	high
0x04	i32	link	next switch in a ring of linked switches (0 none)	high
0x08	i16	target	>=0: <code>WState[target]</code> = on; <0: run trigger -target	high
0x0A	i16	sfx	sound record on toggle (0 = prop's own sound)	high
0x0C	i16	keyitem	item required (0 none)	high
0x0E	u8	isentity	1 BSP entity, 0 placed object	high
0x0F	u8	flags	1 animate model, 2 consume key, 4 give key back when switched off, 8 hide polys, 0x10 key latches, 0x20 key needed even when on, 0x40 only via "use item"	med
0x10	u8	enabled	usable	high

0x11	u8	on	current state (initial value applied to WState at load)	high
0x12	u8	oneshot	disable after use	high
0x13	u8	unlocked	runtime	med

2.5 D6LINKnn.DAT - nav links, 8 bytes (LinkNavPoints_, pathai.c)

Record 0 = u32 count + 4 pad (file is only the 4-byte count when empty).

off	type	name	meaning	conf
0	i16	navA	nav point id on side A	high
2	i16	navB	nav point id on side B	high
4	i16	bspA	BSP of A, -1 = terrain nav (SPOKEnn.NAV)	high
6	i16	bspB	BSP of B, -1 = terrain	high

Builds terrain<->BSP and BSP<->BSP path-finding portals (_gBTNavLink, _gBBNavLink).

2.6 D6TRAPnn.DAT - locked / trapped containers, 0x24 bytes (InitSpokeTraps_, traps.c) max 255

off	type	name	meaning	conf
0x00	i32	objid	chest/crate object id	high
0x04	u32	trapmask	bit n => trap type n allowed (1..13, effects in TrapEffect_); 0 = untrapped	high
0x08	i32	traptype	armed type (picked from mask by FixTheTrap_)	high
0x0C	i16	locked	0 opened, 1 closed/active, 2 re-armed	high
0x0E	i16	treasure	D6TREAS record spawned on open (SetChestTreasure_)	high
0x10	i16	difficulty	lock/trap difficulty for skill checks	med
0x12	i16	attempts	runtime	med
0x14	u8	power	trap effect level	high
0x15	u8	nolock	non-zero: no random success	med
0x16	u8[6]	-	runtime, zeroed on reset	med
0x1C	u8	resetpct	chance the trap is removed at reset	high
0x1D	u8	monpct	chance of a chest monster	high
0x1E	i16	monrec	chest monster D6MONS record	high
0x20	i32	monmode	1/2 spawn style	med

2.7 ENTRssnn.DAT - named entry points (text) - med

ss = spoke, nn = index. CRLF lines x y z<TAB>name (world units), e.g. 217600 1024 83456<TAB>Town East Gate (matches spoke 0 entry 0). Not referenced by deep6.exe (developer teleport list); useful as editor bookmarks.

3. Object placement lists (.TOL terrain / .BOL BSP / .FOL foliage)

LoadTerrainObjects_, LoadBspObjects_, LoadStaticObjects_ (scenload.c), FetchTerrainObjects_ (mapobj.c). 0x40-byte records; record 0 header, u32 at 0 = record count **including** the header. Record index = object id (1.3).

off	type	meaning	conf
0x00	char	kind: I item, M monster, P prop, B BSP placement (TOL), F foliage (FOL, random prop 1-4/0xE7 or 0xE4-0xE6), 0 deleted	high
0x01	char[3]	ASCII decimal record number in D6ITEM / D6MONS / D6PROP	high
0x04	f32[3]	position (BOL: BSP-relative)	high
0x10	f32[3]	rotation (1024/turn); B: BSP size	high
0x20	u8	M: entry flags (1 ready anim, 2 dead, 4 faded-in, 8 no ground snap, 0x10 hold/ambush until released, 0x20 no hold trigger, 0x40 hold in dungeon); B: BSP number; P (mines): group	high/med
0x21	i8[3]	B: tile offsets to BSP origin; P/I: [2] (+0x23) start inactive	med
0x30	char[16]	B: BSP base name (tcrypt -> tcrypt.bsp.bol)	high

Monsters with flag 0x10 wait for RELEASEMONGROUP (or proximity hold radius 4096).

4. Global databases (record 0 = count header, record i at i*size)

Field-level layouts, enums, limits and how to add records: see databases.md (this section is the short summary).

4.1 D6MONS.DAT 0x154 (MonRec_Load_, monster.c) - 372 records

off	type	meaning	conf
0x00	char[20]	name	high
0xA2	i16	treasure record A (D6TREAS)	high
0xA4	i16	treasure record B	high
0xB0	i16	graphics/model index for LoadMonsterGFX_	high
0x132	u8	bit 8: may use hold/ambush	med
0x14E	i16	D6MONSND record (0x B8 bytes each)	high
rest		stats/resists/attacks - kept raw, not decoded	-

4.2 D6ITEM.DAT 0x11C (LoadItems_, deep6.c; count <= 999) - 857 records

name at 0 (high), price i32 at 0x58 (high), flags u8 at 0x98 (0x10 = charges) (med), model index i16 at 0x114 into the compiled item model table _ItemMDLData (stride 0x51, item/<name>.mdl) (high). Other fields raw.

4.3 D6PROP.DAT 0x38 (LoadProp_, LoadPropObject_) - 238 records

name char[24] at 0 (high); 0x18 i32 model index into compiled prop table (prop/<name>.mdl, < 250) (high); 0x1C i16 object class (med); 0x20 flags: 1 switch, 2 custom scale (0x24), 4 collide, 8 follow terrain, 0x10 container/chest, 0x20 snap in BSP, 0x40 animated texture, 0x80 animated (high/med); 0x21 render flags (med); 0x28 f32 (>0 disables ground adjust) (low); 0x30 i16 ambient sound, 0x34 i32 its parameter (high).

4.4 D6NPC.DAT 0x24 (LoadNPC_, LoadNPCNames_) - 137 records

name char[24] (high) + u32/4xu16 copied into the NPC runtime struct (low).

4.5 D6TREAS.DAT 200 bytes (SetChestTreasure_, MonRec_Load_) - 236 records

10 entries x 20 bytes: i16 type, i16 chance%, i32 a, i32 b, i32 c, i16 pA, i16 pB. type 1/2: item = DiceRoll(a,b,c); type 3: item from treasure lists a/b/c with chances pA/pB (else c); type 4: gold/exp DiceRoll(a,b,c). (high)

4.6 D6TRLIST.DAT 0x3C (LoadTreasureLists_) - 62 lists

name char[20]; 10 x (i16 first, i16 last) item ranges (last 0 = single item). (high)

4.7 D6HELM.DAT (InitHelmData_, GetHelmData_)

u32 nModels (24), u32 nHelms (must be 16), i16 modelIdx[128]; then nModels*16 records of 0x26 at 0x108 + (model*16+helm)*0x26 (helmet attachment per PC model). Record contents raw (low).

4.8 D6SMITnn.DAT shop inventory (townsmit.c) - high

0x800 bytes = 256 x (u16 item, u16 pad, i32 count), then u32 CheckSum_ of the 0x800 bytes (algorithm in d6_checksum, verified). Same layout for D6MAGE/PAWN/ DOJO/YARD/TMPL/HALL/TVRN nn. Created at runtime from _gSmitInitInv if missing - the copy in the install dir is runtime-generated (Oct 2 timestamp), as are D6SEG00.GAM and D6ARCHIV.DAT.

4.9 emitters.dat (LoadEmitterDesc_, particle.c) - med

u32 n (31), n x 0x80 descriptors (no dummy record). name at 0; +0x74 anim/FXC index, +0x78 alt sprite, +0x7C sprite index (resolved at load); +0x60 frame count. Rest raw.

5. Event script: EVENTS.DCL + EVENTS.COD

5.1 EVENTS.DCL (LoadEventDCL_) - high

Game part (55300 bytes):

```
u32 n (115)
u32 codeoff[1024] entry offset in EVENTS.COD
char name[1024][42] "@OPENWCL0SE" ...
u32 firstparam[1024] index of first parameter name
u32 nparams[1024] (byte used -> trigger+0x33)
```

Editor part (ignored by the game, decoded):

```
u32 descoff[1024]; u32 poolsize; char pool[poolsize] one-line descriptions
u32 nparamnames (0x253); char paramname[2048][42] "D00R", "SPEED", "!STATE", ...
```

Parameter names of event i = paramname[firstparam[i] .. +nparams[i]]. A leading ! marks an index into another table (state, switch, bound, trigger).

5.2 EVENTS.COD bytecode (ExecuteEventCode_, ProcessEventQueue_) - high

Flat code; routine i starts at codeoff[i]. Instruction:

```
u16 opcode bit15 = "queued" op (runs over time in ProcessEventQueue_)
i16 n operand count
i16 arg[n] each = index into the trigger's params[] (value = params[arg])
u32 target only for jumps and queued ops (absolute COD offset;
queued ops resume at target when done, see QEventDone_)
```

Every routine decodes cleanly to the next routine (verified). Opcodes (n = observed operand count):

op	mnemonic	n	semantics
01/02	END / END2	0	stop
03	GOTO	0	jump
04	IFSTATE	1	jump if WState[a0]

05 Q	OPENDOOR	4	door entity,speed,navpnt(unblocked),sfx
06 Q	CLOSEDOOR	4	same
07 Q	DELAY	1	ms
08	SETSTATE	2	WState[a0]=a1
09	TOGGLESTATE	1	
0A/0B	SETSWITCH/TOGGLESWITCH	2/1	via linked switch ring
0C/0D	ENABLE/DISABLETRIG	1	
0E/0F	ENABLE/DISABLEBOUND	1	
10/11	ENABLE/DISABLESWITCH	1	
12	CREATEITEM	5	item,x,y,z,bsp
13-16,18,1A,1D,1F,20	no-op	0	
17	PCBLESSING	3	type,value,questflag (shrine/pool rewards, abilities, cures)
19	SETWEATHER	1	
1B	GENMONSTER	7	monrec,x,y,z,bsp,init,maxcount (encounter group)
1C	TEXTMSG	2	TEXTPAK id, time
1E	IFHASITEM	1	jump
21/22	GOTOBSP/GOTOBSPTErr	3	bsp,destnav,lastnav
23/24 Q	OPENDBL/CLOSEDBL	5	doorA,doorB,speed,nav,sfx
25	RELEASEMONS	4	object ids leave hold mode
26	EFXATOBJ	5	efx,sfx,spell,obj,yoff
27/28	MAGICPOOL/MAGICFIRE	1	
29	TWIDDLEPROP	3	obj.type,value
2A	LOADSEGMENT	5	seg,bsp,switch,status,bound
2B	IFALLINBOUND	1	jump
2C	IFUSINGITEM	1	jump
2D	DAMAGEPC	2	dmg,potion
2E	SHOWTEXT	1	
2F Q	MOVEENTITY	7	
30/31	ZEROUSEITEM/USEITEMCHARGE	1	
32/33	IFNOTOCCUPIED/IFOCCUPIEDBY	1/2	WOccupied slots, jump
34/48	PLACEOCCUPY(Y)	8/9	item,x,y,z,bsp,slot,rot,flag(,yoff)
35	CLEAROCCUPIED	1	
36/4B	EFXFROMTO	9	
37	CALLEVENT	1	run trigger
38	SETNOTSTATE	2	
39/45	MOVETONAV(SPD)	3	
3A	COMBO4	7	4 states as decimal digits == value -> set state
3B	TELEPORT	4	x,y,z,bsp
3C	IFOBJINBOUND	2	jump
3D	ROTATETONAV	2	
3E/3F	SWITCHFROMSTATE/STATEFROMSWITCH	2	
40	ANIMOBJ	2	
41	SWAPOBJPOS	2	
42	EFXATPC	5	
43	IFSWITCHON	1	jump
44	PLAYSFX	1	
46	HIDEPOLYS	3	
47	CHANGEITEMS	2	
49	TELEPORTOBJ	5	
4A	LADDER	4	
4C	SPAWNMONSTER	8	monrec,x,y,z,bsp,dir,radius,init
4D	SETNAVFLAG	3	
4E	EFXPOINTS	10	
4F	ENTERTOWN	1	0..2
50	ITEMTOITEM	2	
51	IFEQUIPPED	1	jump
52	MOVEPROP	5	
53	SETQFLAGIF	3	
54	ENDGAME	1	movie
55	IFMONALIVE	1	jump
56	PROPWALK	6	

Disassembler: --disasm (uses DCL param names as operand names). Example:

```
@LOPENWCLOSEW params D00R, SPEED, DELAY, !STATE, NAVPNT, SFXREC
002e: Q:OPENDOOR D00R, SPEED, NAVPNT, SFXREC -> 003e
003e: Q:DELAY DELAY -> 0048
0048: Q:CLOSED00R D00R, SPEED, NAVPNT, SFXREC -> 0058
0058: IFSTATE !STATE -> 0066
0062: END2
0066: Q:DELAY DELAY -> 0070
```

0070: GOTO -> 002e

New script routines can be added by appending code and a DCL entry (max 1024); an assembler is straightforward from this table.

5.3 Assembler

formats/d6events.py disassembles every routine to re-assemblable text (labels instead of offsets, operands as parameter names) and assembles it back byte for byte (--selftest). EventSet.set_event() adds or replaces an event: the code is appended to EVENTS.COD, parameter names to the end of the name table, and the DCL editor part (descriptions, parameter names) is rewritten.

6. Text

- **D6STRING.DAT** (LoadStrings_, StringAddress_, d6string.c) - high. u32 n; n x (u32 id, u32 file offset) sorted by id (binary searched); NUL strings follow.
- **TEXTPAK.000** (OpenTextMessages_, textmsg.c) - high. u32 n; n x (i32 id, u32 offset, u32 size); text blobs, @ = paragraph break. TEXTMSG param MSG.
- **NPCDATA.PAK / GMDATA.PAK** (LoadNpcPakHead_, ReadNpcPakFile_, LoadGmPakHead_) - high for container. Header = (u32 offset, u32 size) slots; header length = first offset. NPCDATA: 481 slots = lexicon, 160 x code, 160 x strings, 160 x keywords (indexed by NPC id); GMDATA: 97 slots = lexicon, 32 x code, 32 x strings, 32 x a response table the game never uses. The bytecode (two VMs, npc.c and guild.c) is decoded in npc.md (formats/d6npc.py).
- ***.ldi** (language.ldi, string.ldi, string_e.ldi, default.ldi) - low. Encrypted with a fixed per-position XOR keystream (same key for all files; plaintext is a #-commented "global_strings { ... }" text). Not referenced by deep6.exe (launcher data). Key not recovered (common LCGs ruled out); passthrough only.

7. D6ARCHIV.DAT (save archive, SaveArchive_/RestoreArchive_, segwrite.c) - high

Header 0x2B4 = 173 u32: [0] 0x140 version, [1] warship flag, [2] GM unique bits, [3] fog/marker blob offset, [4] ROSTER offset, [5] ROSTER size, [6] D6WORLD offset, [7] D6WORLD size, [8..55] D6SEGnn.GAM present/offset/size x16, [56..100] JOURNAL.nnn x15, [101..172] 24 shop files (D6SMIT/MAGE/PAWN/DOJO/YARD/TMPL/HALL/ TVRN 00..02). --list / --extract supported. This is save state, not content.

8. Models

- **models/*.lst** - first line count, then \$M/\$I/\$P + file name. The game itself uses compiled tables (_ItemMDLData, prop table _PropMDLData at 0x5E27D5) and only chardemo reads model.lst. (high for format, med for role)
- ***.mdl** (Model_Read_, model.c) - structure walker, high:

```
" LDM" magic; u32 version (9|10, bit24 = has frame bitmap); u32 parttable offset;
u8 paltype: 0/3 -> 32x256 u16 shade palette (16384 B), 1 -> 768 B RGB, 2 -> char[64]
    shared palette name (models/refpal/);
u16 nframes; [u8 framebits[nframes]]; anim table 256 x (u16 start,u16 end,u16 ?);
u8 nattach; per attach: u16 nf, nf x 12 (translation), nf x 36 (3x3 matrix);
u8 nparts; parttable: u32 offset per part. Part:
    u32 ?, u32 sprite_section_off, u32 ?, u32 flags, u32 nverts;
    mip levels (1 for v9, 4 for v10): u32 nuv, nuv x 6; u32 npoly, npoly x 4, npoly x 1;
    u8 ?, u16 nframes, u32 npaths, u16 pathlen[npaths], u32 framesize;
    per frame: [framesize bytes: per path 6 + (len-1)*3 packed verts] (omitted when
    framebit==0), then 24 bytes bbox;
    u8 ntex, u32 texoffset[ntex] -> each: u32 w, u32 h, w*h 8-bit texels;
    at sprite_section_off: u8 n, n x u32.
```

728/728 files walk with full coverage; PC models and a few others carry unreferenced tail data (kept). Vertex packing, UVs, animation and the writer are in models.md (formats/d6model.py, formats/d6mdllo.py).

9. Sanity check: spoke 00 (Valeia wilderness), /tmp/data/area00.txt

- 238 TOL records: 14 terrain BSPs (tcrypt, torcave, ttemple, ttmpcave, ttoada/b, tgatea/b/c = Valeia gates, TRUINSA..F ruins, 152 props, 70 monsters.
- Signposts (P030) at (218568, 89020) - 5.6k units from the default entry (217600, 83456) - plus four more along the roads.
- Trolls: Troll Guni/Mangu groups (#104-111, #144-148) held in ambush (flag 0x01/ 0x10) and released by triggers T56/T57/T72 via bound areas B44/B45/B57 (state 33); T51 @MAKEMONSTER Troll Guni x5 at (241963, 103623) fires from bound B42 just east of the gate road. Nearest placed monsters to the entry are Worgur packs released by T48/T49 (state 32, bound B38).
- B49/B50/B51 (PCs near tgatea/tgateb/tgatec) set states 34/35/36; switches W5/W7/W8 on the gate entities call T34/T38/T39 -> @ENTERTHETOWN (T63..T65).
- Graveyard by the crypt (tcrypt): skeletons with hold flag, crypt doors T1 (state 1, lever W6 "Graveyard Guardian"), crypt text T2/T4 from bound B1/B3 (BSP-relative boxes land at x 265728.., z 148736.., next to tcrypt).
- 12 treasure chests with D6TRAP entries (treasure records 83..96).

10. Open items listed when this document was started, and where they went

1. Terrain heightfield, BSP geometry, nav points, lights: terrain.md, levels.md.
2. Dungeon BSP placement (hard-coded in LoadSpokeSegment_): decoded, d6level.SPOKE_TABLE (levels.md section 11).

3. Monster/item/prop records: field by field in `databases.md`.
4. NPC/GM dialogue bytecode: `npc.md`, `formats/d6npc.py`.
5. .idl key: still not recovered; the game does not read these files.
6. MDL vertex packing, UVs, animations, writer: `models.md`.
7. D6SEGnn.GAM supersedes the placement files once a spoke has been visited; editors must move it away (d6edit offers this on save).
8. Model tables compiled into `deep6.exe`: `databases.md` section 9 and `models.md` section 3; d6edit can point a slot at a new file.

Chapter 6. Deep6 content databases - record layouts

Source: docs/formats/databases.md

Wizards & Warriors (2000, deep6.exe). Field-level reverse engineering of the global content databases, aimed at a modding SDK that edits monsters, items, props and treasure and adds new records. Companion to data.md (container formats) and the reference implementation formats/d6data.py (FIELDS lists mirror these tables).

Conventions

- All integers little-endian. Offsets are into the on-disk record (= in-memory record, the game reads records verbatim).
- Ghidra pattern `*(int *) (p + X) >> 0x10` is the i16 at `X + 2`; all offsets below are already corrected for that.
- Confidence: **high** = read from the consuming code and consistent with data; **med** = from code but meaning partly inferred, or meaning from data only with a code reference; **low** = guess from data distribution.
- "Source" names the function(s) in `re/decomp/*.c` where the field is consumed.
- Item globals: `_Item = 0x66E280`, record `i` at `_Item + i*0x11C` (so `DAT_0066e2ac` = item field `0x2C`). Monster runtime: `_gMonster = 0x628370`, stride `0x2A8`, `+0` = pointer to the loaded `D6MONS` record.
- Enumeration names come from `D6STRING.DAT` (string id given) - they are the names the game itself shows.

Database file layout (all RecordTables below): record 0 is a header whose first u32 is the record count; record `i` (1-based) is at file offset `i * SIZE`. The game always lseeks to `index * SIZE`, so the index is the identity of a record everywhere (object lists `M###/I###/P###`, scripts, treasure dice, shops, saves).

1. D6ITEM.DAT - items, 0x11C bytes, 857 records

Loader: `LoadItems_` (deep6.c) reads the u32 count (error if > 999) and then **all** records into `_Item[1..count]` at startup. Display: `ItemBoxText_` (pcinvent.c) is the best single reference - it prints most fields with `D6STRING` format strings 2701..2813.

An *inventory instance* (0x56 bytes, PC inventory at `pc+0x296`, 78 slots) is created by `ItemToInv_` (townsmit.c) from the record: `+0` i16 item, `+2` i16 durability (DiceRoll of durdice, or max for shop items), `+4` u8 flags (1 ?, 2 cursed, 4 identified, 8 ?, 0x10 invoked), `+6` i32 charges/quantity, `+0xA` name, `+0x20` spell, `+0x22` damage[3], `+0x2E` dmgextra, `+0x32` resist[16], `+0x42` enchant, `+0x44` hitbonus, `+0x46` unk108, `+0x48` ac, `+0x4A` regen, `+0x4C` tough. Enchanting/blessing changes the instance copy; `ItemBoxText_` prints instance-minus-record differences ("Dam+n", "AC+n").

off	type	name	meaning	conf	source
0x00	char[22]	name	identified name; natural attacks: verb phrase, "\$" = target	high	PCItemName_, ItemName_
0x16	char[22]	unidname	name while unidentified ("?Sword?"); natural attacks: attack name	high	PCItemName_, ItemName_ (combat.c), InvName_
0x2C	i16	type	0 Special 1 Weapon 2 Ammo 3 Armor 4 Shield 5 Jewelry 6 Scroll 7 Potion 8 Powder 9 Key 10 Book 11 Food 12 Gold 13 Storage 14 Light 15 Instrument	high	ItemBoxText_, CheckEquipItem_, GetValidEquipSlots_
0x2E	i16	subtype	Weapon: 0 hand 1 throwing 2 ranged; Ammo: 1 arrow 2 bolt; Armor: 0 body 1 leg 2 head 3 hand 4 foot 5 helm(D6HELM); Jewelry: 0 ring 1 amulet 2 bracelet 3 other	high	ItemBoxText_, CheckEquipItem_
0x30	i16	icon	inventory icon (exe_Icondata, 246 entries: itemicon<name>.bmp + grid w,h)	high	ItemBoxReq_ (icondata w,h), CheckEquipItem
0x32	i16	weight	weight in 0.1 lb	high	ItemBoxText_, CarryLoad_
0x34	u8	invoketype	book/tome effect: 1 ability+1 2 skill+100 3 give trait 4 learn spell	high	InvokeTheItem_
0x35	u8	invokeparam	ability 0..7 / skill 1..32 / trait 0..65 / spell 1..105	high	InvokeTheItem_, ItemsRestricted_
0x36	i16	invokeconsume	!=0 -> item destroyed after invoking	high	InvokeTheItem_
0x3C	u8	usemode	0 not usable 1 usable 2 give/show to NPC 3 lock pick (bonus = minstr)	high	ItemsUsable_, OpUse_, ExecFight_
0x4C	i32[3]	durdice	durability DiceRoll(n,s,b); max = n*s+b; 0 = never wears	high	ItemToInv_, TryToRepairItem_, ItemBoxText_, ArmorDamage_
0x58	i32	price	base price (per unit for stackables)	high	GetSaleItemPrice_, NPC trading, traps (luck reroll keeps pricier)
0x5C	i16	spell	spell cast on use (ItemSpell_ id; >=1000 special power)	high	ItemToInv_ (-> inst +0x20), OpUse_, ItemBoxText_
0x60	i32[3]	chargedice	charges / quantity at creation DiceRoll(n,s,b) (GetItemCharges_)	high	GetItemCharges_, GetSaleItemPrice_
0x6C	i32	readtext	TEXTPAK id shown by the READ option (letters, inscriptions); 0 = none	high	ItemBoxOpList_, ItemBoxReq_ (READ)
0x70	u32	restrict	NOT usable by: bits 0-9 clans, 10-24 roles, 25-26 gender, 27-29 alignment Evil/Neutral/Good (IsItemUsableByPC_)	high	IsItemUsableByPC_
0x74	u8	cursed	1 = cursed (sticks when equipped)	high	CheckPCItemCurses_, ReEquip_
0x75	u8	curseeffect	1 = drains 1 HP every cursetick ticks	med	CheckPCItemCurses_
0x76	i16	cursetick	curse period	med	CheckPCItemCurses_
0x96	u16	unk96	unknown (1/2 on 35 items)	low	data only
0x98	u16	flags98	1 quest 2 always identified 0x10 stackable 0x20 off-hand 0x40 two-handed 0x80	high	many (see enum)

			missile 0x200 show model 0x400 random charges 0x800 not sellable 0x1000 unstealable 0x2000 not enchantable 0x4000 0x8000 off-hand only		
0x9C	i32	range	weapons: max range (melee 1280, bows 6144); helms: D6HELM helm index; body armour: PC costume (MyPCModel_)	high	CombatWeaponRange_, InWeaponRange_, GetHelmData_, MyPCModel_
0xA0	i32	minrange	minimum range for missile weapons	high	ExecFight_, EvaluateWeaponInSlot_
0xA4	i32	unka4	unknown (1280 on 20 items)	low	data only
0xA8	i16	minstr	minimum Strength; lock picks: pick bonus	high	ItemBoxText_ ("Minimum Strength"), OpUse_ (lock pick)
0xAA	i16	ammotype	launcher: ammo subtype required (1 arrow 2 bolt)	high	EvaluateWeaponInSlot_, PCReQuiver_, ReEquip_
0xAC	u8	atkflags	0x40/0x80 unarmed kung-fu attack (uses Kung Fu skill / monster kungfu)	med	OpFight_, ExecFight_, ExecMissile_
0xAD	u8	atkflags2	2 natural attack 4 alt anim 8 thrust anims 0x10 damage scales with level 0x20 no use/charges	med	OpFight_, ExecFight_, ItemIsUsable_
0xB0	i32[3]	damage	damage DiceRoll(n,s,b): shown "Damage n+b - n*s+b"	high	ItemToInv_ (-> inst +0x22), ItemBoxText_
0xBC	i16[2]	dmgextra	copied to instance +0x2E/+0x30 (0..4)	low	ItemToInv_ (-> inst +0x2E)
0xC0	i32[8]	specials	4 x (u8 type, u8 chance%, i16 a, i32 b): type = Sleep..Disease; duration (rand(a)+b) s; Poison/Drain use a,b; see specials_list()	high	SpecialAttacks_, ItemBoxText_
0xF0	u8[16]	resist	resistance bonus % per resist type (Magic..Mavin)	high	ItemToInv_ (-> inst +0x32), ItemBoxText_
0x104	i16	enchant	enchantment level (+N)	high	ItemToInv_ (-> inst +0x42), ItemBoxText_ ("Enchant")
0x106	i16	hitbonus	to-hit bonus	high	ItemToInv_ (-> +0x44), ItemBoxText_ ("Hit")
0x108	i16	unk108	negative on heavy armour (penalty, copied to instance +0x46)	med	ItemToInv_ (-> +0x46)
0x10A	i16	ac	armour class bonus (shields: "Rating", jewelry: "AC Protection")	high	ItemToInv_ (-> +0x48), SetMonEquipment_, ItemBoxText_
0x10C	i16	regen	regeneration bonus	high	ItemToInv_ (-> +0x4A), ItemBoxText_ ("Regen")
0x10E	i16	unk10e	unknown	low	data only
0x110	i16	tough	toughness % (damage resistance of the item itself)	high	ItemToInv_ (-> +0x4C), ItemBoxText_ ("Tough")
0x112	i16	costume	PC body/robe texture set (40..43 = robes, PCInRobes_)	high	PCInRobes_, PCMSetCostume_
0x114	i16	model	index into exe _ItemMDLData (item/.mdl), must be < 240	high	LoadItemObject_, LoadAttachItemObject_, PCMLoadAttach_
0x118	i16	skill	skill used (1 Sword ... 9 Shield, see SKILLS)	high	ItemBoxText_, ExecFight_, ExecMissile_
0x11A	u16	enchantmask	allowed enchantments bitmask: Zap Flamestrike Iceball ProFire Prolce ProMagic Hit Damage Toughness Armor Regenerate Special Recharge	high	GetItemEnchantFlags_, BasicEnchantOK_, EnchantText_, BlessText_

Bytes not listed (0x38..0x3B, 0x3D..0x4B, 0x78..0x95, 0xE0..0xEF, 0x100..0x103, 0x116..0x117) are zero in all 857 records, except byte 0x94 (= 6 in a single record). Unlisted monster bytes (0x3C..0x4B, 0xA0..0xA1, 0xA6..0xA7, 0x11E..0x11F, 0x122..0x127, 0x133..0x13B) and prop bytes (0x23, 0x32..0x33) are zero in all records.

1.1 Enumerations

- **type** [9400]: 0 Special, 1 Weapon, 2 Ammo, 3 Armor, 4 Shield, 5 Jewelry, 6 Scroll, 7 Potion, 8 Powder, 9 Key, 10 Book, 11 Food, 12 Gold, 13 Storage, 14 Light, 15 Instrument. Equipable = 1..5 and 14 (ItemIsEquipable_); Instrument needs Music skill or trait 65.
- **subtype**: Weapon [9500] 0 Hand, 1 Throwing, 2 Range (launcher); Ammo [9550] 1 Bow (arrow), 2 Crossbow (bolt); Armor [9600] 0 Body, 1 Leg, 2 Head (coif), 3 Hand, 4 Foot, 5 Head (helm, uses D6HELM); Jewelry 0 ring, 1 amulet, 2 bracelet, 3 other (off-hand); Storage: resource kind (ItemIsResource_ returns subtype+1).
- **Equip slots** (CheckEquipItem_ / GetValidEquipSlots_, inventory slot numbers): 0x3C helm (armor 5), 0x3D head (2), 0x3E body (0), 0x3F legs (1), 0x40 feet (4), 0x41 hands (3), 0x42 main hand (hand weapon), 0x43 off-hand (shield, light, jewelry 3, flag 0x20/0x8000 weapons, 2nd hand weapon), 0x44-0x46 ammo/throwing, 0x47 amulet, 0x48/0x49 rings, 0x4A bracelet, 0x4B-0x4D belt (only 1x1 icons of types 0,6..11). Two-handed (flags 0x40) blocks the off-hand slot.
- **flags98** (u16): 0x0001 quest/plot item, 0x0002 always identified, 0x0010 stackable (charges = quantity), 0x0020 off-hand weapon, 0x0040 two-handed, 0x0080 missile special collision, 0x0200 shows a model on the wearer (LoadAttachItemObject_), 0x0400 random charges, 0x0800 not sellable, 0x1000 cannot be stolen, 0x2000 cannot be enchanted, 0x4000 helm/armour equip conflict check, 0x8000 off-hand only.
- **restrict** (u32, set bit = may NOT use): bits 0-9 clans Human, Elf, Dwarf, Gnome, Pixie, Omphaaz, Whiskah, Gourk, Ratling, Lizzord [1300]; bits 10-24 roles Warrior, Wizard, Priest, Rogue, Ranger, Bard, Samurai, Paladin, Barbarian, Monk, Ninja, Warlock, Assassin, Zenmaster, Valkyrie [1400]; bits 25-26 gender (male, female); bits 27-29 alignment Evil/Neutral/Good (PC alignment/33). Verified: Long Sword excludes Wizard Priest Rogue Bard Monk Warlock; Ninja Cowl only Ninja+Assassin.
- **skill** [1800]: 1 Sword 2 Axe 3 Mace 4 Pole&Staff 5 Dagger 6 Bow 7 Throwing 8 2nd Weapon 9 Shield 10 Kung Fu 11

Sorcery 12 Spiritcraft 13 Suncraft 14 Mooncraft 15 Vinecraft 16 Stonecraft 17 Fiendcraft 18 Leadership 19 Athletics 20 Scout 21 Traps&Locks 22 Pickpocket 23 Stealth 24 Forge 25 Artifacts 26 Enchants 27 Blessings 28 Gallantry 29 Prowess 30 Deathstrike 31 Incantation 32 Music.

- **specials** type [9900]: 1 Sleep 2 Stun 3 Knock-Out 4 Paralyze 5 Fear 6 Blind 7 Poison 8 Stone 9 Death Strike 10 Drain (mana, amount a) 11 Insane 12 Silence 13 Break 14 Destroy 15 Disease. Effect time = (Random(a) + b) * 1000 ms, resisted via ResistValue_.
- **resist** order [9950]: Magic, Fire, Mind, Paralysis, Death, Petrification, Cold, Wind, Earth, Poison, Elements, Dispel, Silence, Light, Charm, Mavin (same order as the monster resist array and PC +0x4E).
- **enchantmask** [9300]: bit 0 Zap, 1 Flamestrike, 2 Iceball, 3 Pro Fire, 4 Pro Ice, 5 Pro Magic, 6 Hit, 7 Damage, 8 Toughness, 9 Armor, 10 Regenerate, 11 Special, 12 Recharge (weapons typically 0x19C7, armour 0x0338).
- **invoketype**: 1 ability[param] +1 (max 24), 2 skill[param] +100 (if level < 12), 3 give trait[param] (0..65, names [2400]), 4 learn spell[param] (1..105). 103 books use 4.
- **usemode**: 0 none, 1 generic use, 2 give/show to the targeted NPC (SetNPCAct_), 3 lock pick (PryTheBox_ with bonus minstr + Traps&Locks skill).

1.2 Data sanity

Long Sword: damage 1d4+2 (3-6), weight 8.0 lb, min STR 10, range 1280, skill Sword, price 100, durability 2d10+80. Long Bow: subtype 2, ammotype 1 (arrows), range 6144, minrange 1024, skill Bow. Plate Mail: AC 12, unk108 -6, weight 20.0 lb. Natural attacks (items 1-4, 500.. "swings at \$", "bites at \$") are type 1 subtype 0 items equipped by monsters through MonsRec.equip.

2. D6MONS.DAT - monsters, 0x154 bytes, 372 records

Loader: MonRec_Load_ (monster.c) - **on demand**, lseek(index*0x154), cached per area in _gMonRec[64] (error "MAX MONRECS EXCEEDED" on the 64th distinct record; cache cleared by InitMonsters_/DoneMonsters_). The header count is **not** read or checked. It zeroes the runtime pointers 0xA8/0xAC/0x150 and loads treasureA/B (D6TREAS) and the sound record (D6MONSND, preloading its non-resident SFX). MonsterEntry_ then calls LoadMonsterGFX_(gfx), InitMONSToMonster_ (stats -> runtime struct, difficulty scaling), MakeMonInventory_, MakeMonAttachList_ + SetMonEquipment_.

Runtime monster struct _gMonster[240] (stride 0x2A8, slots 0-5 are the PCs): +0 record pointer, +0x1CC abilities[8], +0x1DC resist[16], +0x1FC/+0x1FE HP, +0x202 hit, +0x204 parry, +0x206 armor, +0x208 shield AC (sum of equipped shields' ac), +0x20C attack delay, +0x24C mana[6], +0x258 xp, +0x114 npc id, +0x130 i16 inventory[4] (treasureA items positive = dropped, treasureB items negative = never dropped).

Difficulty (_gMonsterDifficulty, skipped for flags132&0x10 critters and NPCs): Easy HP*3/4 (if > 3), hit/parry/armor -2, atkdelay*4/3, xp*4/5; Hard HP*3/2, +2, atkdelay*3/4, xp*5/4, speed scaled.

off	type	name	meaning	conf	source
0x00	char[22]	name	display name (MyName_ returns the record pointer); 21 chars + NUL	high	MyName_ (combat.c)
0x16	i16	mclass	creature class: 0 animal 1 humanoid 2 undead 3 undead(alt) 4 vampire/seductress 5 lycanthrope 6 demon/magical 7 dragon/reptile 8 insect/serpent 9 slime/fungus 10 plant 11 fish 12 invulnerable(boss phase) 13 construct 14 spirit/non-combat 15 object (statue, cart). 12/14/15 are invulnerable or passive	med	InitMONSToMonster_, ExecFight_, MonsterInvulnerable_, DivvyExp_
0x18	i16	npc	NPC id (D6NPC record / NPCDATA.PAK slot); !=0 -> talking NPC (NpcInitNPC_)	high	MonsterEntry_ -> NpcInitNPC_, LoadMonsterModel_
0x1A	i16	gender	0 male 1 female (NPC script GENDER, PC +0x18 equivalent)	high	EvalExpr_ (npc.c)
0x1C	i16	clan	clan/race 0..9 (Human..Lizzard); 6/7/8 = furred, no hair twiddling	med	EvalExpr_, SetMonEquipment_
0x1E	i16	role	role/class 0..14 (Warrior..Valkyrie), read by NPC scripts	med	EvalExpr_
0x20	i16	alignment	0..99 (/33: 0 Evil 1 Neutral 2 Good); default 50	high	EvalExpr_
0x22	i16	unk22	unknown, 0 or 50	low	data only
0x24	i16	weight	mass for knock-back / pushing (pushy.c object_weight_, wcoll.c)	high	object_weight_ (pushy.c), wcoll.c
0x26	i16	unk26	unknown (128..2048, probably a radius); not read by code found so far	low	data only
0x28	i32	speed	movement speed (-> float runtime +0x70; x constants on Hard)	high	InitMONSToMonster_
0x2C	i16[8]	abilities	STR INT SPI DEX AGI FOR WIL PRE (0..25); FOR*2 = air supply, WIL = spell power	high	InitMONSToMonster_ (-> runtime +0x1CC), GetMaxAirSupply_
0x4C	i16[16]	resist	resistance % per type: Magic Fire Mind Paralysis Death Petrification Cold Wind Earth Poison Elements Dispel Silence Light Charm Mavin (ResistValue_)	high	InitMONSToMonster_ (-> +0x1DC), ResistValue_
0x6C	i32[3]	hpdice	hit points = DiceRoll(count, sides, bonus); count is also the monster LEVEL (ResistValue_, TestForSummoning_, level-scaled weapons)	high	InitMONSToMonster_ (DiceRoll), ResistValue_, TestForSummoning_
0x78	i16	unk78	unknown (0..30000, scales with toughness)	low	data only
0x7A	i16	hit	to-hit rating (runtime +0x202 = PC +0x11C); -2 Easy, +2 Hard	high	InitMONSToMonster_
0x7C	i16	parry	parry/defence rating (runtime +0x204); -2 Easy, +2 Hard	high	InitMONSToMonster_
0x7E	i16	armor	armour rating (runtime +0x206); -2 Easy, +2 Hard	high	InitMONSToMonster_
0x80	i16	kungfu	Kung Fu level used by natural punch/kick attacks (item flags 0xAC&0xC0)	high	ExecFight_, ExecMissile_, OpFight_
0x82	i16	bloodtype	hit effect: 0 red blood (can be vampire-drained) 1/2/3 other effects (0x22/0x23/0x2A)	high	DamageMonster_, VampireSuckBlood_
0x84	i16	npcanimA	extra animation set id loaded / NPC talk mode anim (GetNPCModeAnim_)	med	LoadMonsterModel_, GetNPCModeAnim_
0x86	i16	npcanimB	second NPC mode animation set id	med	LoadMonsterModel_, GetNPCModeAnim_
0x88	i16	shadow	shadow texture (-1 = no shadow) (SetMonsterShadow_)	high	SetMonsterShadow_

0x8A	i16	reach	melee engage distance, <1 -> 1536 (MonsterUpdateMON_)	high	MonsterUpdateMON_, MonsterAICheckLOC_
0x8C	i16	unk8c	unknown (0/256/512)	low	data only
0x8E	i16	hitsound	per-monster hit sound/variant used by ExecFight_ (0..5)	low	ExecFight_, ExecMissile_
0x90	i16	specialA	special power A: <1000 item-spell id (ItemSpell_), >=1000 monster power (breath etc.); used with chance specApct	high	MonsterCombatAI_
0x92	i16	specialB	special power B (chance specBpct)	high	MonsterCombatAI_
0x94	i32	specrange	range of steal / special powers (0 -> 8192, steal 1536)	high	MonsterCombatAI_
0x98	i32	modelheight	overrides model collision height (model +0x69C); <0 -> 0	high	LoadMonsterModel_
0x9C	i32	xp	experience award (x4/5 Easy if >=7, x5/4 Hard)	high	InitMONSToMonster_ (-> +0x258)
0xA2	i16	treasureA	D6TREAS record rolled into the drop inventory (positive ids, dropped on death)	high	MonRec_Load_, MakeMonInventory_
0xA4	i16	treasureB	D6TREAS record for the remaining slots (stored negated, never dropped)	high	MonRec_Load_, MakeMonInventory_
0xA8	u32	rt_treasA	runtime: pointer to loaded treasure A (zeroed by MonRec_Load_)	high	MonRec_Load_, DamageMonster_ -> AddMonTreasure_
0xAC	u32	rt_treasB	runtime: pointer to loaded treasure B	high	MonRec_Load_
0xB0	i16	gfx	model index into exe _MonMDLData (17..120; 0..16 = PC bodies); also MONSND default	high	MonsterEntry_, MyMONModel_, LoadMonsterModel_
0xB2	i16[5]	overlays	skin overlay texture ids (low byte used), applied in order 1,4,2,0,3; [2]!=0 -> no hair	med	LoadMonsterModel_ (ovlyorder), SetMonEquipment
0xBC	u32[16]	equip	16 x (u8 kind, u8 chance%, i16 item): kind 1 main weapon 2 off-hand 6 missile, others = attachments; see equip_list()	high	MakeMonAttachList_, GetMonWeaponType_, ResetMonsterWeapon_
0xFC	i16[6]	mana	mana per realm Spirit Sun Moon Vine Stone Fiend (100 default, 9999 = endless)	high	InitMONSToMonster_ (-> +0x24C), ManaFood_
0x108	u8[16]	spells	spell book: up to 16 spell ids (ItemSpell_) chosen by MonsterAISpell_	high	MonsterAISpell_
0x118	u8	meleepct	!=0 -> fights with weapons (MonsterCombatAI_)	high	MonsterCombatAI_
0x119	u8	spellpct	chance % per AI tick to cast from the spell book (range 8192)	high	MonsterCombatAI_
0x11A	u8	unk11a	unknown	low	data only
0x11B	u8	specApct	chance % to use specialA	high	MonsterCombatAI_
0x11C	u8	specBpct	chance % to use specialB	high	MonsterCombatAI_
0x11D	u8	stealpct	chance % to try stealing (OpSteal_)	high	MonsterCombatAI_
0x120	u8	unk120	unknown	low	data only
0x121	u8	flypct	!=0 -> flying monster (fly/land anim 0x1F..0x21, MonsterUpdateMON_)	med	MonsterCombatAI_, MonsterUpdateMON_
0x128	i32	atkdelay	attack recovery time ms (x4/3 Easy, x3/4 Hard)	high	InitMONSToMonster_ (-> +0x20C)
0x12C	i32	unk12c	11 on 31 small critters, else 0	low	data only
0x130	u8	flags130	1 humanoid hair 2 hood/hat hair 4 ? 8 animal 0x10 ? 0x20 rogue 0x40 zombie	med	SetMonEquipment_, SegRead_SetMonAttachments_
0x131	u8	flags131	1 flyer(bat) 2 mount (horse, trolley) 4 translucent 8 swimmer 0x10 eyes 0x20 amphibian 0x40 ship (BSP rot object) 0x80 shipwreck	med	InitMONSToMonster_, LoadMonsterGFX_, SetMonEquipment_, MonsterUpdateMON_
0x132	u8	flags132	1 raft 2/4 big flyer 8 can be placed in "hold" (ambush) mode 0x10 ambient critter (no difficulty scaling, idle anims) 0x20 ? 0x40 unique	med	InitMONSToMonster_, MonsterEntry_, LoadMonsterModel_
0x13C	i32[3]	groupdice	group size = DiceRoll(count, sides, bonus) for encounters/summons	high	SummonMonsters_ (genenc.c)
0x148	i16	companionpct	chance % that an encounter adds a group of companionA/B (GenerateEncounter_)	high	GenerateEncounter_
0x14A	i16	companionA	D6MONS record of the companion group	high	GenerateEncounter_
0x14C	i16	companionB	alternative companion (50/50 when non-zero)	high	GenerateEncounter_
0x14E	i16	soundrec	D6MONSND record (0 -> no sounds); normally = gfx	high	MonRec_Load_
0x150	u32	rt_sound	runtime: pointer to the loaded D6MONSND record	high	MonRec_Load_, GetMonSound_

2.1 Enumerations / notes

- **equip** (16 x 4 bytes): u8 kind, u8 chance%, i16 D6ITEM. Each entry is rolled with its chance; kind 1 = main weapon slot, 2 = second weapon, 6 = missile weapon (GetMonWeaponType_), other kinds index the attachment list (only one item per kind). Natural attacks are ordinary items (e.g. 284 "claws at \$"). 322 records use 01 64 xxxx (kind 1, 100%).
- **mclass** values 2 and 3 select the "bone" hit SFX set and skip experience sharing rules (DivvyExp_); 9/10 select slime/plant SFX; 12 = invulnerable (MonsterInvulnerable_); 13/15 = inanimate (flag bits [5, no AI]; 14 = passive. The names in the table are inferred from the records using each value.
- **flags131** / **flags132** bit meanings are partly inferred from which records set them (e.g. 0x40 only on "Ship O' The Sea", 0x80 only on "Marooned Shipwreck", 132&1 only on "Wooden Raft", 132&0x10 on birds, fish, rats, butterflies).
- **specialA/B**: values >= 1000 are monster powers (1001.. breath/gaze; spell table entries with id >= 1000 in the exe spell table), < 1000 go through ItemSpell_. Summon powers (0x3FB, 0x3FC, 0x402, 0x403, 0x406, 0x407, 0x409, 0x40D-0x40F, 0x416) are limited to level/5+2 (max 6) conjured monsters (TestForSummoning_).
- **gfx**: model number = index into the exe table _MonMDLData (121 entries; 0-16 are PC bodies and use the PC path in LoadMonsterModel_). LoadMonsterModel_ contains ~30 hard-coded per-model animation fix-ups (gfx == 0x44, 0x5A, 0x30, ...): reuse a model number only for a monster that should get the same fix-ups.

3. D6MONSND.DAT - monster sounds, 0xB8 bytes, 120 records (+ header)

Indexed by `MonRec.soundrec` (normally = gfx, one record per monster model; record label holds the model file name). Loaded by `MonRec_Load`; `soundrec < 1` gives an all-zero record (silent monster).

off	type	name	meaning	conf	source
0x00	u32	modelno	model number (editor label, = record index)	high	-
0x04	char[20]	label	editor label, e.g. "SKELETON.MDL"	high	-
0x18	i16[80]	slots	16 slots x (u8 count 0..3, u8 extra, i16 timer (runtime), i16 sfx[3]); slot 0 alert/move 1 idle growl 2 wound 3 death 4 attack, 5.. attack-anim sounds; sfx = MONSOUND.DAT ids; see <code>slot_list()</code>	high	<code>MonRec_Load</code> , <code>GetMonSound</code> , <code>MonsterUpdateMON</code>

Slot use (`GetMonSound(mon, slot)` picks one of count sfx at random): 0 alert/move (played while hunting, timer at +0x1A), 1 idle growl (timer +0x24, every 10-20 s), 2 wound (`DamageMonster_path`), 3 death, 4 melee attack (`OpFight`), 5+ played by `ATQAnimSfx` for special attack animations. Verified on SKELETON.MDL: 68 move, 69 growl, 70 wound, 72 death, 71 attack (MONSOUND.DAT comments). The byte at slot+1 (25/100 on some records) is not read by any code found (low).

3.1 MONSOUND.DAT - SFX table (text)

Read by `sndread.c` at startup: first number = declared maximum (699), then lines `id,"wavname"[,R]` (* starts a comment, -1 ends). ids index `_gMonSoundName[1024]` (hard limit 1024); R = resident (preloaded), others are loaded on demand (`LoadNonResidentSound`). All SFX numbers in the databases (MONSND slots, prop sound, switch/script sfx) are ids in this file. `formats/d6data.py` class `SfxList`.

4. D6PROP.DAT - props, 0x38 bytes, 238 records

Loader: `LoadProp` (`scenload.c`) - on demand, cached in `_Prop[256]`: **record index must be < 256** ("MAX PROPS EXCEEDED") and <= the header count ("INVALID PROP RECNO"). `LoadPropObject` checks `model <= 0xF9`.

off	type	name	meaning	conf	source
0x00	char[24]	name	display name	high	<code>DrawTargetName</code> , <code>traps.c</code>
0x18	i32	model	index into <code>exe_PropMDLData(prop/.mdl)</code> , must be < 250	high	<code>LoadPropObject</code>
0x1C	i16	objtype	graph object class -> <code>gobj+0x6C</code> (1 = tree/foilage, else 0)	high	<code>LoadPropObject</code>
0x1E	i16	dragicon	drag icon shown when targeted (0..37, <code>exe_dragiconname</code>)	high	<code>DrawReadyData</code> (<code>DrawShadedDragIcon</code>)
0x20	u8	flags	1 switch, 2 custom scale, 4 collide, 8 terrain-follow 4 corners, 0x10 container (chest), 0x20 snap in BSP, 0x40 anim-texture, 0x80 animated	high	<code>LoadPropObject</code> , <code>pcorders.c</code> , <code>traps.c</code> , <code>jhpmap.c</code>
0x21	u8	flags2	1,2 render flags, 4 y-lock sprite, 8/0x10 perspective, 0x20 float, 0x40 smashable (crate/barrel), 0x80 fountain (drink)	med	<code>LoadPropObject</code> , <code>combat.c</code> (smash), <code>fountain.c</code> , <code>pcorders.c</code>
0x22	u8	flags3	1 floats on water, 2 can be pushed/dragged (<code>wcoll.c</code>)	high	<code>wcoll.c</code> (float / latch-drag)
0x24	f32	scale	float copied to <code>model+0x69C</code> (height/scale) when <code>flags&2</code>	high	<code>LoadPropObject</code>
0x28	f32	height	mass (float) for push/drag physics; >0 = movable, disables ground adjust	med	<code>LoadPropObject</code> , <code>pushy.c</code> , <code>pcorders.c</code>
0x2C	i16	broken	D6PROP record that replaces a smashed prop	high	<code>combat.c</code> (smash prop, ~1.12001)
0x2E	i16	smashefx	effect index (<code>_gCrateEfx</code>) when smashed	high	<code>combat.c</code> (<code>_gCrateEfx</code>)
0x30	i16	sound	ambient SFX (MONSOUND id)	high	<code>LoadPropObject</code> , <code>events.c</code> , <code>traps.c</code>
0x34	i32	soundparam	ambient SFX radius	med	<code>LoadPropObject</code> (<code>SetPropSFX</code>)

Hard-coded props: record 0x4E and 0x1D get a particle emitter (type 0x14) in `LoadPropObject`; record 0xA8 chest spawns treasure higher; records 0xD/0xE are special in the smash code. Smashable props: flags2 0x40 + broken + smashefx (e.g. Barrel 13 -> Broken Barrel 223).

5. D6TREAS.DAT - treasure records, 200 bytes, 236 records

Readers: `SetChestTreasure` (`traps.c`, `chests/traps`), `MakeMonInventory` (`monster.c`, via `MonRec_Load`), `LoadNPCInventory` (`npc.c`). Each lseeks `index*200`; no count check.

off	type	name	meaning	conf	source
0x00	i32[50]	entries	10 x (i16 type, i16 chance%, i32 a, i32 b, i32 c, i16 pA, i16 pB); type 1/2 item=DiceRoll(a,b,c), 3 D6TRLIST lists a/b/c, 4 exp; see <code>entries_list()</code>	high	<code>SetChestTreasure</code> (<code>traps.c</code>), <code>MakeMonInventory</code> , <code>LoadNPCInventory</code>

Entry (20 bytes): i16 type, i16 chance%, i32 a, i32 b, i32 c, i16 pA, i16 pB.

type	meaning
0	unused
1, 2	item id = <code>DiceRoll(a, b, c)</code> (count a, sides b, bonus c; fixed item = 0, 0, id)
3	item from treasure lists: list a with chance pA%, list b with pB%, else list c (b,c default to the previous)
4	experience <code>DiceRoll(a,b,c)</code> shared by the party (<code>TreasureDivvyExp</code> , chests only)

Monsters take at most 4 items (treasureA first, then treasureB). With high luck (or trait 0x35) chests roll twice and keep the more expensive item. Data: 598 type-3, 436 type-1, 249 type-2, 39 type-4 entries.

6. D6TRLIST.DAT - treasure lists, 0x3C bytes, 62 lists

LoadTreasureLists_ (treasure.c) reads the u32 count (**max 127**, _gTreasureList[128]) and all records at startup.

off	type	name	meaning	conf	source
0x00	char[20]	name	list name (editor only)	high	-
0x14	i16[20]	ranges	10 (first,last) D6ITEM ranges; first<=0 = unused, last<=first = single item	high	GetTreasureListItem_

GetTreasureListItem_: pick one of the ranges with first > 0 uniformly, then an item uniformly in [first, last] (single item when last <= first).

7. D6NPC.DAT - NPCs, 0x24 bytes, 137 records

LoadNPC_ (npc.c) lseeks npc*0x24 when a monster with MonsRec.npc != 0 enters; LoadNPCNames_ (jentry.c) reads names for the journal. NPC dialogue lives in NPCDATA.PAK slots 1+n / 161+n / 321+n, so **NPC ids are limited to 0..159**.

off	type	name	meaning	conf	source
0x00	char[24]	name	NPC name	high	LoadNPC_, LoadNPCNames_
0x18	u32	gold	gold carried (npc+0x20; NPCGoldToBag_, trading)	high	LoadNPC_, NPCGoldToBag_, GetNPCSellItemPrice_
0x1C	u16	treasureA	D6TREAS record for the NPC inventory (npc+0x68, LoadNPCInventory_)	high	LoadNPCInventory_
0x1E	u16	pricepct	price % used when trading (npc+0xAC, GetNPCSellItemPrice_)	med	GetNPCSellItemPrice_, NPCBuyItem_
0x20	u16	trader	>0 enables pricepct (npc+0xAE)	med	GetNPCSellItemPrice_
0x22	u16	treasureB	second D6TREAS record (npc+0x6A)	high	LoadNPCInventory_

8. D6HELM.DAT - helmet attachment per model

InitHelmData_: u32 nModels (24), u32 nHelms (must be 16), i16 modelIdx[128] (only the first nModels used), then records of 0x26 at 0x108 + (slot*16 + helm)*0x26. GetHelmData_ (model, helm) finds slot with modelIdx[slot] == monster gfx and reads the record; helm = ItemRec.range of the worn helm (0..15). Monsters/PCs whose model is not in modelIdx show no helmet.

off	type	name	meaning	conf	source
0x00	f32[3]	pos	attachment translation	med	model.c attach
0x0C	f32[3]	rot	rotation vector (chardemo editor writes _AttachRotVec)	high	chardemo.c
0x18	f32[3]	scale	scale (_AttachScale)	high	chardemo.c
0x24	i16	hair	hair style used with this helmet (TwiddleHair_)	high	SetMonEquipment_, model.c

9. Exe-side tables (deep6.exe, mdldata.c) and limits

table	VA	entries x stride	entry layout	readers / bound checks
_ItemMDLData	0x5DB3B8	240 x 0x51	char name[0x50] (item\<name>), u8 first _MDLExtend	LoadItemObject_ (model > 0xEF -> error, also item id must be 1..gNumItems), LoadAttachItemObject_ (< 0xF0), LoadShipWheelAttachObject_ (< 0xF0), PCMLoadAttach_ (< 0xF0); models cached in _gItemModel
_MonMDLData	0x5DFFA8	121 x 0x55	char name[0x50], u8 first MDLExtend, i32 Model_Read param (+0x51)	LoadMonsterModel_, PCMLoadModel_, PCMChangeModel_ (index < 0x11 = PC body path), LoadMonMDLExtend_/SetMonMDLExtend_ (monant.c), PrintModelData_; no bound check ; cache _gModelPtr[134] where 0x80+pc (128..133) are the PC slots, so gfx must be < 121 (table) and never >= 128
PropMDLData	0x5E27D5	250 x 0x51	as items (prop\ <name>)	LoadPropObject (model > 0xF9 -> error); cache _gPropModel[250]
_MDLExtend	0x5E76F0	64 x 0x54	char name[0x50] (EFXGFX\<name>), u8 kind (0/1 anim texture, 2 alpha sprite chain), u8 next, i16 param	chained from the model tables; kind-2 param indexes MDLExAlpha (0x5E8BF0, 0x14 stride)
icondata	0x5D1840	246 x 8	char *name (itemicon\%s.bmp), u8 grid w, u8 grid h	LoadItemIcons (_gNumItemIcons = 0xF6 hard-coded), ItemRec.icon
dragonname	-	38	dragon\%s.bmp	LoadItemIcons (_gNumDragIcons = 0x26), PropRec.dragicon
_gSpellData + table at 0x5D7D9E	-	105 x 0x30	spell table (names from D6STRING 9000+id)	spells are compiled into the exe; no spell database file

All model references located through relocations (.reloc, every absolute address use): LoadMonsterModel_ 0x499D8E/0x499DC9/0x499DD9/0x499E4E, LoadItemObject_ 0x4D5934..0x4D5A0F, LoadShipWheelAttachObject_ 0x4D5D2A, LoadPropObject_ 0x4D5E71..0x4D5FB6, LoadAttachItemObject_ 0x4D7631..0x4D76FE, PCMLoadAttach_ 0x4FB57A..0x4FB650, PCMLoadModel_ 0x4FBDDA..0x4FBE02, PCMChangeModel_ 0x4FC029..0x4FC051, PrintModelData_ 0x508868, LoadMonMDLExtend_ 0x58638D..0x586438, SetMonMDLExtend_ 0x58646D..0x586490. exe_model_table(root, kind) in d6data.py dumps them.

9.1 Record-count limits

database	count source	hard limit	where
D6ITEM	header u32, all loaded	999 (MAX_ITEMS) ; also 3-digit I### in object lists	LoadItems_, _Item[1000]
D6MONS	none (lseek)	i16 index; M### in object lists = 999; 63 distinct records per area	MonRec_Load_ (_gMonRec[64])
D6PROP	header u32 checked	index < 256; P###	LoadProp_ (_Prop[256])
D6TREAS	none	i16 index	readers above
D6TRLIST	header u32, all loaded	127	LoadTreasureLists_
D6NPC	none	0..159 (NPCDATA.PAK slots), 127 active NPCs	LoadNPC_, MonsterEntry_
D6MONSND	none	i16 index	MonRec_Load_
MONSOUND.DAT	declared max	1024 ids	sndread.c
D6HELM	header	16 helms per model, 128 model slots	InitHelmData_
models	exe tables	items 240, monsters 121 (<128), props 250	see above
live objects	-	240 monster slots, 5120 placed objects, 32 hold triggers	_gMonster, _gTerrObj

10. Adding records (what an SDK must do)

- Items:** append a 0x11C record to D6ITEM.DAT and bump the header count (<= 999). Fill name + unidname (<= 21 chars each), type/subtype, icon (< 246), weight, price, model (< 240, existing _ItemMDLData entry), skill, damage/ac, durdice, restrict mask, flags98 (0x0002 to skip identification), enchantmask. Reference it from object lists as I%03d, from D6TREAS/D6TRLIST ranges, from shops (D6SMITnn u16 item ids) or scripts (CREATEITEM). Item records are global: saves store item ids only, so appending is save-compatible; changing an existing id is not.
- Monsters:** append a 0x154 record (D6MONS header count is not used by the game, but keep it correct for tools). Required: name, gfx (existing model 17..120), soundrec (= gfx, or 0), hpdice (level in [0]), hit/parry/armor, abilities, resist, speed, atkdelay, xp, equip entries with natural attack items, treasureA/B, groupdice (1,1,0 for a single monster). npc = 0 unless a D6NPC record + NPCDATA.PAK slots exist. Place via M%03d (so index <= 999) or scripts GENMONSTER/SPAWNMONSTER/chest traps. Keep <= 63 distinct monster records per area.
- Props:** append a 0x38 record, index must stay < 256 and the header count must be updated (it is checked). model < 250.
- Treasure:** append 200-byte records (no count check, but keep the header right); new treasure lists: append to D6TRLIST (<= 127, header count is used).
- NPCs:** D6NPC record + NPCDATA.PAK members (code/strings/responses) for id < 160.
- Monster sounds:** a new D6MONSND record (index referenced by soundrec) and, for new wav files, a new id, "name" line in MONSOUND.DAT (id < 1024, raise the declared maximum on the first line).
- New models / icons / spells** cannot be added through data files: _ItemMDLData, _MonMDLData, _PropMDLData, _MDLExtend, _icondata, the drag icon list and the spell table are compiled into deep6.exe with fixed sizes (240/121/250/64/246/38/105). Options: (a) replace an unused existing entry's .mdl file under the same name, (b) patch the table string in the exe in place (same entry size), or (c) in the OpenDWWandW port, make the tables data-driven and raise the limits (and the cache arrays _gItemModel, _gModelPtr (PC slots at 128), _gPropModel).
- Always rebuild with RecordTable.build() (count in record 0) and keep untouched bytes; run python3 formats/d6data.py --selftest GAMEDIR.

11. Open questions / low confidence

- Monster fields marked low (0x22, 0x26, 0x78, 0x8C, 0x11A, 0x120, 0x12C) have values in the data but no reader was found in the decompile (they may be editor-only or read through the copied runtime struct).
- Exact meaning of several flag bits (MonsRec flags130/131/132, ItemRec atkflags/ atkflags2, flags98 0x0080/0x4000/0x8000) is inferred from the tests and the records that set them.
- Item 0xBC/0xBE (dmgextra), 0x108, 0x10E, 0xA4, 0x96: copied or present but meaning unknown.
- NPC pricepct/trader exact semantics (buy vs sell) - med.
- D6MONSND slot byte +1 and the HelmRec translation are not confirmed by a reader.

Chapter 7. Deep6 3D models (models/**/*.mdl)

Source: docs/formats/models.md

Wizards & Warriors (2000, engine "Deep6"). This document covers the .mdl model format well enough to draw monsters, items, props and PCs textured, and how the D6MONS / D6ITEM / D6PROP databases select a model.

Reference implementation: formats/d6model.py (decoder, record->model mapping, numpy software renderer). The region walker d6data.Mdl (byte-identical rewrite) uses the same layout.

```
python3 formats/d6model.py --render GAMEDIR M 1 out.png [--yaw D] [--pitch D] [--frame N] [--size PX]
python3 formats/d6model.py --render GAMEDIR prop/chest.mdl out.png
python3 formats/d6model.py --info GAMEDIR P 6
python3 formats/d6model.py --selftest GAMEDIR
```

Self test on the GOG install: 728 .mdl decode with 0 failures (geometry, uv and poly index ranges checked). Every D6MONS (372), D6ITEM (857) and D6PROP (238) record resolves to an existing file except P28 (prop/BUTTON-CRYPT.MDL, missing from the retail data; only button-crypt-flat.mdl ships).

Sources (decompile module / function): model.c Model_Read_, Model_ResetTextures_, Part_FindSprites_, CalcModelHeight_, Model_BoundSphere_, Part_Draw_, Part_DrawSprite_, Attachment_Draw_, Attachment_DrawModified_, Model_Draw_; mpol.c drawTri_; pal16.c RefPal16_Calculate_, Pal16_SetRGBGamma_; graphobj.c GraphObj_Create_, GraphObj_UpdateMatrix_, GraphObj_HidePolys_; vector.c Mat3_Vec3_; monster.c LoadMonsterModel_, LoadMonsterGFX_, TwiddleHair_, TwiddleEyes_; scenload.c LoadItemObject_, LoadPropObject_, GetHelmData_; mdldata.c tables in deep6.exe.

Confidence: **H** = read from loader/consumer code and confirmed in data and renders; **M** = from usage, plausible; **L** = guess.

All values little-endian.

1. Conventions (units, axes, placement)

Item	Value	Conf
Units	Model coordinates are world units; the engine applies no scale (object matrix is a pure rotation, GraphObj_UpdateMatrix_). 1 terrain tile = 1024 units, so 1 unit is about 1 mm: HM (human male) is 1665 tall, skeleton 1563, horse 1980, barrel 782, wooden chair 1098, long sword 925 long.	H
Handedness	Left-handed, y up: camera space is x right, y up, z into the screen (Camera_ProjectVert3_: $sx = cx + xf/z$, $sy = cy - yf/z$). All engine matrices are rotations, so world space is left-handed too. Text on prop/sign-cemetery.mdl reads correctly under this convention.	H
Origin	The ground point of the object: feet / base at $y \sim 0$ (min y is 0..20 for monsters/props); x/z roughly centred. CalcModelHeight_ = max y of frame 0. Items are authored around their grip / attach point (e.g. sword grip at $z = 0$, blade along +z).	H
Facing	At yaw 0 the model faces +Z (horse/monster heads point +Z; walk-cycle root motion runs along +Z).	H
Placement	$world = v * M + pos$ (row vector, row-major 3x3). $M = RotY(yaw)$ then Z and X rotations (gobj+0x18 yaw, +0x14 Z, +0x1c X, angles in 1/1024 turn, GraphObj_UpdateMatrix_). $RotY(a) = [[c, 0, -s], [0, 1, 0], [s, 0, c]]$, so local +Z maps to $(\sin a, 0, \cos a)$, the engine's usual forward vector for heading a.	H (yaw) / M (X/Z order)
Front faces	Clockwise on screen (y down): drawTri_ draws when $(x0-x2)*(y1-y2) - (x1-x2)*(y0-y2) > 0$, otherwise only if poly flag 0x80 (two-sided). Checked on a closed barrel (47/51 faces agree; the rest are near edge-on).	H
D6PROP scale (0x24)	Not a render scale: copied to model+0x69c when prop flags&2, which is the collision radius (default 528.0, set in Model_BoundSphere_; monsters use D6MONS+0x98).	H
Model_Read_ last arg	float stored at model+0x700 (0.5 for items/props, from _MonMDLData for monsters); only passed to Anim_Load_ (animated-texture speed). Not a scale.	M

2. File layout

2.1 Header

Off	Type	Field	Conf
0x00	u32	magic 0x4D444C20 (bytes " LDM")	H
0x04	u32	version: 9 or 10; bit 0x1000000 = framebits table present	H
0x08	u32	file offset of the part offset table	H
0x0C	u8	paltype (0, 1, 2, 3)	H
0x0D	...	palette, size by paltype (below)	H
+0	u16	nframes (model+0x80)	H
+2	u8[nframes]	framebits, only if version bit 0x1000000: 0 = frame not stored in the parts	H
+..	u16[256][3]	animation table (model+0x82): (first frame, last frame, action frame) per animation id; all zero = unused; last < first plays backwards. The third value is the "event" frame (hit / SFX trigger, atqlist.c). Animation 0 is the idle/stand loop the game starts with (LoadMonsterGFX_ sets frame = anims[cur].first).	H (first/last) / M (third)
+..	u8	nattach	H
+..	per attachment	u16 n; f32[n][3] translation; f32[n][3][3] rotation (row-major). n is 0 or nframes.	H
+..	u8	nparts	H

Palette (paltype):

paltype	Size	Content	Conf
0, 3	0x4000	u16 RGB565 [shade 0..31][index 0..255]; shade s = rgb*(s+1)/32 (RefPal16_Calculate_), so shade 31 is the full colour. 589+106 files.	H
1	0x300	256 x RGB888 (PC models pc/*.mdl, 33 files); colour 0 is the key colour (0,255,0). The engine remaps it into a shared reduced palette.	H
2	0x40	char[64] name of a shared palette models/refpal/<name>; unused in retail (dir empty).	H (code)

At load time Model_ResetTextures_ sets colour 0 of every shade row to 0, so **index 0 draws black on normal polys and is transparent on masked polys and sprites**. The engine then applies a small gamma lift (Pal16_SetRGBGamma_: c + min(c*g/128, g)) and a per-vertex/terrain light level that selects the shade row (0x1f = full). d6model returns shade 31 as palette and all rows as shades.

2.2 Part table and part

The part table (header +0x08) is u32[nparts] file offsets. 717 models have one part, 10 have two, 1 has three. The engine keeps parts in a linked list in **reverse** order (model+0x68c = last part read); the first-read part's group count is what GraphObj uses, which is the same for all parts in practice.

Off	Type	Field	Conf
+0x00	u32	file offset of this part's npaths field (start of the path/frame block)	M (holds for all files except the 33 in efx/, where it is stale; the loader ignores it)
+0x04	u32	file offset of the texture offset array (just after the ntextures byte)	M (holds for all files except the 33 in efx/, where it is stale; the loader ignores it)
+0x08	u32	file offset of the "sprite list" (u8 n + u32[n]); n is 0 in all retail files	M
+0x0C	u32	part flags (model part+0x1c); 0 in all files; runtime bits 8 Y-lock sprite, 0x20 Y-lock, 0x40 perspective-correct, 0x10 hidden	H (runtime)
+0x10	u32	nverts	H
+0x14	per LOD (1 for v9, 4 for v10)	see below	H
+..	u8	ngroups: number of mesh groups (GraphObj visibility table size)	H
+..	u16	nframes (= model nframes)	H
+..	u32	npaths	H
+..	u16[npaths]	path lengths (sum = nverts)	H
+..	u32	framesize: bytes of one stored frame = sum(6 + 3*(len-1))	H
+..	per frame	frame data (framesize bytes, absent if framebits[f] == 0), then f32[3] unknown (almost always 0) and f32[3] root-motion translation	H / L / M
+..	u8	ntextures	H
+..	u32[ntextures]	file offsets of textures	H

Per LOD level (v10 has 4: engine picks 0..3 by camera depth <= 1536, 2560, 4096, else 3):

Type	Field	Conf
u32	nuv	H
(s16 vertex, s16 u, s16 v)[nuv]	corner list: vertex index into the part, texel coordinates in the polygon's texture (0..w, 0..h); divide by w/h for 0..1 (the D3D path does u*scale/w)	H
u32	npolys	H
(u16 firstuv, u8 ncorners, u8 texture)[npolys]	polygon = corners uv[first .. first+ncorners-1]; drawn as a fan (0, k+1, k+2); ncorners <= 32; ncorners == 1 = sprite; texture = index into the part's texture table	H
u8[npolys]	poly flags	H

Poly flags:

Bits	Meaning	Conf
0x1F	mesh group; drawn only if gobj visibility[group] != 0. GraphObj_Create_ enables group 0 only; game code toggles others with GraphObj_HidePolys_	H
0x20	50% translucent	H
0x40	masked: colour index 0 transparent	H
0x80	two-sided (no back-face cull)	H

Mesh groups in data: 9-group humanoids (NPCs, PCs, several monsters) use 0 = body, 1..4 = hair variants (TwiddleHair_ shows one), 5..8 = eye states (TwiddleEyes_, 5 = normal). Two-group props use group 1 for an alternate state (key in a lockplate, chest contents, book on a lectern...; prop flags&0x10 "container" toggles it). campfire.mdl: 0 = flame sprite, 1 = unlit logs, 2 = burning logs. Some item models (book-plain, heart) put everything in group 1. Model.default_groups() returns (0,1,5) for 9-group models, otherwise (0,) plus the lowest solid group when group 0 has no triangles.

2.3 Frame data (vertex paths)

For each path in order: s16 x, y, z start vertex, then len-1 times s8 dx, dy, dz cumulative deltas. Vertices are numbered consecutively over all paths (Part_Draw_, Model_BoundSphere_). **H**

Frames missing from framebits (PC models: e.g. HM stores 96 of 955 frames) were removed by the developers' pcreduce.exe, which keeps only the animations listed in <model>.seq (section 6). The game never needs the others: PC models appear only on the character sheet. Model.default_frame = anims[0].first if stored, else the first stored frame.

The second per-frame vector is the root motion of the animation (skeleton walk anim 6: z grows 0..2790 over the cycle, y bobs ~100). The vertices themselves do not include it; the game moves the object. **M**

2.4 Textures

At each texture offset: u32 w, u32 h, u8[w*h] row-major palette indices. (w/h may be 0 for unused slots.) The engine optionally downsamples by `_g_modtexmip` (and shifts uv to match). **H**

2.5 Sprites (1-corner polys)

`Part_DrawSprite_`: a camera-facing quad centred on the corner's vertex, half size (u/2, v/2) world units (the uv entry holds the size, not texels), showing the whole texture, masked. 44 models have sprites (flames on candles/torches/ sconces, bushes, fire). Bush/tree sprites store height 0; d6model keeps the texture aspect there (**L**). Flame sprites usually show placeholder textures: the real ones are animated textures attached at load time (MDLExtend chain in the model-name tables, EFXGFX files, `Model_AddAnimTex_`), not decoded.

2.6 Attachments

nattach slots (28 in humanoid/monster files, many empty), each with a per-frame rotation + translation. A child model (weapon, shield, helm, rider) is drawn at `v_child @ rot[f] + trans[f]` in the parent's model space (`Attachment_Draw_`: `child.M = rot . parent.M`, `child.pos = parent.pos + trans . parent.M`; verified by disassembly of `Mat3_MulMat3Copy_`/`Vec3_MulMat3Copy_`). Slots used by `PCMSetCostume_`: 8 = helm, 12 = shield, 13 = left-hand weapon (flag 0x20 items, e.g. bows), 14 = right-hand weapon. **H** (transform) / **M** (slot names)

Helms go through `Attachment_DrawModified_` with a D6HELM.DAT record ((`race_row*16 + helm`)*0x26 + 0x108): f32 offset[3], f32 rotX/rotY/rotZ (1/1024 turn), f32 scale[3], u16 ?, i16 slot. Child vertex = `v @ diag(scale) @ Rz @ Ry @ Rx @ rot + offset @ rot + trans`, with `Rx = [[1,0,0],[0,c,s],[0,-s,c]]`, `Rz = [[c,s,0],[-s,c,0],[0,0,1]]`. Checked by render: HM + helm-norman with record 0 sits exactly on the head. **H**

3. Record -> model mapping

Model names live in three tables compiled into `deep6.exe` (`mdldata.c`), not in the `.lst` files (those are alphabetical tool lists). `d6model.model_table()` reads them from the exe (PE section mapping; DGROUP VA 0x5C0000 -> file 0x1A5400).

Kind	Record field	Table (VA)	Stride	Count	Path	Conf
Monster	D6MONS +0xB0 (s16 gfx)	_MonMDLData 0x5DFFA8	0x55: char[0x50] name, u8 ext, f32 Model_Read_arg	121	gfx < 17: pc/<name> (HM, HF, EM ... LM); else monster/<name>	H
Item	D6ITEM +0x114 (s16 model)	_ItemMDLData 0x5DB3B8	0x51: char[0x50] name, u8 MDLExtend chain	240 (index must be < 0xF0)	item/<name>	H
Prop	D6PROP +0x18 (s32 model)	_PropMDLData 0x5E27D5	0x51: char[0x50] name, u8 MDLExtend chain	250 (< 0xFA)	prop/<name>	H

Record numbers are 1-based (record 0 of each .DAT is the header). Names are stored in mixed case (CHEST.MDL); open files case-insensitively. For PCs in robes (`PCInRobes_`) the engine appends a robe suffix to the race name (`HMrobe.mdl` etc.); the polymorph/costume paths are not modelled by `model_for`.

Examples: M001 Graveyard Skeleton -> monster/skeleton.mdl, M006 Worgur -> monster/ratwolf.mdl, M015 Horse -> monster/horse.mdl, P006 Treasure Chest -> prop/chest.mdl, P013 Barrel -> prop/barrel-1.mdl, P097 Sconce -> prop/sconce-dungeon.mdl, I010 Short Sword -> item/sword-short-plain.mdl.

Load-time modifications not stored in the file: monster-specific animation copies (`CopyAnimSeq_` in `LoadMonsterModel_`), texture overlays from D6MONS (+0xB2., `Model_AddOverlay_`), animated textures / particle attachments from the MDLExtend chain, prop flags2 (Y-lock sprites, perspective, float).

4. Python API (formats/d6model.py)

```
import d6model as dm
m = dm.load_model(gamedir, 'monster/skeleton.mdl') # or dm.model_for(gamedir, 'M', 1)
m.version, m.paltype, m.nframes, m.anims, m.default_frame, m.height()
m.palette # (256,3) uint8 full-bright colours (index 0 -> black)
m.textures # [Texture]: .w .h .index (h,w) uint8, .rgba (h,w,4) uint8 (index 0 alpha 0)
m.parts # [Part]: nverts, ngroups, path_len, frame_off, frame_vec, mips, textures, tex_base
mesh = m.mesh(frame=None, mip=0, groups='default')
# vertices (V,3) f32, tris (T,3) i32, uv (T,3,2) f32 0..1, tex (T,) global texture index,
# flags (T,) u8 PF_*, part (T,), sprites [dict(vertex, tex, width, height, flags)]
m.part_vertices(part, frame) # (nverts,3)
m.attachment(slot, frame) # (rot 3x3, trans 3) -> child v @ rot + trans
dm.model_for(gamedir, 'M'|'I'|'P', recno) # 'monster/skeleton.mdl' (relative to models/)
dm.render(m, size=512, yaw=35, pitch=25) # (size,size,3) uint8 preview
```

5. Not decoded / limitations

- Animated textures and particle emitters attached via MDLExtend / EFXGFX are decoded (effects.md) but the editor preview does not animate them (flame sprites show placeholder textures).
- Texture overlays (D6MONS +0xB2, `Model_AddOverlay_`) and helm data selection per item.
- LOD 1..3 of v10 files decode but were only spot-checked.
- `prop/bushgrp*.mdl` have an all-black palette in the file (terrain foliage, not in the prop table; probably drawn with another palette).
- Part header words +0x00/+0x04 and the first per-frame vector are not understood (not needed for drawing).

6. Animation ids and PC .seq files (H unless marked)

The 256-entry `anim` table of a model is indexed by animation id. The engine asks for ids through `Anim *` helpers (`assets.c`, `combat.c`, `monster.c`, `npc.c`, `traps.c`); a missing id falls back to 0 (the bit set at `model+0x6D4` says which ids exist). The developers' own id names are in `mdlinfo.prn` in the game folder. Exported glTF animations are named `anim_<id>`.

Importing rigged (skinned) glTF

`d6mdllo.import_glTF` bakes skeletal animation into vertex frames, since the `.mdl` format only stores frames: when every mesh primitive has a skin with `JOINTS_0 / WEIGHTS_0`, frame 0 is the rest pose and each glTF animation is sampled at 16 frames per second (`SKIN_FPS`; `LINEAR`, `STEP` and `CUBICSPLINE` channels, linear blend skinning with up to 4 weights). The animation name picks the id: `anim_<id>`, or a name such as `walk`, `run`, `attack`, `death`, `cast` (`ANIM_NAMES`; Blender's `Armature|Walk.001` works). Animations without an id are skipped with a warning; the action frame is the middle frame. Keep clips short: every baked frame stores every vertex. Tested on a synthetic rig (`tests/test_skin_import.py`) and in the game (a two-bone column with `idle / walk / run` in place of the `Traveler's Inquisitor.mdl`, attachments copied from it: it renders and bends while walking). Not yet tried with a Blender export (M).

id	meaning
0 / 1 / 2	READY (idle) / READYALT / READYLOOK
5, 6, 7	walk start / loop / end (<code>Anim_Walk_</code> uses 6)
10, 11, 12	run start / loop / end (11)
15 / 16 / 17	left turn / right turn / back step
20 / 21	wound small / large
25 / 26	death / dead on the ground
28 / 29	sleep / asleep
30, 31, 32, 33	fly start / loop / end / flying idle
34-36	sneak
47	combat stance (ready)
49	left-hand swing
50, 51, 52	sword swing / chop / jab (random of 3)
54	sword block
56-59	pole arm attacks (57 chop, 59 jab)
61 / 63	two-handed chop / backhand
70 / 75	long bow / crossbow
80	throw (falls back to 51, then 50)
85	block
90 / 91	punch / double punch (unarmed)
93 / 94	kick / flying kick
100-103	spell casts A-D (per spell, <code>effects.md animseq</code>)
105 / 110	use item left / use item
115	gaze
120	bite
125 / 126	claw / claw left
130 / 131	change to mode 2 / back (two-mode monsters, e.g. flying)
135-147	mode 2 set: ready, block 136, wound 140, death 142, dead 143, attacks 145-147
150	breath attack
159 / 160	land / jump
170	steal
171, 172, 173	kneel start / loop / end (disarm traps, use)
181-192	mounted set: ready, swing, throw, bows, wound, death, jump, spell (190, M), use, misc
193 / 194	mounted walk / run (M)
195-197	swim ready / walk / run (M)
200	victory
210 / 211	special 1 / 2
220 / 225	two- / three-headed breath
230-232	talk A-C (NPCs, random)
233-235	stir pot
250 / 251	entry / exit

`LoadMonsterModel_` copies or splits sequences for particular monster graphics at load time (`CopyAnimSeq_`, `SplitAnimSeq_`, `MakeAnimSeq_`, e.g. graphic 0x45: 33 <- 0, 6/11 <- 31, 50/80/110 <- 100). So an imported model needs at least 0 (ready), 6 (walk), 20 (wound), 25/26 (death), an attack (50) and, for spell casters, 100.

PC .seq files (`models/pc/*.seq`, all 33 identical, reader `formats/d6seq.py`, 33/33 round trip): ASCII, one animation id per line followed by `,`, CRLF, ending with an empty line: 0, 47, 50, 91, 110. They are build-time keep lists for `pcreduce.exe`, which keeps the frames of the listed animations (the stored frames of all 33 PC models equal exactly those ranges) and drops the rest. `deep6.exe` never opens them; it hard-codes the same five ids (`SetModelReadDefaults_`, `_gDataSetFlag`). PC models are only shown on the character sheet: stance 47, fidgets 50 / 91 / 110 at about 16 frames per second. Equipment is drawn at the attachment points (8 helm, 12 shield, 13 left hand, 14 right hand); robes are separate `<race>robe.mdl` models.

Chapter 8. Spells and visual effects

Source: docs/formats/effects.md

Reader/writer: formats/d6efx.py (python3 formats/d6efx.py GAMEDIR runs the round-trip self test and prints the spell table). Round trips, byte identical: emitters.dat 1/1, efxgfx .ant 132/132 (122 ANT, 10 CNT; every CNT row re-encodes identically), .alf 20/20, the spell table in deep6.exe. Confidence: **H** read from code and checked against data, **M** likely, **L** guessed.

1. Spell table (deep6.exe, H)

_gSpellData, VA 0x5D7D80, 105 records of 0x30 bytes (d6efx.SpellTable, from_exe / patch_exe). There is no data file. ItemSpell_(id) turns the external id (+0x20) into a table index. Ids of 1000 and up are monster powers, handled by hard-coded switches.

off	type	field	meaning
0x00	char[32]	name	overwritten at start (see below)
0x20	i16	id	external id 1..105 (items +0x5C, monster spells, tomes, CAST events)
0x22	i8	school	0 Spirit, 1 Sun, 2 Moon, 3 Vine, 4 Stone, 5 Fiend, -1 none: mana pool and spellbook page
0x23	u8	level	1..7 (_gSpellIndex[school][level][slot], known-spell bits)
0x24	u8	slot	1..n within the level
0x26	i16	recover_ms	1000 * level (summons 4000 / 7000)
0x28	i16	mana	10/20/30/40/50/75/100 by level
0x2A	u8	target	0 self/party, 1 needs a target
0x2B	u8	flags	bit 0 needs line of sight; underwater: bit 1 fizzles, bit 2 hits the caster
0x2C	u8	category	0/1 hostile, 2 heal, 3 cure, 4 buff, 5 resurrect, 6 mind, 7 utility, 8 artifact, 9 trap/lock (ally AI targeting; 5 may target the dead)
0x2D	u8	animseq	cast animation 100..103
0x2E	u16	pad	0

Names: LoadStrings_(d6string.c) clears every name and copies D6STRING.DAT string 9000 + id - 1 into it, so the names the game shows (spellbook, combat log, "learns ... spell") come from D6STRING.DAT, not from the exe (checked in the game: string 9019 renamed Burn). The other fields are used from the mapped exe.

Damage dice, the effect type, the attach point and the sound are **not** in the table: they are hard-coded per spell id in CastSpell_(0x4BABB0: queues ATQAnimEfx_(caster, anim, target, EFX type, spell, attach slot, flags) and ATQAnimSfx_) and SpellCollide_(0x4B7BC0: DiceRoll_, SpellDamage_, RadialDamage_, RadialAffli_, HealMonster_, AddEnchant_ ...). Strength scales with the caster's spell skill (monsters: level / 5). So names, schools, levels, mana, costs and targeting can be changed in the table; new spell behaviour needs code (planned for OpenDWWandWExpanded). Editor: Tools > Spells (names go to D6STRING.DAT). Target 2 is used only by Spirit Eye (meaning unknown, L).

2. Effects (EFX types, H)

EFX_Init_(0x47D6F8) handles 97 effect types (0..0x60). Per type: speed (_g_VelocityInitArray 0x468870, f32[97]), range (_g_MaxDistanceInitArray 0x4689F4, all 14336) and flags (_g_EFXFlags 0x468B78, bit 0 = colliding projectile). Effect_Create_(0x472AF8) picks the code: lightning, magic missile, burn, zap, fist, skull, spheres, buff columns, light, vortex, flare, blood, portal, explosions, fire wall, bubbles, shields, storms, gas balls, swarm, freeze, fire drop, particle trails, fire breath / clouds and so on. Effect models are loaded from models/efx.

Sprite animations (FXC_GetFilename_, i = 0..93): 0 dragfire, 1 dragice, 2 pois, 3 fbomb, 4 portal, 5 nukeatom, 6 shroom, 7 bloodc1, 8 bloodc2, 9 bonec1, 10 slimec1, 11 gblood, 12 firewall, 14 dispell, 15 poof, 16 hbreath, 17 dbreath, 18 gaze, 19 gazeglow, 20 slimespit, 21 light, 22 burnex, 23 bluelight, 24 painex, 25 painexp, 26 bless, 27 spikeball, 28 sparkball, 29 sparkex, 30 yellowexp, 31 zap, 32 zapglow, 33 lightbeam, 34 greenexp, 35 mmex, 36 lightning, 37 lightnglow, 38 drain, 39 drainglow, 40 vomit, 41 eye, 42-50 coloured sparks, 51 explosion, 52 meteorex, 53 iexplode, 54 bbexp, 55 flash, 56-93 particle/ files (blobs, bolts, pulses, stars, fire, pois, pain, ice, fly, smoke, mm, waterdrp, spore). **Replacing these files changes the effect look; keep the frame size.**

3. emitters.dat (particle emitters, H unless marked)

i32 n (31), then n records of 0x80 bytes (d6efx.EmitterFile):

off	type	field
0x00	char[24]	name
0x18	u32	flags: 1 angles relative to the emit direction, 2 pass particles to the child, 4 child emitter on destroy, 8 fixed count per frame (M)
0x1C/0x20	f32	pitch base / random (radians)
0x24/0x28	f32	yaw base / random
0x2C/0x30	f32	speed base / random (units/s)
0x34/0x38	i32	particle life base / random (ms)
0x3C/0x40	i32	particle size base / random
0x44	i32	glow sprite size (<<3)
0x48	f32	emission rate, particles/s (M)
0x4C	u32	unknown (L)
0x50	f32[3]	acceleration per second (gravity y = -1200 / -2048)
0x5C	u32	0
0x60	i32	frame count (0 = from the animation, 255 none)

0x64	i32	child emitter index + 1 (spawned when a particle dies / hits)
0x68	i32	particle limit
0x6C	i32	emitter duration ms (< 1 = endless)
0x70	i32	ms to pre-advance new particles
0x74	i32	animation selector 1..9 = coloured spark FXC 0x2A..0x32; 0 = dots
0x78	i32	palette selector (1 particle, 2 rainbow, 3 exanim, 4 light.p16)
0x7C	i32	glow selector 1..16: mmglow, nukeGlow, eyeGlow, sparkFlare, discGlow, skullAlpha, fistAlpha, zapglow, fbirdFlare, fireAlpha, lightnglow, lightray, lightbeamGlow, fireglow2, blueglow, greenglow

The selectors are replaced by pointers at load time. An emitter calls `SpellCollide_` when it is destroyed or hits something. (This table replaces the emitter notes in `data.md` / `terrain.md`, whose offsets from +0x44 on were wrong.) A second, separate particle system (`jhp_part.c`) has 31 descriptors compiled into the exe (not decoded).

4. Sprite files

- `.ant` (`Anim_Load_`, `H`): 'ANT ' (bytes TNA), u32 frames, w, h, a 0x4000 byte shade table (32 rows x 256 RGB565, as the level `.twd`), then frames x w*h palette indices; index 0 is transparent.
- `CNT .ant` (`Anim_Compressed_Load_`, `H`): 'CNT ', u32 frames, u16 w, h, shade table, then per frame u32 size, u32 rowoff[h], u8 data[size]. A row is alternating runs: skip k (transparent), copy k (k literal bytes follow), until w pixels; rows always end on a copy run (a trailing skip gets a 0). `Ant.encode_row` writes them exactly like the shipped tool.
- `.alf` (`Alpha_Load_`, `H`): 'ALF ', u32 1, w, h, u8 alpha[w*h] (0..31), u16 rgb565[w*h]: additive / alpha glow sprites.
- `.p16`: shade tables.

5. Animated model textures and glows (_MDLExtend, H)

`_MDLExtend` (0x5E76F0, 64 x 0x54): char name[0x50] (EFXGFX\name), u8 kind, u8 next, i16 param. Kind 1: animated texture, `Model_AddAnimTex_` puts the `.ant` on texture slot param of the model (BIGFIRE, BALFIRE, FIREWALL, TORCH-WOOD, CANDLE, FOUNTAIN ...). Kind 2: glow, param indexes `_MDLExAlpha` (0x5E8BF0, 0x14 stride: i16 glow, u32 size (M), f32[3] offset, i16 next). Replacing the named `.ant` files animates those models differently.

6. Spellbook UI

`spellmask.bmp` (230x238, 8 bit, at screen 100,136): pixel 0..6 = school tab / button (`SpellMaskField_`). `vxspell.bmp` is a 31x31 cursor; `VXSPELL.PTR` is not referenced by the exe (leftover, L).

Chapter 9. Sound, music, speech and the journal

Source: docs/formats/audio.md

From the decompile (audioc.c, sfxcache.c, soundefx.c, playsam.c, combat.c, monster.c, jentry.c, guild.c, npc.c). Not yet checked by playing modified sounds in the game.

1. Sound effects (MONSOUND.DAT, H)

MONSOUND.DAT (d6data SfxList, documented in databases.md) maps ids to sounds/<name>.wav; R marks resident sounds (meant to be loaded at start; a missing resident wav is an error). In the test setup (SDL dummy audio) no wav was opened before it was first played.

- The first number (the max line, 699) is a real limit: only ids below it are loaded. **To add sounds, raise it** (at most 1024; the loader does not check, ids ≥ 1024 overwrite memory). Checked in the game: max 720 and a new id 700, used by a monster sound slot (D6MONSND), played its wav.
- Duplicate ids: the last line wins (585-588 are defined twice). Negative ids are skipped.
- Engine limits: 700 wave references in total, only handles 0-499 play (including speech), 8 instances per wave, 64 cached buffers (purged to 2 MB).
- WAV: canonical RIFF with the fmt chunk at offset 12 (the format is read at file offset 0x14), uncompressed PCM. Shipped: mono 16 bit 11025 Hz.
- File names are looked up case-sensitively on Linux.

Positional playback

NetMessage_SFX_(id) plays 2D; NetMsg_SFX_Ex_(id, mode, owner, pos) plays in 3D (world units, 1 tile = 1024): volume falls off past a near distance, pan from the listener direction.

mode	source	near / range	slider
0, 8	2D (8 = narrator / GM speech)	-	master / speech
1	monster or PC, followed	2048 / 32768	sfx
2	effect, followed	2048 / 32768	sfx
3	missile	2048 / 32768	sfx
4 / 5	fixed point / no falloff	2048 / 32768	sfx
6	fixed, followed (bell)	7168 / 262144	ambient
7	NPC speech, followed	2048 / per NPC	speech
9	ambient point	2048 / 65536	ambient
10	terrain prop	1024 / the prop's soundparam	sfx
11	event entity	2048 / 32768	sfx

Which ids the game plays (hard-coded)

event	MONSOUND ids
melee hit (random)	4, 18, 20, 566-568, 571-574; undead (mclass 2/3) 569, 570, 575; slime/plant (9/10) 577, 578, 575
magic damage	582, 584, 566, 571, 583
punch	5, 601, 602
armour absorbs	1, 16, 17, 21, 2, 19, 563, 564, 565 (by armour)
bashing doors	686, then 1, 16, 17, 21, 2, 19, 563, 564
explosion	14
PC wounds / death	630-649 by race and gender (some races 640 + 2 * gender); gasps 660 + 3 * gender + random
monster alert, growl, wound, death, attack	D6MONSND slots (databases.md)
moving	dirt 689, stone 688, horse walk/run 677/678, lava 685, swim 683 (terrain) / 49 (BSP), underwater 684, ship 695 + 697
panting	662 male, 665 female
falling	scream 675, landing 687 (terrain) / 686 (BSP), splash 50
mounting	horse 676, ship 590, others 696
heal / NPC	10, 61, 62, 59
bell	15
doors, props, events	the SFXREC operand of event scripts; prop sound + soundparam (looping, mode 10)

There is no per-terrain footstep table.

Ambient sounds (PlayAmbients_, one or two every 3-6 s around the camera): outdoors by time of day (dawn/morning/dusk 30, 31, 612; day 27-29, 616; night 32-36, 615, 626; spokes 3 and 11 sometimes insects 613/614); inside BSPs by spoke (5/6/7: 41, 626-629; 2/8/12: 622-625; others 37-41, 617-621, 628, 629). The wav files can be replaced freely; the id lists are exe tables at 0x5D79E8-0x5D7A58.

2. Music (playsam.c, Miles MP3 streams)

_musicName[8] (0x5E9A00): 0 daytime1, 1 intense1, 2 suspense1, 3 suspense2, 4 eerie1, 5 caution1, 6 theme1, 7 game_new (.mp3 in the MUSIC folder of SSPATH.DAT). Each track plays once; when it ends: in town and menus 5 s of silence then daytime1; in a spoke 120 s of silence, then the spoke's track and intense1 alternate:

spokes	tracks
0, 3, 11	daytime1 / theme1
1, 8	eerie1 / intense1
2, 7	suspense1 / intense1
4, 9	suspense2 / intense1
5, 10	caution1 / intense1
6, 12	game_new / intense1

No combat or day/night music. Replace the mp3 files under the same names; more tracks need an engine change.

3. Speech (soundefx.c)

Folders from SSPATH.DAT (SPEECH, NARRATOR), language prefix E:

kind	file	when
NPC	Speech/NNN-1611wav/EC<msg:4><seg:2>_<npc:3>.wav (NNN = NPC id)	every SAY of string msg; 3D at the NPC
guildmaster	Kgg-1611wav/EK<msg:4><seg:2>_<gm:3>.wav	GM messages (only GM 21 ships)
narrator	000-1611wav/EN<line:6>_<seg:3>.wav	quest texts and dialogs

The segment number counts up from 00 after each file; playback stops at the first missing file, and missing files are skipped silently. **So a voice can be added to any NPC line by dropping a wav with the right name** (msg = the string index in the NPC's string table, see npc.md). Queue: 32 lines. Checked in the game: a new Traveler line s64 opened Speech/001-1611WAV/EC006400_001.WAV when said (file names are matched without case on Linux).

4. Journal (JOURNAL.nnn, H)

Reader/writer: formats/d6journal.py (all shipped files round trip; add() appends an entry).

off	type	meaning
0x000	i32	next free stream offset
0x004	(i32 first, i32 last)[192]	chain per key: 0-159 NPC id, 160-191 = 160 + guildmaster
0x604	1 KB blocks	entries: i32 next (0 = end), i16 msg (string index of that NPC/GM), char text[] (cached prompt, starting with `

Entries never cross a 1 KB block. Every NPC message is journaled automatically for the PCs talking to that NPC (checked in the game; the journal page shows the raw string, so \$N stays unexpanded there, and the file is written when the game flushes the journals, not per message); guildmasters journal after GM op JOURNAL (title) until CLRJOURNALFLAGS. Quest texts are therefore the dialogue strings themselves; there is no separate quest file.

Quest flags: 256 per PC, set by NPC ops SET/ADD/SUBQFLAG, GM ops 23-25 and the event ops SETQFLAGIF / PCBLESSING; read by the QFLAG expressions. Their meaning lives only in the scripts.

Chapter 10. NPC and guildmaster dialogue (NPCDATA.PAK, GMDATA.PAK)

Source: docs/formats/npc.md

Reverse engineered from `ExecuteCode_ / EvalExpr_ (npc.c)` and `GMEExecuteCode_ / GMEvalExpr_ (guild.c)`. Library and assembler: `formats/d6npc.py` (--selftest disassembles and reassembles all 137 NPC and 21 guildmaster scripts byte for byte and rebuilds both PAKs identically). Editor: Tools > NPC dialogue (Ctrl+Shift+D).

Confidence: the operand layouts are **H** (every shipped script decodes with all jump targets on instruction boundaries); meanings of opcodes used in the data are **M**; opcodes never used in the data are **L** (layout from the decompile only). Checked in the game (Traveler, NPC 1, spoke 0): a SAY of a new string inserted at the start of the main path (later addresses all shift), \$N replaced by the PC name, an edited REPLY branch reached through ONREPLY with GIVEGOLD 77 (gold 200 -> 277), and the journal entries.

1. Files

Pak layout (d6data.Pak): a directory of (u32 offset, u32 size) pairs whose length is the first offset.

- NPCDATA.PAK, 481 slots: 0 = lexicon, 1+n = code of NPC n, 161+n = strings, 321+n = keywords (n = 0..159, the monster field `npc / D6NPC` record n). 137 NPCs have scripts.
- GMDATA.PAK, 97 slots: 0 = GM lexicon, 1+n code, 33+n strings, 65+n response table for 32 guildmasters (21 used). The response table is not a string table: u32 count, count x {i16 key, u8 type, u32 offset} plus the strings (`GMFindResponse_`); it serves GM talk, which the game never sends, and all shipped copies are the same dummy. `d6npc` keeps it byte for byte and shows it as a comment.
- String and keyword tables: u32 count, u32 offset[count] (from the table start), NUL-terminated Latin-1 strings. \$N = name of the PC talking, @ starts a new message, ~word~ marks a keyword.
- Lexicon (slot 0): u32 count, u32 string base, count x {u32 offset, u8 type} (5-byte entries): the words of the talk keyword panel.
- D6NPC.DAT record n-1 holds NPC n's name, gold, inventory treasure and trading price (`databases.md`).

2. Execution model

A code blob has no header. Every activation (greeting, talk, reply, event) starts at offset 0; the shipped scripts begin with

```
IF NPCFLAG(0) == 1 / JUMP init / JUMP main
```

`init` registers the event handlers with `ONACT` and sets flag 0. Blobs end with `END` and one pad byte.

- IF / IFNOT / IFKEY: when true the next 3 bytes are skipped. They are always a JUMP, so IF c / JUMP then / JUMP else reads as written.
- SAY, REPLY, WAIT, BYE, LEAVE yield: the interpreter returns and continues after the message has been shown / the player answered / the time is up.
- REPLY prompt, [options] shows up to 16 answer buttons; the engine uses only the first option index and the count, so the options must be consecutive strings. It is always followed by `ONREPLY` with one address per answer.
- `ONACT` slot, address sets the handler of a game event: 0 attacked, 1 PC leaves, 2 used / fought with, 3 steal attempt, 4 item given, 7 gold given, 9 talk (keyword typed or clicked), 11 all PCs gone (8 and 10 are internal: reply and resume).
- NPC script flags: 32 shorts per NPC (`npc+0x24`). Per PC and NPC: a 32-bit register (`pcreg[pc][npc]`) and an attitude (-100..100). Per PC: 256 quest flags (`qflag`). World: `_wState[spoke*256+n]` (the same states the event scripts use).

who operands: -1 the PC talking, -2 this NPC, -3 its target, 0-5 a party slot, 6+ a monster index.

3. NPC opcodes (ExecuteCode_)

Operand types: i16, u8, addr (u16 offset), str (string index), rsp (keyword index), expr (u8 kind + its operands, section 4), val (u8 0 + i16 constant, or u8 1 + expr), cmp (u8: 0 ==, 1 !=, 2 <, 3 >).

op	mnemonic	operands	meaning
01	END		stop
03 / 04	IF / IFNOT	expr, cmp, val	skip 3 when true / false
07	JUMP	addr	
08	SAY	str	NPC says the string (yields)
09	REPLY	u8 n, str prompt, str opt[n]	answer buttons (yields)
0A	ONREPLY	u8 n, addr[n]	jump by the chosen answer
0B	SETFLAG	i16 f, val	<code>npcflag[f] = v</code>
0C / 0D	GIVEITEM / TAKEITEM	i16 item	
0E / 0F	GIVEGOLD / TAKEGOLD	i16 amount	between the PC and the NPC's purse
14	ATTEND		the NPC turns to the PC
15	BYE		end of conversation (yields)
18	ONACT	u8 slot, addr	event handler
1A	SETPCREG	i16 bit, i16 v	<code>pcreg[PC][this] bit</code>
1B	SETWSTATE	i16 spoke, i16 n, i16 v	world state
1C	MODPC	i16 who, i16 kind, val	kind HP (0 = full heal), MAXHP, ABIL+, ABIL-, TRAIT+, TRAIT-, SPELL+, SPELL-, SKILL+100, POISON

1D	SETPCREGOF	i16 who, i16 bit, i16 v	
1E / 1F	ADDATTITUDE / ADDATTITUDEOF	[i16 who,] val	
20	IFKEY	rsp	talk text matches the keyword (& = wildcard, \$BLANK\$ = anything); skip 3
21	WAIT	i16 ms	(yields)
22	LEAVE	i16	the NPC leaves (yields)
23	SETPARTYREG	i16 bit, i16 v	for every linked party PC
24	CLEARWAITS		
25	GOTO	i16 bsp, i16 nav id	walk to a nav point (bsp -1 = terrain)
26	SETMODE	i16	NPC behaviour mode
27 / 28 / 32 / 45	SETBIT20 / SETBUSY / SETBIT8 / SETBIT4	i16 on	monster flag bits (SETBUSY defers talking)
29	CLEARTARGET		
2A	CAST	i16 spell item	at the target
2B	FIGHT		attack the target
2C	FINDTARGET	i16 kind, i16	
2D	ONEXPR	u8 n, expr, JUMP default, addr[n]	switch
2F / 30	ADDFLAG / SUBFLAG	i16 f, val	
31	GIVENPCITEM	i16 item	from the NPC's inventory
33	KILLMSGQ		
34	SUICIDE		
35 / 36	SETNPCREG / SETPARTYNPCREG	i16 npc, i16 bit, i16 v	pcreg of another NPC
37 / 38 / 39	SETQFLAG / ADDQFLAG / SUBQFLAG	i16 who, i16 q, val	quest flags
3A / 3B	SETTRADE / TRADE	i16 / -	allow / open the trade window
3C	SETSELL	i16 item, i16 on	NPC sell list (8 items)
3D / 3E	SETE8 / SETEC	i16	unknown NPC fields
3F	ADDPCLIST		remember the PC
40	SETGROUPREG	i16 bit, i16 v	
41	SETPARTYREGIFQ	i16 bit, i16 v, i16 q, i16 qv	
42	TAKEPARTYITEM	i16 item, i16 once	
43	GIVEEXP	i16 who, val	experience (who >= 0: party)
46	TAKEPARTYGOLD	i16 amount	
47	SETMBIT	i16 bit, i16 v	global per-NPC bits

Placeholders that only consume operands: 10 (i16,i16), 13 / 17 (i16), 16, 19 (i16,i16,i16), 44 (i16,val); 11 and 12 report "not implemented".

4. NPC expressions (EvalExpr_)

NPCFLAG(f), RANDOM(n), STAT18/STAT1A/STAT1C/STATF2(who) (unnamed short fields), STATE(who) (alive/dead), NPCTIMER(npc), ZERO4(a,b), PCREG(bit), WSTATE(spoke,n), HASITEM(item), NOITEM(item), GOLD(who), ACTGOLD() (what was just given), PCREGOF(who,bit), ATTITUDE(), ATTITUDEOF(who), ATHOME(), INTALK(who), FIELD476(), TARGET(), TARGETISPC(), TARGETDIST(), HPLOST(who), HP(who), MAXHP(who), INRANGE(who), NPCBIT4() (busy), NPCHASITEM(item), PARTYHASITEM(item), NPCREG(npc,bit), QFLAG(who,q), PARTYQFLAG(q,v), HASTRAIT(who,t), MBIT(bit).

5. Guildmasters (GMExecuteCode_ 0x54c834, GMEvalExpr_ 0x54bc1c; H unless marked)

Every operand layout in d6npc (GMOPS, GMEXPR) was checked against the interpreter, including the opcodes no shipped script uses.

5.1 Who is who

Each building reads its GM from a per-town table in the exe (_gHallGM, _gSmitGM, _gTempleGM, _gMageGM, _gTavernGM, _gDojoGM, _gPawnGM, _gYardGM), so the ids are fixed:

GM	town	building	name
1-7	Valeia	hall, temple, smith, mage, tavern, dojo, pawn	Sir Elgar, Onabe, Smitty, Roendalf, (tavern), Master Wu, Bratsol
8-12	Ishad N'ha	hall, temple, smith, mage, tavern	Lord Barrenhawk, Munsey, Damosh, Xander, (tavern)
13-19	Brimloch Roan	dojo, pawn, hall, temple, smith, mage, tavern	Sinsei Asami, Miruth, Duke Brinsly, Malakai, Strumbold, Sabastio, Holthorne
20	-	ship yard	Buckly
21	-	Gareth's intro (InitGareth_)	Gareth

The dojo and pawn tables hold 6 and 7 for Ishad N'ha as well (M: whether that town has those buildings was not checked).

5.2 How a GM script runs

- Entering the building: the script runs from offset 0 with PC 0 (GMFLAG(0) is 0): the *init* path sets ONACT 6 (and 0, 4) and flag 0, then ENDS.
- Greeting: whenever a PC steps up (entry, party portrait, prev/next), the script runs again from offset 0 for that PC: the

main path.

- Buttons: every button and item action runs ONACT 6 with ACTBUTTON() = the code and ACTITEM() = the item (0 for plain buttons). This is the only event slot that fires: 0 (PC leaves) and 4 (talk) are set by the shipped scripts but nothing sends them, so GM talk, IFKEY and the response table are dead in the shipped game.
- SAY, REPLY and WAIT yield; the script resumes after them. Every event starts a new run, so a button pressed during a SAY starts a second flow (the shipped Leave handlers use CLEARWAITS + PURGEMSGQ for that). A SAY of an empty string never resumes.
- GMFLAGS reset on every visit. Lasting state lives in QFLAG (per PC), NPCREG (per PC, GMs use 161..166), ATTITUDE, WSTATE, UBIT (32 global bits, saved), GUILDRANK, ROLEINIT and the timers.
- "Local" means the party members present at the building.

5.3 Opcodes

01-0F as for NPCs (section 3), with these differences:

op	mnemonic	operands	meaning
0E / 0F	GIVEGOLD / TAKEGOLD	i16	to / from the PC (GMs have no purse)
10, 12, 1D, 34	NOP	as listed in d6npc	no effect
11	ONACT	u8 slot, addr	slot = event: 6 button (the only one sent), 0 PC leaves, 4 talk
13	SETNPCREG	i16 reg, i16 bit, i16 v	pcreg[PC][reg] bit
14	SETWSTATE	i16 spoke, i16 n, i16 v	world state (v is a constant)
15	MODPC	i16 who, i16 kind, val	kinds 1 HP (not capped while alive), 2 MAXHP, 3 ABIL+, 4 ABIL-, 5 TRAIT+, 6 TRAIT-, 7 SPELL+, 8 SPELL-, 9 SKILL+10 (+10, not +100); no POISON
16	SETNPCREGOF	i16 who, i16 reg, i16 bit, i16 v	for PC who
17 / 18	ADDATTITUDE(OF)	[i16 who,] i16 npc, val	-100..100
19	IFKEY	str	whole-string match, & = prefix wildcard, no \$BLANK\$ (talk only, never sent) (M)
1A	WAIT	i16 ms	yields; ignored when 16 waits are queued
1B	SETLOCALREG	i16 reg, i16 bit, i16 v	for every local PC
1C	CLEARWAITS		drops pending waits and greetings, journal flags off
1E	ONEXPR	u8 n, expr, JUMP default, addr[n]	switch
20 / 21	ADDFLAG / SUBFLAG	i16 f, val	
22	PURGEMSGQ		drops queued messages, ends reply mode, journal flags off
23-25	SET/ADD/SUBQFLAG	i16 who, i16 q, val	who -1 PC, -2 every local PC, 0-5 slot
26	EXIT (was ENDGAME)		_gGameState = 2: leaves the building (or ends Gareth's intro); the script goes on
27	SETGUILDRANK	i16 guild, val	0 smith, 1 mage, 2 temple, 3 pawn, 4 dojo; ranks 1-7
28	SETROLEINIT	i16 role, val	0 none, -1 role quest running, 1 granted
29	SETROLE	val	class change: keeps the old level, role trait and starter spells, exp 0, level 1, drops unusable items
2A	JOURNAL	str	journal title (key 160 + GM id) and journaling on: following SAYs are journaled
2B	CLRJOURNALFLAGS		journaling off
2C / 2D	TIMER0 / TIMER1	i16 on	start (reset) / stop the per-PC timers (who advances them: not found, M)
2E	TAKEQTY	i16 item, i16 n	
2F	JOURNALALL	i16 on	journal for every local PC (always on for GM 21)
30	TAKELOCALITEM	i16 item, i16 once	once > 0: one item from the first local PC; else every copy
31	TAKELOCALQTY	i16 item, i16 n	n units over the local PCs
32	ADDSHOPITEM	i16 shop, i16 item	+1 stock in the open building (shop = town); an unlimited item becomes 1
33	GIVEEXP	i16 who, val	who < 0: the PC, else every local PC (no level-up here)
35	SETUBIT	i16 bit, i16 v	global unique bits (saved)

Opcodes 00, 02, 05, 06, 1F and > 0x35 print "COMMAND ERROR".

5.4 Expressions

GMFLAG(f), RANDOM(n), GENDER(who), CLAN(who), ROLE(who) (who -1 is broken in the exe: it reads PC -1; use 0-5), ALIGNMENT(who) (0-100, 50 for who > 5), STATE(who) (3 = dead), NPCSTATE(npc) (_gNpcStatus, M), ZERO4, NPCREG(reg,bit), WSTATE(spoke,n), HASITEM(item), NOITEM(item), GOLD(who), ACTITEM() (item of the last item button), ACTGOLD() (NPC VM value, 0 for GMs), NPCREGOF(who,reg,bit), ATTITUDE(npc), ATTITUDEOF(who,npc), ZERO13, ZERO14, LOCALHASITEM(item), ACTBUTTON(), QFLAG(who,q), GUILDRANK(guild), ROLEINIT(role), GTIMER(0) (timer 0 in hours) / GTIMER(1) (raw), QTY(item), LOCALQFLAG(q,v) (v 0: no local PC has it; else any has v), LOCALQTY(item), UBIT(bit). The assembler still accepts the old names STAT146, STAT148, STAT220, NPCTIMER and ENDGAME.

5.5 Buttons (ACTBUTTON)

building	0	1	2	3	4	5
Hall	news	bank	leave			
Tavern	news	ale	leave			

Smith	buy	sell	repair	identify	guild	leave
Mage	buy	sell	enchant	identify	guild	leave
Temple	rites	blessing (M)	donate	guild	leave	curse lifted
Dojo, Pawn	buy	sell	guild	leave		
Ship yard	browse	warship bought	quest	leave		

Results: 700 / 701 / 702 / 703 item bought / sold / repaired / identified (ACTITEM = item), 710 / 711 bank deposit / withdraw, 800 + role train role, 820 quest button, 821 / 822 guild specials yes / no, 850 + ability ability trained, 865 skill or trait trained, 900 not enough gold / cannot join, 901 already donated, 902 no rites needed, 903 item not usable, 904 nothing to repair, 905 already identified, 909 cannot carry, 910 no more drinks, 1000 + n drink served.

Roles: 0 Warrior, 1 Wizard, 2 Priest, 3 Rogue, 4 Ranger, 5 Bard, 6 Samurai, 7 Paladin, 8 Barbarian, 9 Monk, 10 Ninja, 11 Warlock, 12 Assassin, 13 Zenmaster, 14 Valkyrie.

Editor: Tools > NPC dialogue, *Guildmasters*; "New from template" gives a hall-style script (greeting, news, bank, leave).

6. Adding an NPC

1. D6NPC.DAT record for the id (name, gold, inventory, prices): Game databases.
2. A script in the NPCDATA.PAK slot (NPC dialogue, "New from template").
3. A monster record with field npc = the id, placed in a spoke. Ids are limited to 0..159 by the PAK layout.

Chapter 11. Deep6 area transitions ("exits")

Source: docs/formats/exits.md

Wizards & Warriors (2000, deep6.exe). This document covers how the party moves between areas: town gates, spoke-to-spoke travel (dungeon entrances and exits), BSP-to-BSP seams inside a spoke, teleports, and where the party appears on arrival. Reference implementation: formats/d6exits.py.

```
python3 formats/d6exits.py GAMEDIR [spoke]      # readable, with source trees
python3 formats/d6exits.py --json GAMEDIR [spoke] # everything as JSON
python3 formats/d6exits.py --markdown GAMEDIR   # the tables in section 7
```

Confidence: **H** = read from the code (decompile and disassembly) and confirmed in the data; **M** = from code, plausible in the data; **L** = guessed.

Related: data.md (trigger, bound and switch tables, event bytecode), levels.md (BSP placement and nav points), terrain.md.

1. Overview

There are only **two** code paths that change `_gSpokeNumber` (H; every write to the global was checked in the disassembly):

1. `SetSpokeEntry_` (deep6.c): the party leaves a **town hub** through a gate. The arrival point comes from the hard-coded `GetEntryPosition_` table (section 5).
2. `NewSpokeSegment_` (d6spoke.c): the script op **LOADSEGMENT** (0x2A). The party keeps its position *relative to the BSP it stands in*. That position is then re-based onto a BSP of the new spoke (section 3).

(`InitStuff_` also sets it to 0 at start-up, and `RestoreCurrentGame_` reads it from a save. In multiplayer, `ProcMsgSEGLoad_` copies the host's `_gSegLoadSeg`.)

Everything else stays inside one spoke, or returns to a town:

mechanism	op / code	scope	arrival
town gate	townhub GATE hot spot -> <code>SetSpokeEntry_</code>	town -> spoke 0, 3 or 11	<code>GetEntryPosition_</code> table
LOADSEGMENT	0x2A	spoke -> spoke	party's BSP-local position re-based on the destination BSP
GOTOBSP / GOTOBSPERR	0x21 / 0x22	BSP -> BSP in the same spoke	the actor's BSP-local position re-based, then a free-space search
TELEPORT	0x3B	inside the actor's current BSP (or on the terrain)	given x,y,z (BSP-relative), then a free-space search
ENTERTOWN	0x4F	spoke -> town hub	town hub screen
ENDGAME	0x54	spoke -> end movie, then town 2	-
recall spell (<code>SpellCollide_</code> effect 0x31)	combat.c	terrain spokes 0/3/11 -> the PC's home town	town hub
portal spells (effects 0x32/0x33)	combat.c	same spoke (<code>_gPortalBase[spoke]</code>)	the portal point
party wipe	<code>CheckPartyWipe_</code> -> game state 0x12	spoke -> town	town hub

The game always starts in a town (`PlayDeep6_` -> `InitTown_`), so a spoke is always entered through a town gate or a **LOADSEGMENT**.

2. Id spaces

name in the DCL	meaning	space
SEGMENT	destination spoke	0..12 (<code>_gSpokeNumber</code> ; file suffix nn)
BSPNUM	BSP slot in <code>_bspListBSP[]</code>	dungeon spokes: 1..n in <code>LoadSpokeSegment_</code> order (0 = null BSP). Terrain spokes: TOL 'B' record slot byte (0-based). See <code>d6level.spoke_bsps()</code> . -1 = terrain/world
DESTNAV, LASTNAV, NAVPNT	nav point id (.nvs u16 id, < 20000) of the BSP named by BSPNUM	<code>GetNavPntIdx_</code> (bsp, id) = bsp+0x7c9c u16 table, id -> index; an id >= 20000 is fatal (<code>D6Err_</code>)
!BOUNDNUM	D6BOUNnn record (1-based)	per spoke
!SWITCH, SEGSWITCH	D6SWITnn record (1-based)	SEGSWITCH is in the destination spoke's table
STATUS, SEGSTATUS	switch value 0/1	<code>SetLinkSwitch_</code> (whole linked ring)
!STATE	WState index	<code>_WState[spoke*256 + n]</code>
DOOR*	brush entity id	<code>bsp*100000 + d6entid</code> (bsp 0 = slot 0)
TOWN	town hub	0 Valeia, 1 Ishad N'ha, 2 Brimloch Roan (<code>D6STRING</code> 0x21fc..0x21fe)
XTILE/YTILE/ZTILE	position	int32 world units relative to BSPNUM's origin (absolute if BSPNUM = -1)

BSP origin = `bsp+0x118ec` (world units). For a dungeon BSP it is `((bx-offx)*1024, -offy*1024, (by-offz)*1024)` from the `Terrain_BSPLoad_` arguments. For a terrain BSP it comes from the TOL 'B' record. `d6level.spoke_bsps()` returns it as origin. BSP file coordinates are world/16 relative to the origin.

3. LOADSEGMENT (0x2A): spoke to spoke (H)

3.1 The op

LOADSEGMENT SEGMENT, BSPNUM, SEGSWITCH, SEGSTATUS, !BOUNDNUM (ExecuteEventCode_ case 0x29) only stores its five operands:

```
_gSegLoadSeg    = SEGMENT      (spoke)
_gSegLoadBsp    = BSPNUM       (BSP slot in the new spoke, -1 = none)
_gSegLoadSwitch = SEGSWITCH    (switch of the new spoke, -1 = none)
_gSegLoadStatus = SEGSTATUS
_gSegLoadBounds = !BOUNDNUM    (stored, never read again - unused)
```

InitSpoke_ resets them to (current spoke, 0, -1, 0, 0). The spoke loop (Deep6Spoke_, CheckTerminateGameLoop_) sees _gSegLoadSeg != _gSpokeNumber and calls NewSpokeSegment_:

- For each of the 6 party creature slots: dismount, unlink from the BSP or terrain, and slot = InBSPArea_(pos). If it is in a BSP: pos -= origin(slot), and if the **old** spoke is 8 also pos.y -= 256 (only for PCs in a BSP).
- Save D6SEGnn.GAM for the old spoke plus world data, UnloadSpokeSegment_, _gSpokeNumber = _gSegLoadSeg, LoadSpokeSegment_ (this loads the new spoke's D6SEG save if one exists, otherwise its pristine tables).
- If _gSegLoadBsp != -1: pos += origin(_gSegLoadBsp) (if that slot is loaded) and, if the **new** spoke is 8, pos.y += 256 (even when the slot is not loaded or the PC came from the terrain). Single player / host only. The position is then used as it is, with **no** free-space search.
- SetLinkSwitch_(-1, SEGSWITCH, SEGSTATUS) if SEGSWITCH != -1. SetLinkStateOpposed_ makes the watched state triggers fire on the next frame, so the arrival door opens and the arrival lever shows the right state.
- PEntry_QuestInit_. GetEntryPosition_ is **not** used.

So the arrival position in the new spoke is

```
arrival = (pos - origin_old[InBSPArea(pos)]) (- 256y if old==8) (+ 256y if new==8) + origin_new[BSPNUM]
```

A PC that stands on the terrain (InBSPArea = -1) keeps its world coordinates and still gets + origin_new[BSPNUM]. With BSPNUM = -1 nothing is added.

Checked in the game: spoke 8 exit T3 (lever W1, party in box B2) to spoke 11 BSP 3 moved the PC by exactly (157696, 13056, -34816) = origin 11/3 - origin 8/1 - (0, 256, 0).

The shipped game builds every spoke entrance as a pair of BSPs that share geometry in their **local** frames. The small overworld piece (tcrypt, tshurua., tdragona, togree, tmines, tlich, tsunkena, tshrinea, tpyram, ttmpcave, ttemple) and the matching vestibule of the dungeon BSP (CRYPTA, SHURU, ...) are the same room. The "exit box" (!BOUNDNUM) has identical local coordinates in both spokes. For example spoke 0 B4 (bsp 0 tcrypt) and spoke 1 B1 (bsp 1 CRYPTA) are both local (-47360, 1024, -51200) .. (-37888, 5120, -44032). tcrypt and CRYPTA even share the same world origin (327680, 0, 197632).

3.2 The events that use it

event	params	logic
@SWITCHDDT0SEGMENT	!STATE, DOORA, DOORB, SPEED, NAVPNT, !BOUNDNUM, !SWITCH, STATUS, SEGMENT, BSPNUM, SEGSWITCH, SEGSTATUS, DELAY, SFXREC	state==0: open both doors. state!=0: close the doors (queued), then IFALLINBOUND !BOUNDNUM ? (DELAY ms, LOADSEGMENT) : SETSWITCH !SWITCH:=STATUS (lever back)
@SWITCHD0ORT0SEGMENT	same with one DOOR	same
@OFFSWITCHDDT0SEGMENT	as DD	inverted: state!=0 opens; state==0 closes and tests the bound
@OFFSWITCHD0ORT0SEGMENT	as DOOR	inverted

The trigger has state = !STATE and runs from CheckStates_ (actor -1) when that WState changes. The WState is driven by the lever switch (D6SWIT.target = !STATE). Some levers are also flipped by a script, e.g. spoke 0 T7 @SETASWITCH from box B5. IFALLINBOUND (op 0x2B) requires **every** existing creature in slots 0..5 to be inside the bound's world AABB. The bound records used for this are disabled (enabled=0, trigger=-1) and serve only as volumes.

Outward exits (overworld -> dungeon) use the plain variants with SEGSTATUS=1. The lever in the dungeon is set "on" on arrival, which opens the vestibule doors. The way back uses the OFF variants with SEGSTATUS=0.

Data check (all 13 spokes): 34 LOADSEGMENT triggers (17 pairs). Every arrival point falls inside the !BOUNDNUM box of a LOADSEGMENT trigger in the destination spoke that leads back (d6exits.find_return). For example spoke 0 T8 (crypt gate, lever W1) -> spoke 1 bsp 1 CRYPTA, and back through spoke 1 T1 (lever W21) -> spoke 0 bsp 0 tcrypt.

4. GOTOBSP / GOTOBSPERR (0x21 / 0x22): BSP to BSP in one spoke (H)

Both opcodes share one handler (@GOTOBSPONLY "Teleport To BSPOnly", @GOTOBSPERR "Teleport To TerrainBSP"; only GOTOBSPONLY is used in the data). Operands are BSPNUM, DESTNAV, LASTNAV:

```
bsp = BSPNUM (slot; D6Err "XLATETOBSP BSP IS NULL" if not loaded)
dnav = DESTNAV ? GetNavPntIdx_(bspList[bsp], DESTNAV) : 0
lnav = LASTNAV ? GetNavPntIdx_(bspList[bsp], LASTNAV) : 0
TranslateToBSP_(actor, bsp, dnnav, lnav)
```

TranslateToBSP_ does nothing if the actor is < 0 (so a state trigger cannot use it), is not alive, is mounted, is not in a BSP (InBSPArea_ == -1) or is already in bsp. Otherwise:

- monster+0xB0 = dnnav (path target index), +0xB2 = lnav or the old +0xB8, and the path state +0xA8..+0xBA is cleared.

Both nav ids index the **destination** BSP's nav table. In the data, DESTNAV is 0.5-3k units from the arrival point and LASTNAV is its neighbour.

- $pos = pos - origin[current] + origin[bsp]$, i.e. the same BSP-local position in the other BSP.
- a free-space search (FindValidBspSpace_, radius 16 growing to 1024) moves the actor slightly if needed.
- relink, recompute collision, reset the sky if it is the camera actor, and send NetMsg_MONXLATE_.

The source is always a bound box with who=5 (all creatures), trigger = the GOTOBSP trigger and bsp = the source BSP slot. The box sits in the overlapping seam of two BSPs that contain the same corridor in their local frames, and each seam has one box per direction. Monsters cross the seams as well. The path AI uses D6LINKnn for the nav graph links across the seam.

Data check: 45 GOTOBSP triggers. The re-based arrival point is near DESTNAV in every case except spoke 6 T18: DESTNAV 892 does not exist in OGREB.nvs, so the index lookup returns whatever the id table holds (probably 0, no path target).

5. Town gates and the entry table (H)

5.1 Path

Town hub (Deep6Town_, townhub.c): the AVI mask fields of type 'GATE' (value 0/1) set _gSpokeEntryNum = gate and _gGameState = 4. Brimloch Roon gate 1 also needs _gPartyWarshipFlag. PlayDeep6_ state 4 -> SetSpokeEntry_:

town	gate	label (D6STRING 0x2260+)	spoke	entry
0 Valeia	0	The Old Road To The Graveyard Ruins Of Bersault	0	0
0 Valeia	1	The Forest Trail To Nymph Lake	0	1
1 Ishad N'ha	0	The Marsh Trail To The Temple Of Ishad N'ha	0	2
1 Ishad N'ha	1	The Forest Road To The Knight's Castle At Shurugeon Ruins	3	1
2 Brimloch Roon	0	The Mountain Pass To The Dragon's Spire	3	0
2 Brimloch Roon	1	The Wharf On The Enchanted Sea (warship)	11	1 (ignored)

Deep6Spoke_ -> InitSpoke_ -> ... -> AssignPartyCamera_(localPlayer) -> GetEntryPosition_.

5.2 GetEntryPosition_ (d6spoke.c 0x489d20) = d6terrain.ENTRY_POINTS

This is a hard-coded switch (_gSpokeNumber) that writes _Entry_X/_Y/_Z/_Dir (int, world units). Spokes 0 and 3 also test _gSpokeEntryNum. The other spokes have a single entry. Spoke 0 entry > 2 and spoke 3 entry > 1 leave the globals unchanged (stale values). The dungeon entries (spokes 1, 2, 4-10, 12) cannot be reached in the retail flow, because no gate leads there. They are probably leftovers from development or testing.

spoke	entry	x	y	z	dir
0	0	217600	0	83456	0x100
0	1	185856	0	74240	0x300
0	2	10752	7168	281600	0x100
1	-	290304	2080	152320	0x100
2	-	345600	5160	307712	0x300
3	0	7954	3072	284160	0x100
3	1	284160	1090	10752	0x300
4	-	113152	1024	110592	0
5	-	131072	-2816	128000	0x100
6	-	215808	-15104	146176	0
7	-	180736	-2048	117248	0
8	-	109568	1024	154112	0
9	-	211456	-2304	147456	0
10	-	224256	-5120	209920	0x200
11	-	382976	1024	25600	0x300
12	-	170496	-5376	139776	0

dir is a yaw in 1024 units per turn. It becomes the PCs' facing (monster+0x20). Placement (AssignPartyCamera_, R = 1280.0 at 0x5c5a3a, _partyDir = {0,170,-170,340,-340,512}):

```
C = E + R*(sin,cos)[(dir+0x200)&0x3ff]          (R behind the entry point)
r = R                                           (set once, before the PC loop)
for each PC: first k in 0..5 with C + r*(sin,cos)[(dir - partyDir[k])&0x3ff] free
              (ObjectSpaceOK_); none free: r += 1024 and retry, up to r = 9472
```

_gSin/_gCos (0x616860 / 0x615860) are filled by BuildTrigTables_ with sin/cos(i * 6.28 / 1024) (6.28, not 2 pi; x uses sin, z cos). So PC 0 stands about 2 units from the entry point and the others on an arc behind it. Checked against a save made after leaving Valeia: the PC stood at (217600, 83453.96), exactly what 6.28 gives (2 pi would give 83456.0). A PC whose spot is blocked takes the next free direction, and the radius is not reset for later PCs. y is snapped with D6_AdjustBase_ ("Unable To Assign New PC" when no ring fits). Each town gate arrival is next to the gate door switch that leads back to that town (e.g. spoke 0 entry 0 (217600, 83456) and W5 tgatea door (209648, 82944)).

5.3 ENTR files

ENTRssnn.DAT (text x y z<TAB>name) are **not** read by deep6.exe, and there is no "ENTR" string in the exe. They are developer bookmarks (e.g. "Crypt Entry", "Wharf Start Point"). The first line of ENTR0000 matches entry (0,0).

5.4 Leaving to a town: ENTERTOWN (0x4F) (H)

ENTERTOWN TOWN: if 0 <= TOWN <= 2, set _gTownNumber = TOWN and _gGameState = 0xe. The spoke loop ends, PlayDeep6_ calls InitTown_(_gTownNumber), and the D6SEG of the spoke is saved by the normal shutdown. The shipped pattern is a gate door brush switch (isentity=1, target -> trigger @IFSTATECALLEVENT) plus a "party near the gate" bound (who=0, trigger -1, sets a state). The IFSTATECALLEVENT checks that state and then CALLEVENTs @ENTERTHETOWN. There are 6 town exits: spoke 0 x3, spoke 3 x2, spoke 11 x1.

ENDGAME (0x54) sets _gEventMovieFlag = movie+1, _gTownNumber = 2 and _gGameState = 0xe.

6. Other movers

- **TELEPORT** (0x3B, @EXECTELEPORT, @IFSTATELEPORT, @IFSTATEPAINTP) x,y,z,bsp: p = (x,y,z) + origin[bsp] (if bsp != -1), then Teleport_(actor, bsp, p). This only works if the actor is alive, not mounted and InBSPArea_(actor) == bsp, so it is same-BSP only. y gets the model's height offset, a free-space search runs (FindValid0BJSspace_), and y is never lower than the given y (D6_AdjustBase_). It never changes the spoke. Used for trap pits and puzzle resets (spoke 2 T155-160, spoke 6 T63-73, spoke 10 T144, spoke 12 T89).
- **TELEPORTOBJ** (0x49, @TELEPORTMONSTER): moves a placed monster. It is not a party exit.
- **Portal spells** (combat.c SpellCollide_): effect 0x32 stores the caster's position in _gPortalBase[spoke] and sets _gPortalFlag[spoke] (not allowed in spoke 12). Effect 0x33 (TeleportToPortal_) moves the group there, in the **same** spoke. Effect 0x31 (recall, terrain spokes only) stores the PC's position in _gPortalHome[pc] and sets _gPortalHRet[pc]=1, then _gTownNumber = pc home town and state 0xe. _gPortalHome/HRet are only saved and loaded (segwrite.c) and are never read for placement. The return trip uses the normal town gate entry. All four arrays live in the save (D6WORLD/D6ARCHIV) and are cleared at new game.
- **Party wipe**: CheckPartyWipe_ -> after _gGraveTime (5 s) game state 0x12 -> town.
- **D6LINKnn**: nav links only (path AI across BSP seams and terrain/BSP joins). They never move the party.

7. All exits in the shipped game

Generated with d6exits.py --markdown (GOG data). Sources: w switch, b bound, s special; "via Tn" is the intermediate trigger (CALLEVENT or SETSWITCH chain). "box / switch" is the world AABB the party must be in: the IFALLINBOUND box for segments, the trigger box for GOTOBSP, and the "near the gate" box for towns. "arrival" is the box centre after re-basing, in world units of the destination spoke. w21:=1 means switch 21 of the destination spoke is set to 1 on arrival.

Spoke 0 - Valeia wilderness (terrain SPOKE00)

trig	event	action	source	box / switch (world)	src bsp	destination	arrival (world)
T8	@SWITCHDDTOSEGMENT	segment	switch W1 B4 via T7	(280320, 1024, 146432)..(289792, 5120, 153600)	0	spoke 1 bsp 1 CRYPTA, W21:=1 (back: T1)	(285056, 3072, 150016)
T12	@SWITCHDDTOSEGMENT	segment	switch W2 B7 via T10	(3328, -2, 219648)..(10496, 3072, 226304)	2	spoke 2 bsp 1 TEMPLEB, W1:=1 (back: T1)	(117504, 1535, 325376)
T15	@SWITCHDDTOSEGMENT	segment	switch W3 B9	(43520, 1260, 216064)..(54272, 7954, 221184)	3	spoke 2 bsp 2 TEMPLEA, W2:=1 (back: T2)	(355072, 4607, 307712)
T20	@GOTOBSPONLY	bsp	box B19	(247072, -5472, 266448)..(249776, -2784, 267568)	4	bsp 5 ttoadb nav 137/120	(231016, -4128, 267008)
T21	@GOTOBSPONLY	bsp	box B20	(227696, -5472, 263120)..(230880, -2784, 264944)	5	bsp 4 ttoada nav 118/119	(246696, -4128, 264032)
T63	@ENTERTHETOWN	town	switch W5 via T34	(206848, -20480, 77824)..(215040, 20480, 87040)	-1	town 0 Valeia	-
T64	@ENTERTHETOWN	town	switch W7 via T38	(187392, -20480, 70656)..(194560, 20480, 77824)	-1	town 0 Valeia	-
T65	@ENTERTHETOWN	town	switch W8 via T39	(4096, -20480, 278528)..(10240, 20480, 284672)	-1	town 1 Ishad N'ha	-

Town gate arrivals: Valeia gate 0 -> entry 0 (217600, 0, 83456) dir 0x100; Valeia gate 1 -> entry 1 (185856, 0, 74240) dir 0x300; Ishad N'ha gate 0 -> entry 2 (10752, 7168, 281600) dir 0x100

Spoke 1 - Crypt (CRYPTA/CRYPTB)

trig	event	action	source	box / switch (world)	src bsp	destination	arrival (world)
T1	@OFFSWITCHDDTOSEGMENT	segment	switch W21 B1	(280320, 1024, 146432)..(289792, 5120, 153600)	1	spoke 0 bsp 0 tcrypt, W1:=0 (back: T8)	(285056, 3072, 150016)
T38	@GOTOBSPONLY	bsp	box B25	(289280, -10248, 224256)..(291840, -8192, 225792)	1	bsp 2 CRYPTB nav 77/495	(290560, -9220, 365312)
T39	@GOTOBSPONLY	bsp	box B26	(289280, -10248, 218112)..(291840, -8192, 219648)	1	bsp 2 CRYPTB nav 77/76	(290560, -9220, 359168)
T40	@GOTOBSPONLY	bsp	box B27	(272384, -7172, 177152)..(273408, -5120, 179712)	1	bsp 2 CRYPTB nav 101/100	(272896, -6146, 318720)
T41	@GOTOBSPONLY	bsp	box B28	(276480, -7172, 317440)..(277504, -5120, 320000)	2	bsp 1 CRYPTA nav 101/83	(276992, -6146, 178432)

Spoke 2 - Serpent Temple (bsp1 TEMPLEB, bsp2 TEMPLEA)

trig	event	action	source	box / switch (world)	src bsp	destination	arrival (world)
T1	@OFFSWITCHDDTOSEGMENT	segment	switch W1 B1	(113920, -2, 322048)..(121088, 113920, -2, 322048)	1	spoke 0 bsp 2 ttmpcave,	(6912, 1535,

				3072, 328704)		W2:=0 (back: T12)	222976)
T2	@OFFSWITCHDDTOSEGMENT	segment	switch W2 B2 via T115	(349696, 1275, 305152).. (360448, 7936, 310272)	2	spoke 0 bsp 3 ttemple, W3:=0 (back: T15)	(48896, 4606, 218624)
T70	@GOTOBSPONLY	bsp	box B33	(324608, 1020, 308224).. (330240, 4096, 311296)	2	bsp 1 TEMPLEB nav 339/340	(179968, 25086, 354816)
T71	@GOTOBSPONLY	bsp	box B34	(187904, 23548, 353280).. (193536, 26624, 356352)	1	bsp 2 TEMPLEA nav 920/969	(338176, 2558, 309760)
T110	@GOTOBSPONLY	bsp	box B39	(264192, -8192, 306176).. (267264, -1024, 309248)	2	bsp 1 TEMPLEB nav 365/374	(118272, 17920, 352768)
T113	@GOTOBSPONLY	bsp	box B42	(281088, -14848, 300544).. (286208, -11264, 304128)	2	bsp 1 TEMPLEB nav 314/376	(136192, 9472, 347392)
T114	@GOTOBSPONLY	bsp	box B43	(123648, 7680, 345600).. (128512, 11264, 349184)	1	bsp 2 TEMPLEA nav 960/972	(273536, -13056, 302336)

Intra-BSP teleports: T155 @IFSTATETELEPORT B69 -> (287616, 6656, 280848); T156 @IFSTATETELEPORT B70 -> (289408, 6656, 279680); T157 @IFSTATETELEPORT B71 -> (291184, 6656, 278528); T158 @IFSTATETELEPORT B72 -> (294544, 6656, 278528); T159 @IFSTATETELEPORT B73 -> (296320, 6656, 279696); T160 @IFSTATETELEPORT B74 -> (298112, 6656, 280848)

Spoke 3 - wilderness (terrain spoke03)

trig	event	action	source	box / switch (world)	src bsp	destination	arrival (world)
T3	@SWITCHDDTOSEGMENT	segment	switch W1 B2 via T1	(140896, 6592, 31744).. (146800, 12842, 38528)	3	spoke 4 bsp 1 SHURU, W1:=1 (back: T116)	(113128, 1525, 110912)
T6	@SWITCHDOORTOSEGMENT	segment	switch W2 B4 via T4	(167088, 304, 49568)..(175536, 6336, 58656)	2	spoke 4 bsp 1 SHURU, W26:=1 (back: T31)	(146736, -4872, 130912)
T18	@SWITCHDOORTOSEGMENT	segment	switch W5 B11 via T16	(85936, -224, 224224).. (109520, 13664, 240096)	6	spoke 7 bsp 1 DRAGONA, W1:=1 (back: T3)	(187840, -3520, 119520)
T21	@SWITCHDDTOSEGMENT	segment	switch W6 B13 via T19	(122864, 18208, 238560).. (127520, 20480, 243504)	7	spoke 7 bsp 2 DRAGONB, W3:=1 (back: T51)	(512264, 17296, 78216)
T30	@SWITCHDDTOSEGMENT	segment	switch W8 B16 via T28,T31	(50176, 6224, 29200)..(60432, 11520, 38672)	10	spoke 5 bsp 1 MINESA, W77:=1 (back: T282)	(131080, -2392, 128144)
T34	@SWITCHDDTOSEGMENT	segment	switch W9 B18 via T32	(268240, 5792, 268464).. (275376, 10976, 275504)	8	spoke 6 bsp 1 OGREA, W16:=1 (back: T107)	(220608, -5952, 143984)
T38	@SWITCHDDTOSEGMENT	segment	switch W11 B21 via T36	(220416, 6912, 278496).. (227104, 10368, 284080)	9	spoke 6 bsp 2 OGREB, W12:=1 (back: T113)	(416272, 5568, 122568)
T66	@ENTERTHETOWN	town	switch W12 via T53	(284672, -20480, 8192).. (291840, 20480, 16384)	-1	town 1 Ishad N'ha	-
T67	@ENTERTHETOWN	town	switch W13 via T54	(1024, -20480, 279552)..(9216, 20480, 287744)	-1	town 2 Brimloch Roon	-

Town gate arrivals: Ishad N'ha gate 1 -> entry 1 (284160, 1090, 10752) dir 0x300; Brimloch Roon gate 0 -> entry 0 (7954, 3072, 284160) dir 0x100

Spoke 4 - Shuru (SHURU)

trig	event	action	source	box / switch (world)	src bsp	destination	arrival (world)
T31	@OFFSWITCHDOORTOSEGMENT	segment	switch W26 B64 via T117	(142512, -7888, 126368).. (150960, -1856, 135456)	1	spoke 3 bsp 2 tshurub, W2:=0 (back: T6)	(171312, 3320, 54112)
T116	@OFFSWITCHDDTOSEGMENT	segment	switch W1 B62 via T1	(110096, -1600, 107520).. (116080, 3536, 113520)	1	spoke 3 bsp 3 tshuruc, W1:=0 (back: T3)	(143808, 9160, 34744)

Spoke 5 - Mines (MINESA..MINESD)

trig	event	action	source	box / switch (world)	src bsp	destination	arrival (world)
T6	@GOTOBSPONLY	bsp	box B6	(180656, -30864, 63312).. (184960, -24960, 67760)	1	bsp 2 MINESB nav 829/830	(387608, -27912, 65536)
T7	@GOTOBSPONLY	bsp	box B7	(385392, -30864, 75520).. (389760, -24960, 80240)	2	bsp 1 MINESA nav 131/601	(182776, -27912, 77880)
T8	@GOTOBSPONLY	bsp	box B8	(177856, -30832, 125248).. (184928, -25408, 129744)	1	bsp 2 MINESB nav 723/832	(386192, -28120, 127496)
T9	@GOTOBSPONLY	bsp	box B9	(369888, -30832, 125248).. (376464, -25408, 129744)	2	bsp 1 MINESA nav 314/315	(168376, -28120, 127496)
T19	@GOTOBSPONLY	bsp	box B12	(462848, -44208, 164864).. (468480, -38912, 169472)	2	bsp 4 MINESD nav 8/259	(506624, -41560, 453888)
T20	@GOTOBSPONLY	bsp	box B13	(493936, -44208, 451584).. (499440, -38912, 456192)	4	bsp 2 MINESB nav 769/770	(455728, -41560, 167168)
T32	@GOTOBSPONLY	bsp	box B25	(370576, -41056, 205312).. (376048, -35664, 210400)	2	bsp 3 MINESC nav 848/847	(199232, -38360, 433136)
T33	@GOTOBSPONLY	bsp	box B26	(206032, -41056, 430592).. (210832, -35664, 435680)	3	bsp 2 MINESB nav 384/385	(382512, -38360, 207856)

T34	@GOTOBSPONLY	bsp	box B27	(326880, -36016, 174624).. (330512, -29920, 178800)	2	bsp 3 MINESC nav 928/929	(154616, -32968, 401992)
T35	@GOTOBSPONLY	bsp	box B28	(161920, -36016, 399904).. (167088, -29920, 404080)	3	bsp 2 MINESB nav 839/840	(338584, -32968, 176712)
T282	@OFFSWITCHDDTOSEGMENT	segment	switch W77 B161 via T280,T283	(125952, -5040, 123408).. (136208, 256, 132880)	1	spoke 3 bsp 10 tmines, W8:=0 (back: T30)	(55304, 8872, 33936)

Spoke 6 - Ogre (OGREA..OGREC)

trig	event	action	source	box / switch (world)	src bsp	destination	arrival (world)
T18	@GOTOBSPONLY	bsp	box B16	(161376, -15744, 197936).. (166208, -11808, 200240)	1	bsp 2 OGREB nav 892/893 (nav id missing)	(320464, -3536, 141744)
T19	@GOTOBSPONLY	bsp	box B17	(317792, -5552, 134784).. (323360, -1856, 136672)	2	bsp 1 OGREA nav 894/897	(163904, -13944, 193072)
T20	@GOTOBSPONLY	bsp	box B18	(152432, -16272, 153936).. (156432, -12656, 155200)	1	bsp 3 OGREC nav 821/911	(154432, -14464, 379848)
T21	@GOTOBSPONLY	bsp	box B19	(152432, -16272, 385408).. (156432, -12656, 386768)	3	bsp 1 OGREA nav 820/917	(154432, -14464, 160808)
T23	@GOTOBSPONLY	bsp	box B21	(154928, -16032, 139104).. (158208, -12352, 140400)	1	bsp 3 OGREC nav 834/908	(156568, -14192, 365032)
T24	@GOTOBSPONLY	bsp	box B22	(154928, -16032, 368832).. (158208, -12352, 370160)	3	bsp 1 OGREA nav 833/915	(156568, -14192, 144216)
T25	@GOTOBSPONLY	bsp	box B23	(223488, -8464, 165664).. (225392, -4560, 169088)	1	bsp 2 OGREB nav 725/733	(381112, 3728, 110032)
T26	@GOTOBSPONLY	bsp	box B24	(373616, 1776, 108320).. (376304, 5680, 111744)	2	bsp 1 OGREA nav 871/870	(218288, -6512, 167376)
T107	@OFFSWITCHDOORTOSEGMENT	segment	switch W16 B56 via T105	(217040, -8544, 140464).. (224176, -3360, 147504)	1	spoke 3 bsp 8 togre, W9:=0 (back: T34)	(271808, 8384, 271984)
T113	@OFFSWITCHDDTOSEGMENT	segment	switch W12 B61 via T111	(413184, 3616, 119824).. (419808, 7536, 125280)	2	spoke 3 bsp 9 togreb, W11:=0 (back: T38)	(223984, 8648, 281272)

Intra-BSP teleports: T63 @IFSTATEPAINTP B33 -> (364928, -2048, 153728); T64 @IFSTATEPAINTP B34 -> (351360, -2048, 158720); T65 @IFSTATEPAINTP B35 -> (363136, -2048, 160384); T66 @IFSTATEPAINTP B36 -> (354432, -2048, 168352); T67 @IFSTATEPAINTP B37 -> (355200, -2048, 150144); T68 @IFSTATEPAINTP B38 -> (362880, -2048, 171648); T69 @IFSTATEPAINTP B39 -> (364416, -2048, 145536); T70 @IFSTATEPAINTP B40 -> (334752, -2048, 145200); T71 @IFSTATEPAINTP B41 -> (371072, -2048, 166272); T72 @IFSTATEPAINTP B42 -> (340352, -2048, 155392); T73 @IFSTATEPAINTP B43 -> (346112, -2048, 146432)

Spoke 7 - Dragon (DRAGONA..DRAGONC)

trig	event	action	source	box / switch (world)	src bsp	destination	arrival (world)
T3	@OFFSWITCHDOORTOSEGMENT	segment	switch W1 B2 via T1	(176048, -10464, 111584).. (199632, 3424, 127456)	1	spoke 3 bsp 6 tdragona, W5:=0 (back: T18)	(97728, 6720, 232160)
T34	@GOTOBSPONLY	bsp	box B34	(477488, -11264, 97488).. (487312, -8160, 100176)	2	bsp 1 DRAGONA nav 83/82	(226400, -9712, 98832)
T35	@GOTOBSPONLY	bsp	box B35	(221840, -11264, 90000).. (230048, -8160, 93840)	1	bsp 2 DRAGONB nav 369/370	(481944, -9712, 91920)
T36	@GOTOBSPONLY	bsp	box B36	(477392, -9040, 98576).. (482608, -5728, 102256)	2	bsp 1 DRAGONA nav 72/73	(224000, -7384, 100416)
T37	@GOTOBSPONLY	bsp	box B37	(221392, -9040, 88448).. (225920, -5728, 93600)	1	bsp 2 DRAGONB nav 364/363	(479656, -7384, 91024)
T40	@GOTOBSPONLY	bsp	box B40	(225360, 3024, 113808).. (227904, 7536, 118912)	1	bsp 2 DRAGONB nav 464/465	(482632, 5280, 116360)
T41	@GOTOBSPONLY	bsp	box B41	(473152, 4448, 112688).. (475936, 9872, 118224)	2	bsp 1 DRAGONA nav 223/222	(218544, 7160, 115456)
T42	@GOTOBSPONLY	bsp	box B42	(277600, 8336, 375824).. (279472, 11952, 380320)	3	bsp 2 DRAGONB nav 323/322	(534536, 10144, 122072)
T43	@GOTOBSPONLY	bsp	box B43	(536784, 7104, 120368).. (538464, 11040, 124944)	2	bsp 3 DRAGONC nav 320/321	(281624, 9072, 378656)
T44	@GOTOBSPONLY	bsp	box B44	(277744, -11072, 376800).. (279456, -5520, 381696)	3	bsp 2 DRAGONB nav 291/292	(534600, -8296, 123248)
T45	@GOTOBSPONLY	bsp	box B45	(537568, -11072, 119808).. (539648, -5520, 124560)	2	bsp 3 DRAGONC nav 294/293	(282608, -8296, 378184)
T51	@OFFSWITCHDOORTOSEGMENT	segment	switch W3 B49 via T49	(509936, 16160, 75744).. (514592, 18432, 80688)	2	spoke 3 bsp 7 tdragonb, W6:=0 (back: T21)	(125192, 19344, 241032)

Spoke 8 - Lich (LICHA/LICHB)

trig	event	action	source	box / switch (world)	src bsp	destination	arrival (world)
T3	@OFFSWITCHDDTOSEGMENT	segment	switch W1 B2 via T1	(106000, -1392, 150944)..(113024, 6080, 157968)	1	spoke 11 bsp 3 tlich, W5:=0 (back: T14)	(267208, 15400, 119640)

T79	@GOTOBSPONLY	bsp	box B45	(101504, -18912, 152496).. (103520, -13792, 153226)	1	bsp 2 LICHB nav 1024/1023	(307312, -16352, 152861)
T197	@GOTOBSPONLY	bsp	box B5	(307808, -17712, 153632).. (308672, -14576, 155648)	2	bsp 1 LICHA nav 1021/1366	(103440, -16144, 154640)

Spoke 9 - Sunken (SUNKENA/SUNKENB)

trig	event	action	source	box / switch (world)	src bsp	destination	arrival (world)
T3	@OFFSWITCHDDTOSEGMENT	segment	switch W1 B2 via T1	(208032, -3584, 144304).. (215472, 480, 151568)	1	spoke 11 bsp 4 tsunkena, W6:=0 (back: T17)	(81704, -26128, 109024)
T24	@GOTOBSPONLY	bsp	box B15	(246352, -1344, 402064).. (247888, 6400, 407536)	2	bsp 1 SUNKENA nav 3/4	(247120, 2528, 200000)
T39	@GOTOBSPONLY	bsp	box B25	(254240, -1344, 196688).. (260768, 6400, 202960)	1	bsp 2 SUNKENB nav 9/8	(257504, 2528, 404624)
T41	@OFFSWITCHDDTOSEGMENT	segment	switch W11 B9 via T18	(253936, -2896, 150560).. (264112, 1824, 161392)	1	spoke 11 bsp 5 tsunkenb, W7:=0 (back: T20)	(111568, -29208, 81224)

Spoke 10 - Shrine (SHRINEA)

trig	event	action	source	box / switch (world)	src bsp	destination	arrival (world)
T3	@OFFSWITCHDOORTOSEGMENT	segment	switch W1 B2 via T1	(216992, -6560, 205792).. (231520, 704, 211216)	1	spoke 11 bsp 1 tshrinea, W1:=0 (back: T3)	(285696, 5264, 257656)
T39	@OFFSWITCHDOORTOSEGMENT	segment	switch W6 B24 via T50	(172432, -27680, 132176).. (182224, -19712, 140384)	1	spoke 11 bsp 0 tshrineb, W2:=0 (back: T6)	(260272, -9360, 242776)
T48	@OFFSWITCHDDTOSEGMENT	segment	switch W7 B32 via T46	(208880, -28000, 28768).. (223104, -19024, 41904)	1	spoke 11 bsp 2 tshrinec, W4:=0 (back: T10)	(207800, -21464, 208392)

Intra-BSP teleports: T144 @IFSTATETELEPORT B52 -> (224256, -9728, 193536)

Spoke 11 - wilderness (terrain spoke11)

trig	event	action	source	box / switch (world)	src bsp	destination	arrival (world)
T3	@SWITCHDOORTOSEGMENT	segment	switch W1 B2 via T1	(278432, 1632, 254944).. (292960, 8896, 260368)	1	spoke 10 bsp 1 SHRINEA, W1:=1 (back: T3)	(224256, -2928, 208504)
T6	@SWITCHDOORTOSEGMENT	segment	switch W2 B4 via T4	(255376, -13344, 238672).. (265168, -5376, 246880)	0	spoke 10 bsp 1 SHRINEA, W6:=1 (back: T39)	(177328, -23696, 136280)
T10	@SWITCHDDTOSEGMENT	segment	switch W4 B6 via T8	(200688, -25952, 201824).. (214912, -16976, 214960)	2	spoke 10 bsp 1 SHRINEA, W7:=1 (back: T48)	(215992, -23512, 35336)
T14	@SWITCHDDTOSEGMENT	segment	switch W5 B9 via T12	(263696, 10784, 116339).. (270720, 18432, 123520)	3	spoke 8 bsp 1 LICHA, W1:=1 (back: T3)	(109512, 1552, 154746)
T17	@SWITCHDDTOSEGMENT	segment	switch W6 B11 via T15	(77664, -28656, 105888).. (84448, -24480, 111136)	4	spoke 9 bsp 1 SUNKENA, W1:=1 (back: T3)	(211104, -1992, 147424)
T20	@SWITCHDDTOSEGMENT	segment	switch W7 B13 via T18	(108272, -31696, 76224).. (114784, -26944, 84896)	5	spoke 9 bsp 1 SUNKENA, W11:=1 (back: T41)	(258984, -648, 155312)
T23	@SWITCHDDTOSEGMENT	segment	switch W8 B15 via T21	(65712, -6656, 332800)..(77408, 512, 344992)	7	spoke 12 bsp 1 PYRAMA, W1:=1 (back: T3)	(169864, -3072, 140240)
T62	@ENTERTHETOWN	town	switch W14 via T57	(382976, -20480, 21504).. (390144, 20480, 29696)	-1	town 2 Brimloch Roon	-

Town gate arrivals: Brimloch Roon gate 1 -> entry 1 (382976, 1024, 25600) dir 0x300

Spoke 12 - Pyramid (PYRAMA/PYRAMB)

trig	event	action	source	box / switch (world)	src bsp	destination	arrival (world)
T3	@OFFSWITCHDDTOSEGMENT	segment	switch W1 B2 via T1	(164048, -6240, 133552).. (176000, 464, 145936)	1	spoke 11 bsp 7 tpyram, W8:=0 (back: T23)	(71720, -2888, 338400)
T64	@GOTOBSPONLY	bsp	box B42	(185344, -21232, 95424).. (188912, -16048, 98352)	1	bsp 2 PYRAMB nav 311/310	(187128, -18640, 322168)
T65	@GOTOBSPONLY	bsp	box B43	(180224, -21232, 325696).. (184848, -16048, 328752)	2	bsp 1 PYRAMA nav 607/309	(182536, -18640, 101944)
T168	@GAMEOVER	endgame	special S5	-	-	end movie 1	-
T169	@GAMEOVER	endgame	state -	-	-	end movie 0	-

Intra-BSP teleports: T89 @EXECTELEPORT B52 -> (171616, -10240, 122368)

8. Recipe: adding an exit from spoke A to a new place in spoke B

No exe change is needed for a new **LOADSEGMENT** exit. Its arrival point is fully defined by data and BSP origins. What cannot change without patching the exe: the 13 spokes, the dungeon BSP lists (LoadSpokeSegment_), the town gate -> spoke/entry

mapping (SetSpokeEntry_) and GetEntryPosition_. The entry table is a code switch, not a data table. An entry point for a new gate, or a new town -> spoke route, needs a patch to that function (or an engine port).

Steps for a LOADSEGMENT exit A -> B (and usually B -> A):

1. **Pick the frames.** Choose source BSP slot a in spoke A and destination slot b in spoke B. The party arrives at `local + origin_B[b]`, where `local` is its position relative to `origin_A[a]` (+256 y if spoke 8 is involved).
 - Either build both pieces with the same local geometry around the exit room, as the shipped game does. On terrain spokes a new overworld piece is just a new TOL 'B' record plus a BSP, which is data only.
 - Or reuse existing BSPs and choose the exit room position in A so that `local + origin_B[b]` lands on valid floor in B. There is no free-space search on arrival.
 - Or let the party arrive anywhere valid in B and move it with a TELEPORT trigger fired by a bound box (who 0/1) at the arrival point. TELEPORT only works inside the BSP that the PC is already in.
 - With `BSPNUM = -1` nothing is added. Terrain-to-terrain exits from outside any BSP keep their world coordinates.
2. **Spoke A tables:**
 - `D6BOUN`: one record, `bsp=a`, `trigger=-1`, `enabled=0`, `min/max = exit room (local to a)`. This is the `IFALLINBOUND` volume.
 - `D6SWIT`: a lever (placed prop `isentity=0`, `objid` per `GetObjNum_`) or a brush entity, with `target = S` (a free `WState` index of spoke A, 1..255) and `on=0`.
 - `D6TRIG`: `@SWITCHDDTOSEGMENT` or `@SWITCHD0ORTOSEGMENT`, `state=S`, `enabled=1`, `mode=0`, `params !STATE=S, DOOR(A/B)=a*100000+d6entid, SPEED, NAVPNT` (nav id that the door blocks/unblocks), `!BOUNDNUM`, `!SWITCH` (the lever), `STATUS=0`, `SEGMENT=B`, `BSPNUM=b`, `SEGSWITCH` (lever in B or -1), `SEGSTATUS=1`, `DELAY ms`, `SFXREC`.
3. **Spoke B tables:** the mirror image with the OFF variant (`SEGSTATUS=0`). Its bound box must contain the arrival point. Its lever is the `SEGSWITCH` of step 2, so arriving opens the doors.
4. **Saves:** visited spokes are restored from `D6SEGnn.GAM / D6ARCHIV.DAT` (triggers, bounds, switches, world state), and that state overrides the pristine tables. Test with a new game, or delete those files.
5. Check the new exit with `d6exits.py GAMEDIR A: "way back"` must find the return trigger.

For an exit **within** a spoke between two BSPs, use a `@GOTOBSPONLY` trigger fired by a bound (who=5, `bsp=src`, `trigger=T`). The two BSPs must overlap at the box in their local frames. Add the opposite box in the destination BSP, and `D6LINK` entries so the path AI can cross.

For a new exit **to a town**, copy the gate pattern from 5.4. The way back from the town always uses the hard-coded gate entries.

`d6exits.list_exits()` / `list_entrances()` give an editor everything it needs to draw these exits: source box or switch in world units, destination spoke/BSP, arrival point and the records involved.

9. Resolved questions (from the code and the data)

- **InBSPArea_** (0x45e250) is reproduced exactly by `d6exits.bsp_at`. Both it and `InBSPOnly_` use only the tile rectangle of each placement (`_bspBounds`, written by `Terrain_BSPLoad_ = d6level tile_rect`), `tiles = pos / 1024` truncated toward zero. Dungeon spokes: the first slot whose rectangle contains the tile. Terrain spokes: `InBSPOnly_` never matches (its loop starts after the terrain BSPs); the game walks a tile-space BSP tree built once per spoke by `TerBSP_Calculate_` (4 splitters per placement, the first splitter nothing straddles, a leaf takes the slot of the last splitter on its path), ported as `d6exits.terrain_leaf_tree`; -1 off the map. For the shipped data (no overlapping rectangles in any spoke) the tree gives the same slot as the rectangle test on every tile of spokes 0, 3 and 11. The 2-tile-grown box at BSP +0x118dc is only used by `InBSPBounds_` (collision, particles, spells).
- **Spoke 8 y shift:** the signs are right; see section 3 for the exact conditions (-256 only for PCs in a BSP; +256 whenever `BSPNUM != -1`).
- **_gSin/_gCos:** 1024 steps per turn of 6.28 (section 5.2), `x = sin`, `z = cos`; the arc side of PCs 1-5 is right. Checked against a save (section 5.2).
- **Dungeon rows of GetEntryPosition_:** its only caller is `AssignPartyCamera_`, called on a fresh spoke entry (after `SetSpokeEntry_`, which only picks spokes 0, 3 and 11) and by the host for a multiplayer client (`ProcMsgPENTRY_`, net message 0x23) using the host's spoke. So the dungeon rows are unreachable in single player; they serve a client joining while the host is in a dungeon.
- **GOTOBSPERR** is `GOTOBSP`: both jump table entries (0x4dfb08) point to the same handler (0x4e083c), which never reads the opcode.
- **!BOUNDNUM / _gSegLoadBounds** (0x5d27f0): written by `InitSpoke_` and `LOADSEGMENT`, never read, saved or sent. Unused.
- **Conditions:** `d6exits` now prints the tests of each exit's event (and of the events that call it) as text: `IFSTATE`, `IFHASITEM`, `IFALLINBOUND`, `IFUSINGITEM`, `IFNOTOCCUPIED`, `IFOCCUPIEDBY`, `IFOBJINBOUND`, `IFSWITCHON`, `IFEQUIPPED`, `IFMONALIVE` (jump ops: `true = jump`) and `SETQFLAGIF` (the only quest flag test). A switch can also require a key item (`switch +0xc`). Event operands are indexes into the trigger's parameters; `d6exits` resolves them. It still follows `CALLEVENT`, `SETSTATE` and `SETSWITCH` chains to depth 4.

Chapter 12. Level geometry: TrenchBroom -> Deep6

Source: docs/geometry.md

The indoor levels (BSPs) of Wizards & Warriors can be edited in [TrenchBroom](#) and compiled back into the game. The SDK pieces:

piece	what it does
tools/compiler/build.sh	builds qbsp + vis (ericw-tools 2.0) with the Deep6 collision hulls into tools/compiler/bin
tools/trenchbroom/Deep6/	TrenchBroom game configuration (GameConfig.cfg, deep6.fgd)
formats/d6bspdc.py	decompiler: level -> .map + textures as PNG
formats/d6bspc.py	compiler driver: .map -> .bsp .twd .lf .ls .lgt .rgb .nvs .l2n, install, TrenchBroom setup
formats/d6map.py	.map reader/writer (Valve 220 and Quake 2 Valve faces)
editor: Tools > Level geometry (Ctrl+Shift+G)	the same steps with buttons

Setting up

```
sudo apt install git cmake build-essential libtbb-dev libembree-dev
tools/compiler/build.sh
python3 formats/d6bspc.py setup-trenchbroom ~/Deep6Maps /path/to/game
```

setup-trenchbroom copies the game configuration to ~/.TrenchBroom/games/Deep6 (Windows: %APPDATA%\TrenchBroom, macOS: ~/Library/Application Support/TrenchBroom) and exports the textures of every level to ~/Deep6Maps/textures/<level>/ (plus _special/ skip, clip, nodraw, hint, water, lava). It also sets TrenchBroom's game path (Preferences > Games > Deep6 (Wizards & Warriors) > Game path) to ~/Deep6Maps in TrenchBroom's Preferences.json; close TrenchBroom before running it, as TrenchBroom rewrites that file on exit. Run it again after updating the SDK: it replaces an older copy of the configuration, which TrenchBroom 2026.2 may refuse to load.

TrenchBroom: the official Linux build is TrenchBroom.AppImage. The editor finds it on the PATH or in ~/Applications, ~/Downloads (also in sub-folders), ~/bin, ~/.local/bin, ~/Desktop or /opt, and marks it executable if needed; otherwise set *TrenchBroom command* in the dialog.

Editing an existing level

```
python3 formats/d6bspdc.py /path/to/game crypta ~/Deep6Maps/maps/crypta.map
trenchbroom ~/Deep6Maps/maps/crypta.map
python3 formats/d6bspc.py compile ~/Deep6Maps/maps/crypta.map --name crypta \
--out ~/Deep6Maps/build --game /path/to/game --install
```

--install copies the result into the game folder; the files it replaces go to <game>/d6edit_backup/<date-time>-<level>/ (and stale .lfs/.lss, which the compiler does not write, are moved there). The objects (.bol) and the scripting tables are not touched. When <level>.nvs exists in the game, its nav points are kept (events and D6LINKnn.DAT use their ids) and only the leaf table (.l2n) is rebuilt; --newnav makes a new graph instead.

Decompiled maps start with // Game: Deep6 (Wizards & Warriors), so TrenchBroom opens them with the Deep6 configuration without asking. The brushes that are only solid space around the level (every face _special/nodraw) are put in the layer *Outer hull (nodraw)*, hidden and locked, so they do not box in the view or catch clicks; the compiler treats layer and group brushes (func_group) as world brushes. TrenchBroom's camera starts at the origin: Ctrl+A, Ctrl+U (select all, focus) brings the level into view, Esc clears the selection.

The decompiler turns every solid leaf of the BSP into a brush and splits brushes where the texture or its alignment changes, so the look survives the round trip (checked on CRYPTA: 4457 of 4463 matched faces have the same texture and alignment; walked through it in the game). The brushes are the compiler's split pieces, not the designers' brushes: expect a few thousand small brushes per level.

Making a new level

TrenchBroom: File > New, game Deep6, starts from tools/trenchbroom/Deep6/initial.map: a sealed 768 x 768 x 256 room with CRYPTA textures, one light and a player start, which compiles as it is.

- **Units:** map units = BSP units. 1 BSP unit = 16 game world units, a terrain tile is 64 units, the shipped corridors are 128-256 wide. Texture scale 0.5 gives the density of the shipped levels.
- **Axes:** map z is up. The game's y is up and its x/z plane is mirrored; the compiler swaps y and z, so the level looks the same in the game as in TrenchBroom.
- **Collision:** the party is a box 64 wide and 6 high (hull 1); keep gaps that should be walked through at least 64 wide.
- **Water / lava:** set the content flag *water* or *lava* on a brush (face attributes). The faces are drawn with the animated water/lava texture.
- **Surface flags:** *nodraw* (face kept, not drawn), *terrain* (horizontal face drawn as terrain, used by the overworld pieces), *waterpass*, *translucent*. Content flag *clip*: collision only.
- **Textures:** any PNG under the project's textures folder; textures of the shipped levels are copied with their palettes, other images are resized to 128x128 and quantised (8 per new palette).
- **Lights:** light entities with d6light (radius in BSP units, shipped values 180-300) and _color. Light at a point = d6ambient + the sum over visible lights of d6lightpeak * (1 - d/radius) * (0.4 + 0.6 cos), capped at 30; both worldspawn keys default to the fit to the retail lightmaps (3.5 and 9.6). Raise them for a brighter level. The vertex light and the object light per leaf use ambient + 12.4 and peak x 0.43, like the brighter, flatter retail vertex light. The compiler bakes lightmaps (32 unit luxels) with shadows, vertex light and the object light per leaf, and writes the lights to .lgt/.rgb, which the game loads as

light objects.

- **Doors:** func_door brush entities (d6type 1 slide / 2 rotate, d6axis, d6maxmove, d6entid, d6string). Events address them as bsp slot * 100000 + d6entid.
- Put one info_player_start (or any light) inside the level: qbsp uses it to find the outside and throw away the faces nobody can see. A leak is reported with a .pts file (TrenchBroom: File > Load Point File).

Placing it in the world

- **Terrain spokes 0, 3, 11:** a level is placed by a 'B' record of SPOKEnn.TOL (d6level.ObjectList.add_bsp_placement, or the editor's *Place new level in this spoke*): map origin at a terrain tile, height in steps of 64 map units. A spoke has at most 16 BSP slots (spoke 0 uses 14, spoke 3 all 16, spoke 11 nine). Verified in the game (spoke 0).
- **Dungeon spokes:** their level list is fixed in deep6.exe; compile over one of their levels (same name). New entries need engine changes (planned for OpenDWWandWExpanded).
- Getting there: exits and teleports (Exits panel, event scripts) and nav links (D6LINKnn.DAT, nav tool) as for any level.

What the compiler does

1. reads the map, gives every texture a short alias (qbsp keeps 15 characters), marks water/lava/clip/skip brushes, writes a Valve 220 map without the lights (lights near walls make the 64 unit clip hulls "leak")
2. runs qbsp (-tjunc rotate) and vis; only a leak in hull 0 counts
3. converts Quake BSP 29 into Deep6 BSP 28: y/z swapped, face windings reversed and rotated so they end on a corner (the engine does not draw a face whose last vertex is not a corner), texinfo rescaled to the Deep6 projection, 68 byte models, entities with d6* keys
4. light: lightmaps (.lfs/.lss), vertex light, leaf object light
5. textures (.twd), lights (.lgt/.rgb), nav graph (.nvs/.l2n)

Engine limits checked: 19999 faces, 9999 nodes, lightmaps of at most 16x16 luxels (larger faces are not drawn; the compiler warns).

Known limits

- The software-renderer lightmaps (.lfs/.lss) are not written.
- Light is an approximation of the original tool: fitted to 14279 retail luxels of six levels it is off by 5.8 of 30 on average (RMS); the retail maps have soft detail this model does not reproduce.
- Rotating doors: the pivot keys d6rotx/y/z are in game coordinates (y up), as in the retail data.
- Checked with TrenchBroom 2026.2 (Linux Applmage): the game configuration loads; a decompiled level opens with its textures, lights and doors; in CRYPTA a wall was retextured from the material browser, a water volume made (water texture + the water content flag checkbox), a light recoloured and a new light dragged in from the entity browser (it carries only an origin; the compiler uses the FGD defaults); compiled and installed from the editor, all of it shows in the game. File > New gives the template room, which compiles. Not checked in TrenchBroom itself: lava and clip flags (same path as water), TrenchBroom on Windows/macOS.

Appendix A. deep6.exe modules

Source: docs/exe_modules.md

The game modules of deep6.exe (Watcom debug info: 209 source files; the Watcom run-time library is left out). *Functions / globals* are the named symbols of each module. *Cited in* lists the SDK documents that mention the module, i.e. where its behaviour is described. Roles are only given where the SDK has looked at the code.

module	functions	globals	role	cited in
_alpha.ASM	1	6		
_edge.ASM	1	0		
_font.ASM	4	0		
_math.ASM	3	0		
_modelp.ASM	2	6		
_mpoly.ASM	40	53		
_pixclr.ASM	3	0		
_span.ASM	9	0		
_swc.ASM	5	0		
_tpoly.ASM	7	0		
_trans.ASM	3	16		
_zbuf.ASM	3	0		
alpha.c	8	2		
anim.c	12	16		
animtex.c	4	0		
assets.c	25	0		formats/models.md
assert.c	1	0		
atmos.c	7	17		
atqlist.c	17	12		formats/models.md
audioc.c	67	13	audio device	formats/audio.md
automap.c	38	5	automap drawing	
avi.c	18	13		
bentname.c	1	1		
BinkVid.c	3	2		
bitmap.c	8	4		
blast.ASM	16	1		
blt.c	0	1		
bmp16.c	2	0		
body.c	11	32		
bsp.c	22	22	BSP drawing	
bsp_phys.c	17	28		
bspcache.c	2	2		
bspfile.c	39	13	.bsp/.lfl/.ls loading	formats/levels.md
bspmodel.c	15	22		
bspobj.c	7	2		
bsptex.ASM	5	0		
camera.c	17	0	camera	
canvas.c	17	0		
cardctrl.c	97	49	Direct3D renderer control	formats/walls_textures.md
cardlite.c	8	7	coloured lights (.lgt/.rgb)	formats/levels.md
cardutil.c	71	7		
cdprot.c	1	0		formats/terrain.md
chardemo.c	43	42		formats/databases.md
cheatkey.cpp	1	1		
checksum.c	1	0		
cmd.c	76	5		
collide.c	21	16	collision	
collisio.c	16	25		
color.c	10	0		
combat.c	135	23	combat, recall/portal effects	formats/audio.md, formats/databases.md, formats/exits.md, formats/models.md
compass.c	3	1		
config.c	58	53	options book	
conlex.c	3	4		
d3dtimer.c	8	9		
d6alloc.c	6	1		
d6glob.c	0	209		
d6iosys.c	21	15		
d6pc.c	29	15		
d6spoke.c	151	155	spoke loading (LoadSpokeSegment_),	formats/data.md, formats/exits.md, formats/levels.md,

			entry points, spoke change	formats/terrain.md
d6string.c	3	63	D6STRING.DAT	formats/data.md, formats/effects.md
d6win95.c	2	1		
d_span.c	4	49		formats/levels.md
deep6.c	53	37	start-up, globals, data loading, town/spoke switch (SetSpokeEntry_)	formats/data.md, formats/databases.md, formats/exits.md, formats/terrain.md, formats/walls_textures.md
dialog.c	23	26		
dirfind.c	4	3		
dpcon.c	29	3		
drawhull.c	6	3		
dynlight.c	12	1		
efx.c	8	6	visual effects (EFX)	
efxcache.c	12	6		
enchants.c	5	0		
entity.c	5	6		
events.c	73	48	triggers, bound areas, switches, specials, event VM (EVENTS.COD)	formats/data.md, formats/databases.md
fileutil.c	2	0		
font.c	24	8	fonts (.FNT/.p16)	
fountain.c	3	1		formats/databases.md
fpufuck.ASM	3	0		
frate.c	3	7		
frustum.c	4	2	view frustum	
genenc.c	25	26		formats/databases.md
globals.c	0	41		
graphobj.c	26	1		formats/models.md
guild.c	53	11	guildmaster VM (GMDATA.PAK)	formats/audio.md, formats/data.md, formats/npc.md
indmovie.c	10	1		
jentry.c	29	11		formats/audio.md, formats/databases.md
jhp_part.c	21	12		formats/effects.md
jhpmap.c	51	10	automap (.lm/.fog/.mrk)	formats/databases.md, formats/terrain.md
journal.c	14	11	journal (JOURNAL.nnn)	
langutil.c	4	0		
levelmap.c	54	9		
llist.c	11	1		
lmouse.c	2	1	mouse pointers (.ptr)	
logfile.c	2	1		
mapobj.c	28	17	debug level viewer (FetchObjects_)	formats/data.md, formats/levels.md
mathlib.c	10	0		
mlddata.c	0	5	model name tables	formats/databases.md, formats/models.md
menu.c	79	6	title and in-game menus	
minegrup.c	5	3		
miptest.c	11	1		
missile.c	13	2		
model.c	38	45	.mdl loading and drawing	formats/data.md, formats/databases.md, formats/models.md
monant.c	6	3		formats/databases.md
monster.c	119	48	monsters (D6MONS)	formats/audio.md, formats/data.md, formats/databases.md, formats/models.md
mouse.c	7	12		
mousetim.c	20	36		
MP3Play.c	11	6		
MP3Temp.c	1	0		
mpconn.c	41	13		
mpoly.c	8	5		formats/models.md
msgreq.c	16	1		
mshadow.c	7	4		
myscan.ASM	8	0		
nav_sys.c	8	0	nav graph (.nvs/.l2n/.NAV)	formats/levels.md
netcon.c	2	9		
network.c	332	52	multiplayer messages	
newcam.c	21	26		
newclip.c	2	4		
npc.c	85	53	NPC dialogue VM (NPCDATA.PAK)	formats/audio.md, formats/data.md, formats/databases.md, formats/models.md, formats/npc.md
obj_phys.c	20	18		
objext.c	6	0		
olist.c	7	1		
pakfile.c	3	0		
pal16.c	17	6		formats/models.md, formats/walls_textures.md
pal2048c.c	11	7		

pal2k.c	2	0		
palette.c	3	0		
palmap.c	12	3		
palremap.ASM	2	0		
part_sys.c	10	7		
particle.c	44	14	particle emitters (emitters.dat)	formats/data.md
pathai.c	37	14	path finding, D6LINK	formats/data.md
pccreate.c	130	80	character creation	
pcinvent.c	67	49	inventory screen	formats/databases.md
pcmodel.c	38	30	PC models, PC snapshots (PCSNAP.nnn)	
pcmounts.c	4	1		
pcorders.c	18	3		formats/databases.md
pcskill.c	11	7		
pcspell.c	8	11		
pcstats.c	6	3		
pcstatus.c	10	8		
pcsttext.c	8	12		
pctalk.c	7	1		
pixcolor.c	8	36		
playsam.c	7	10	music	formats/audio.md
pmodel.c	3	0		
pointlgt.c	14	5		
poly.c	10	2		
ppdrawl.c	6	5		
propefx.c	12	1		
pushy.c	12	12		formats/databases.md
rain.c	13	11		
random.ASM	2	0		formats/walls_textures.md
rayterr.c	4	12		formats/walls_textures.md
rgb16.c	3	0		
rotoobj.c	36	16		
rpal.c	10	7		
runtime.c	15	21		
scenload.c	29	28	object lists (.TOL/.BOL/.FOL), object ids	formats/data.md, formats/databases.md, formats/levels.md, formats/models.md, formats/walls_textures.md
screen.c	11	19	DirectDraw screen	
scrncap.c	1	0		
segwrite.c	33	4	saves: D6SEGnn.GAM, D6ARCHIV.DAT	formats/data.md, formats/exits.md
sfxcache.c	22	5	sound cache	formats/audio.md
shadowm.c	8	0		
shopball.c	17	17		
smartcam.c	11	12		
smcoll.c	18	1		
sndread.c	4	3		formats/databases.md
soundefx.c	8	13	sound effects, speech, PC talk	formats/audio.md
spelleff.c	254	69	spell effects	
spells.c	1	2		
sprite.c	5	0		
sr_game.c	21	12		
stars.c	11	12		
sub25616.ASM	2	0		
swcache.c	13	11		
terbsp.c	15	15	terrain BSP leaf tree (InBSPArea_)	
terrain.c	37	33	terrain loading and textures	formats/walls_textures.md
texlist.c	10	8	.twd texture wads	formats/levels.md, formats/walls_textures.md
textmsg.c	31	44	TEXTPAK messages	formats/data.md
texture.c	7	3		
tfast.c	2	2		
thirdeye.c	13	37		
tnew.c	11	11	terrain tile drawing, walls, canopy	formats/walls_textures.md
townavi.c	22	21	town hub videos	
towndojo.c	32	4	dojo	
townhall.c	27	2	town hall	
townhub.c	11	11	town hub screen, gates, Gareth	formats/exits.md, formats/terrain.md
townmage.c	50	7	mage guild	
townpawn.c	32	4	pawn shop	
townsmit.c	100	17	smithy shop	formats/data.md, formats/databases.md
towntmpl.c	64	8	temple	
towntvrn.c	11	3	tavern	

townyard.c	19	5	ship yard	
tpoly.c	16	15		
tpoly2.c	28	15		formats/walls_textures.md
tracklen.cpp	8	1		
trange.c	4	0		
trans.c	8	0		
traps.c	21	11	D6TRAP locks and traps	formats/data.md, formats/databases.md, formats/models.md
treasure.c	15	5	treasure (D6TREAS/D6TRLIST)	formats/databases.md
trig.c	1	2		
vector.c	36	0		formats/models.md
virtualc.c	2	1		
warp.c	4	7		
wbutton.c	59	1		
wclock.c	4	7		
wcoll.c	16	11		formats/databases.md
winmain.c	7	1		
wlight.c	5	4		

Appendix B. In-game verification log

Source: docs/verification.md

What has been checked by running the game (OpenDWWandW on Linux, the "wide test" save, a single level 1 warrior) and what is still open. The format docs carry the details; this page is the overview.

Checked in the game

feature	how	result
Model writer (641 models rebuilt), glTF/OBJ import	rebuilt models, imported trees	render and animate
Terrain sculpt, paint, walls, snow types	edited spoke 0	as edited
BSP compiler (TrenchBroom map -> .bsp, .lfl/.ls, .nvs)	test level in place of crypta	geometry, collision, door, lightmaps
BSP decompile -> recompile	CRYPTA round trip	looks like the original
New BSP level placed through the TOL	spoke 0, with a water pool	enters, water works
NPC script assembler (d6npc)	Traveler: new string, inserted SAY, edited reply branch, GIVEGOLD	as written, \$N expanded
NPC speaker name	monster record renamed through text export / import	message bar shows the new name
Journal (NPC lines)	Characters > Traveler page	edited lines listed
NPC MODPC SPELL+	Traveler teaches spell 20	"Red learns ... spell!"
Spell names (D6STRING 9000 + id - 1)	renamed Burn	new name in the learn message
NPC speech file naming	wav for a new line, file opens traced	opened on SAY
Skinned glTF import	rig in place of the Traveler's model	renders, walk animation plays
MONSOUND.DAT limit raised, new id	id 700 in a D6MONSND slot	wav opened when played
Record field assignment (rec.field = v)	d6data	now writes the field (was silently ignored)
Mod workflow (d6mod)	linked working copy of the full game, played under trace; Traveler script + spell edit captured, installed into a second copy, uninstalled	game-written files found and excluded; pristine game unchanged; install equals the working copy; uninstall restores
Automap redraw (d6automap)	spoke 0 map tab, against the original	aligned; style differs on BSP areas
Guildmaster script (GMDATA.PAK)	Sir Elgar's script disassembled, line edited, reassembled; entered through the Valeia gate and Town Hall (OpenDWWandWExpanded)	edited line spoken; Leave button (ACTBUTTON 2 -> EXIT) leaves the hall
Spoke change with BSP rebase (d6exits)	spoke 8 exit T3 (lever W1, party in box B2) to spoke 11	PC moved by (157696, 13056, -34816) = origin 11/3 - origin 8/1 - (0, 256, 0), exactly as documented
TrenchBroom 2026.2 editing	CRYPTA start corridor: wall retextured in the material browser, water volume (texture + content flag), light recoloured, new light from the entity browser; compile + install	skull wall, water surface and red light seen in the game (before/after screenshots); File > New template compiles
TrenchBroom 2026.2 round trip	editor: set up project, decompile TCRYPT, open in TrenchBroom (Applmage), duplicate a roof beam 512 units up, save in TrenchBroom, compile + install from the editor	configuration loads (after fixes), map opens with the right game, compiled level contains the beam, game loads it; the beam itself not seen in the game (night, forest)
Party entry placement trig (d6exits)	save position against the formula	z 83453.9609375 matches the exe's sin/cos(i*6.28/1024) tables (2 pi gives 83456)

Checked against the retail data

approximation	check	result
Lightmap extents (smin, tmin, w, h)	all faces of crypta and minesb from the retail BSPs	21743/21743 equal
Light model	retail BSP + lights relit, every luxel against the retail .ls	RMS error 5.7 (crypta) and 4.0 (minesb, not used for the fit) of 30; old defaults 16 and 18
Vertex light / leaf object light	same, against the retail BSP lumps	own fit (ambient + 12.4, peak x 0.43): vertex RMS 7.1 / 8.5 (was 10.2), leaf light 4.4 / 7.1
InBSPArea_ / spoke BSP lookup in d6exits	terrain leaf tree (TerBSP_Calculate_) ported; 39000 random positions (terrain and dungeon spokes) against the exe rules	0 mismatches

Not yet checked in the game

- Skinned glTF from a real Blender export.
- Spell table numbers (mana, levels, recovery: needs a caster; the game maps the patched exe, so the bytes arrive). The spellbook pages need magic skill.
- Hearing sounds/music (the test setup has no audio device; file opens can be traced).
- emitters.dat / .ant / .alf edits (the files are loaded at start, traced; the look was not checked: no torch or candle near the test position).
- Guildmaster buttons other than Leave (news, bank, shop results).
- Rotating door pivot keys.

Notes for testing

- Do not move the PC by writing its position (0x628380) while it is on the terrain: the object grid lists are not updated and

the game later loops forever in the path code (FindNearestObstruction_, GatherAmmolnRadius_). Loading a spoke after poking the position (_gSegLoadSeg) can hang the same way. Safe ways: walking (held arrow keys), or editing the save: the PC position is stored twice in game00.sav (x, y, z floats, the second with y + 832; search for the current x), and the game snaps y to the ground on load. (game00.sav starts like D6ARCHIV.DAT, version 0x140, but its header fields differ; not decoded.)

- DW_BIN="strace -f -e trace=openat -o FILE .../opendwawdw" dwstart.sh shows which data files the game opens.
- NPCs approach on their own when their script says so; the Traveler starts talking when the PC reaches the road east of the start (about x 241000, z 70000).